

Departamento de Informática

Manual de Introducción a la Programación Web con JavaScript

Departamento de Lenguajes y Sistemas Informáticos

J.G. Gutiérrez

Marzo 2026

Este documento se encuentra bajo una licencia Creative Commons de Atribución-CompartirIgual (CC BY-SA).

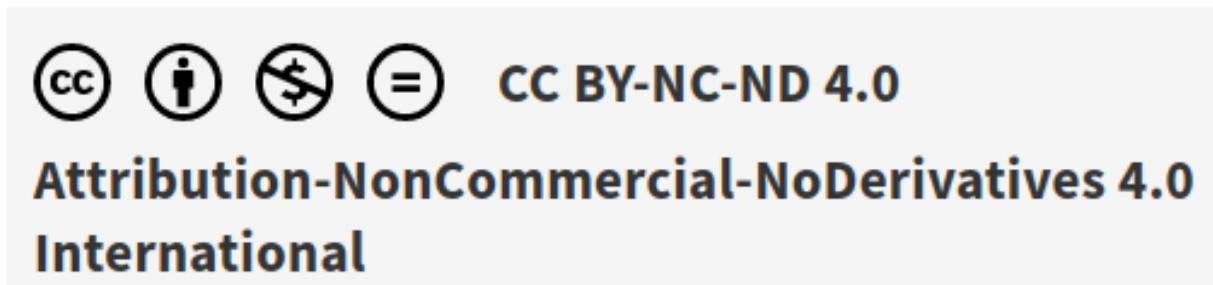


Figura 1: Atribución-CompartirIgual (CC BY-SA)

Esto significa que puedes:

- Compartir: copiar y redistribuir el material en cualquier medio o formato.
- Adaptar: remezclar, transformar y construir sobre el material para cualquier propósito, incluso comercialmente.

Bajo las siguientes condiciones:

- Atribución: debes dar crédito de manera adecuada, proporcionar un enlace a la licencia e indicar si se han realizado cambios. Puedes hacerlo de cualquier manera razonable, pero no de una manera que sugiera que el licenciante te respalda a ti o al uso que hagas del trabajo.
- Compartir igual: si remezclas, transformas o creas a partir del material, debes distribuir tus contribuciones bajo la misma licencia que el original.

Para más detalles, consulta la licencia completa.

¡Atención, futuros maestros del DOM!

Antes de que te lances a programar tu **Buscaminas**, tengo que darte un aviso importante: estos apuntes son como la “electricidad” de nuestra casa web. Si no los usas pronto, podrías quedarte a oscuras.

AVISO DE OBSOLESCENCIA PROGRAMADA

¡Hola, aspirante a programador! Estás ante un manual de **JavaScript moderno**, cargado de módulos, funciones flecha y manipulación del DOM. Pero recuerda: en el mundo del desarrollo web, lo “moderno” de hoy es el “retro” de mañana.

JavaScript fue creado en **1995** y, desde entonces, no ha parado de mutar. Con la llegada de **ECMAScript 2025**, el lenguaje sigue evolucionando más rápido que una partida de Buscaminas en nivel difícil. Por eso, estos apuntes tienen una **fecha de caducidad estimada de 3 años**.

¿Por qué 3 años? * **A los 2 años:** Algunas funciones que hoy nos parecen “lo último” empezarán a parecerse a las paredes viejas de nuestra estructura HTML. * **A los 3 años:** Si intentas leer este manual en el 2028 sin haberlo tocado antes, ¡podría darte un error de sintaxis en el cerebro! Es probable que para entonces ya estemos usando telepatía para manejar el `localStorage`. * **Más de 3 años:** Corres el riesgo de que tu código parezca escrito por **Brendan Eich** en una tarde de café en 1995.

Mi consejo: ¡Estudia ahora! No dejes que tus conocimientos caduquen. Si esperas demasiado, cuando intentes ejecutar tu lógica de juego, te sentirás como si acabaras de **pisar una mina en la primera casilla**.

¡Aprovecha que el **motor de JavaScript** de tu navegador todavía entiende este manual y manos a la obra!

Autores:

- Juan Gualberto Gutiérrez Marín, juangu@ujaen.es
-

Índice

1	Introducción a la programación con JavaScript	10
1.1	Introducción	10
1.2	¿Qué es exactamente JavaScript?	10
1.3	¿Dónde se ejecuta JavaScript?	11
1.4	El DOM: cómo ve el navegador una página web	12
1.5	Primer ejemplo: un botón interactivo	12
1.6	Analizando el código paso a paso	13
1.6.1	Seleccionar un elemento del DOM	13
1.6.2	Escuchar un evento	14
1.6.3	Ejecutar una acción	14
1.7	Qué hemos aprendido	14
1.8	Resultado práctico	14
2	Cómo incluir JavaScript correctamente	16
2.1	Crear la estructura del proyecto	16
2.2	Conectar JavaScript con el documento HTML	17
2.3	Crear el archivo main.js	17
2.4	Comprobar que el script se ejecuta	17
2.5	Crear una función de inicio de la aplicación	18
2.6	Qué hemos conseguido en este capítulo	18
3	Sintaxis básica de JavaScript	20
3.1	Variables	20
3.1.1	Usando <code>let</code>	20
3.1.2	Usando <code>const</code>	21
3.2	Tipos de datos básicos	21
3.2.1	Números	21
3.2.2	Texto (strings)	21
3.2.3	Booleanos	22
3.3	Arrays	22
3.4	Objetos	22
3.5	Condicionales	23
3.6	Bucles	23
3.7	Funciones	24
3.8	Funciones flecha (arrow functions)	24
3.9	Probar código desde main.js	25

3.10	Qué hemos aprendido en este capítulo	25
4	El DOM: cómo interactuar con el HTML desde JavaScript	27
4.1	Cómo representa el navegador una página web	27
4.2	Seleccionar elementos del DOM	28
4.2.1	Seleccionar por clase	28
4.2.2	Seleccionar varios elementos	28
4.3	Leer y modificar contenido	28
4.4	Modificar clases CSS	29
4.5	Crear elementos desde JavaScript	29
4.6	Ejemplo práctico: crear elementos dinámicamente	30
4.7	Añadir eventos a elementos	30
4.8	Qué papel tendrá el DOM en el juego Buscaminas	31
4.9	Qué hemos aprendido en este capítulo	31
5	Generando el tablero	33
5.1	El contenedor del tablero	33
5.2	Representar el tablero como una matriz	33
5.3	Crear el módulo del tablero	34
5.4	Crear una función para generar el tablero	34
5.5	Seleccionar el contenedor del tablero	34
5.6	Limpiar el tablero	35
5.7	Crear las celdas con bucles	35
5.8	Crear cada celda	35
5.9	Añadir la celda al tablero	35
5.10	Importar la función en main.js	35
5.11	Qué ocurre cuando ejecutamos el código	36
5.12	Preparar las celdas para futuros eventos	36
5.13	Código completo del módulo board.js	36
5.14	Qué hemos conseguido en este capítulo	37
6	Detectar clics en las celdas del tablero	38
6.1	Eventos en el navegador	38
6.2	Añadir un evento a una celda	38
6.3	Saber qué celda ha sido pulsada	39
6.4	Convertir los valores a números	39
6.5	Separar la lógica en una función	40
6.6	Código actualizado de board.js	40
6.7	Qué ocurre cuando el jugador pulsa una celda	41

6.8	Qué hemos conseguido en este capítulo	41
7	Matrices	43
7.1	El tablero visible y el tablero interno	43
7.2	Qué es una matriz en JavaScript	43
7.3	Crear una función para generar la matriz vacía	44
7.4	Cómo funciona esta función	45
7.5	Probar la matriz desde main.js	45
7.6	Qué significará cada valor de la matriz	46
7.7	Colocar minas aleatoriamente	46
7.8	Cómo funciona <code>Math.random()</code>	47
7.9	Evitar que dos minas caigan en la misma celda	47
7.10	Probar la colocación de minas	47
7.11	Guardar la matriz para usarla más adelante	48
7.12	Código completo actualizado de board.js	49
7.13	Qué hemos conseguido en este capítulo	50
8	Pensamiento computacional: calculando los números para las celdas	51
8.1	Calcular los números de cada celda	51
8.2	Qué significa “celdas vecinas”	51
8.3	Qué debemos calcular	52
8.4	Crear una función para contar minas vecinas	52
8.5	Cómo funciona esta función	53
8.6	Comprobar los límites de la matriz	53
8.7	Contar minas	54
8.8	Calcular todos los números del tablero	54
8.9	Probar el cálculo desde main.js	54
8.10	Qué aspecto tendrá la matriz ahora	55
8.11	Por qué todavía no se muestran los números en pantalla	56
8.12	Código completo actualizado de board.js	56
8.13	Qué hemos conseguido en este capítulo	58
9	Abrir celdas y mostrar el contenido	60
9.1	Qué debe ocurrir al pulsar una celda	60
9.2	Necesitamos acceder a la matriz desde el tablero	60
9.3	Modificar <code>crearTablero</code> para recibir una función	61
9.4	Crear una función para abrir una celda	62
9.5	Llamar a <code>abrirCelda</code> desde <code>crearTablero</code>	62
9.6	Localizar visualmente la celda pulsada	63

9.7	Mostrar el contenido de la celda	63
9.8	Evitar abrir dos veces la misma celda	64
9.9	Actualizar iniciarAplicacion	64
9.10	Qué conseguimos ahora	66
9.11	Código completo actualizado de board.js	66
9.12	Qué hemos conseguido en este capítulo	68
10	Destapando el vacío	70
10.1	Qué significa expandir una zona vacía	70
10.2	El problema de la repetición	70
10.3	Idea general de la solución	71
10.4	Crear una función para abrir vecinas	71
10.5	Recorrer las ocho vecinas	72
10.6	Por qué esta función no entra en bucle infinito	73
10.7	Código completo de abrirCelda con expansión	73
10.8	Qué ocurre ahora cuando el jugador pulsa una celda vacía	74
10.9	Resultado práctico	74
10.10	Código completo actualizado de main.js	75
10.11	Qué hemos conseguido en este capítulo	76
11	Eventos de ratón	78
11.1	Qué evento se produce al hacer clic derecho	78
11.2	Añadir el evento de clic derecho a cada celda	78
11.3	Código actualizado de crearTablero	79
11.4	Crear una función para marcar banderas	80
11.5	Evitar abrir una celda con bandera	81
11.6	Actualizar la llamada a crearTablero	82
11.7	Comportamiento esperado	82
11.8	Código completo actualizado de main.js	82
11.9	Código completo actualizado de board.js	85
11.10	Qué hemos conseguido en este capítulo	87
12	Game Over	88
12.1	Necesitamos saber si la partida sigue activa	88
12.2	Reiniciar el estado al empezar una nueva partida	88
12.3	Bloquear acciones si la partida ha terminado	89
12.4	Detectar derrota	89
12.5	Mostrar todas las minas al perder	90
12.6	Detectar victoria	91

12.7	Crear una función para comprobar victoria	91
12.8	Comprobar la victoria después de abrir una celda	92
12.9	Reorganizar abrirCelda para comprobar la victoria	92
12.10	Bloquear banderas cuando la partida termina	94
12.11	Qué ocurre ahora en una partida	94
12.11.1	Si el jugador pulsa una mina	94
12.11.2	Si el jugador abre todas las celdas seguras	95
12.12	Código completo actualizado de main.js	95
12.13	Qué hemos conseguido en este capítulo	98
13	Timers	100
13.1	El panel superior del juego	100
13.2	Qué es un temporizador en JavaScript	100
13.3	Variables para controlar el tiempo	101
13.4	Mostrar el tiempo en pantalla	101
13.5	Iniciar el temporizador	102
13.6	Detener el temporizador	102
13.7	Reiniciar el reloj	103
13.8	Poner en marcha el temporizador al iniciar la partida	103
13.9	Detener el reloj al perder	104
13.10	Detener el reloj al ganar	104
13.11	Añadir el botón de reinicio	105
13.12	Cuándo debemos llamar a configurarBotonReinicio	105
13.13	Evitar múltiples listeners en el botón	105
13.14	Resultado práctico	106
13.15	Código completo actualizado de main.js	106
13.16	Qué hemos conseguido en este capítulo	110
14	Añadir el contador de minas y la selección de dificultad	112
14.1	Qué dificultades tendrá el juego	112
14.2	Crear un objeto de configuración	112
14.3	Guardar la dificultad actual	113
14.4	Modificar iniciarAplicacion para usar la dificultad elegida	113
14.5	Mostrar el número de minas en pantalla	114
14.6	Actualizar el título de la partida	115
14.7	Detectar los clics del menú de dificultad	115
14.8	Crear una función para configurar el menú	115
14.9	Qué significa usar dataset aquí	116

14.10	Configurar también el botón de inicio fácil	116
14.11	Cuándo llamar a configurarMenuDificultad	117
14.12	Qué ocurre ahora cuando el usuario pulsa una dificultad	117
14.13	Código completo actualizado de main.js	118
14.14	Qué hemos conseguido en este capítulo	124
15	Navegación entre paneles de la aplicación	125
15.1	Qué entendemos por panel	125
15.2	La idea general de la navegación	126
15.3	Cómo ocultar y mostrar paneles	126
15.4	Crear una función para mostrar un panel	127
15.5	Cómo funciona querySelectorAll(".panel")	127
15.6	Mostrar el panel inicial al cargar la aplicación	128
15.7	Crear una función para configurar la navegación	128
15.8	Qué ocurre con los botones de dificultad	129
15.9	Qué pasa con el temporizador al salir del panel de partida	129
15.10	Configurar la navegación al arrancar	130
15.11	Qué comportamiento tendrá ahora la aplicación	130
15.12	Código completo actualizado de main.js	131
15.13	Qué hemos conseguido en este capítulo	138
16	Detener la partida al salir del panel de juego	139
16.1	Qué entendemos por “salir de la partida”	139
16.2	Necesitamos recordar qué panel está activo	140
16.3	Crear una función para abandonar la partida	140
16.4	Modificar mostrarPanel para detectar la salida del juego	141
16.5	Qué hace esta nueva versión	142
16.6	Por qué esta solución es sencilla y didáctica	142
16.7	Qué ocurre si el usuario cambia de dificultad	143
16.8	Qué ocurre si el usuario pulsa Inicio o Ayuda en mitad de una partida	143
16.9	Ajustar el arranque inicial	143
16.10	Código completo actualizado de main.js	144
16.11	Qué hemos conseguido en este capítulo	151
17	LocalStorage	152
17.1	Qué es localStorage	152
17.2	Qué información guardaremos en cada puntuación	153
17.3	Crear el módulo scores.js	153
17.4	Guardar y recuperar arrays con JSON	153

17.5	Crear una función para obtener las puntuaciones guardadas	154
17.6	Crear una función para guardar una nueva puntuación	154
17.7	Limitar el número de puntuaciones guardadas	155
17.8	Crear una función para borrar las puntuaciones	155
17.9	Generar el HTML de la tabla de puntuaciones	156
17.10	Código completo de <code>scores.js</code>	157
17.11	Importar el módulo de puntuaciones en <code>main.js</code>	158
17.12	Crear una función para calcular los puntos	159
17.13	Guardar la puntuación al ganar	159
17.14	Mostrar el diálogo para guardar la puntuación	160
17.15	Guardar realmente la puntuación	161
17.16	Crear una función para refrescar el panel de puntuaciones	162
17.17	Configurar el botón de borrar puntuaciones	162
17.18	Actualizar las puntuaciones al navegar al panel correspondiente	163
17.19	Registrar las nuevas configuraciones al arrancar	163
17.20	Código completo actualizado de <code>main.js</code>	164
17.21	Qué hemos conseguido en este capítulo	173
18	Retoques finales	174
18.1	El problema del arranque actual	174
18.2	Introducir la idea de “no hay partida iniciada”	175
18.3	Preparar la interfaz inicial	175
18.4	Iniciar partida solo cuando corresponda	176
18.5	Ajustar el botón de reinicio	176
18.6	Mejorar el abandono de partida	177
18.7	Ajustar la navegación para que no rompa el flujo	177
18.8	Mejorar la experiencia visual de las celdas	178
18.9	Añadir estilos mínimos recomendados	178
18.10	Ajustar dinámicamente el tamaño del tablero	180
18.11	Refinar el flujo de arranque definitivo	181
18.12	Código final de <code>main.js</code>	182
18.13	Código final ajustado de <code>board.js</code>	191
18.14	Qué hemos conseguido al finalizar el tutorial	194

1 Introducción a la programación con JavaScript

El propósito de este manual es la realización un Buscaminas utilizando JavaScript moderno con módulos.

1.1 Introducción

Cuando visitamos una página web moderna, no solo vemos información estática. Las páginas actuales reaccionan a nuestras acciones: podemos pulsar botones, enviar formularios, abrir menús, jugar a juegos o interactuar con aplicaciones completas.

Todo este comportamiento interactivo es posible gracias a **JavaScript**.

JavaScript es el lenguaje de programación que permite añadir **lógica y comportamiento** a una página web.

En el desarrollo web moderno existen tres tecnologías fundamentales:

Tecnología	Función
HTML	Define la estructura de la página
CSS	Define el diseño y la apariencia
JavaScript	Define el comportamiento y la interacción

Podemos imaginar una página web como una casa:

- **HTML** sería la estructura: paredes, puertas, habitaciones.
- **CSS** sería la decoración: colores, muebles, iluminación.
- **JavaScript** sería la electricidad y los mecanismos: interruptores, puertas automáticas, sensores, etc.

Sin JavaScript, la mayoría de las páginas web serían simplemente documentos estáticos.

1.2 ¿Qué es exactamente JavaScript?

JavaScript es un **lenguaje de programación interpretado** diseñado inicialmente para ejecutarse dentro del navegador web.

Fue creado en **1995 por Brendan Eich** mientras trabajaba en Netscape. El objetivo era permitir que las páginas web pudieran responder a las acciones del usuario sin tener que recargar completamente la página.

Hoy en día JavaScript se ha convertido en uno de los lenguajes más utilizados del mundo.

Con JavaScript podemos:

- Detectar cuando el usuario hace clic en un botón
- Validar formularios
- Modificar el contenido de una página sin recargarla
- Crear animaciones
- Construir aplicaciones web completas
- Desarrollar juegos en el navegador

En este tema aprenderemos JavaScript utilizando un ejemplo práctico: **el desarrollo de un juego de Buscaminas en el navegador**.

1.3 ¿Dónde se ejecuta JavaScript?

En este módulo trabajaremos con JavaScript que se ejecuta en el **navegador web**.

Los navegadores modernos incluyen un **motor de JavaScript** que es capaz de interpretar y ejecutar el código.

Cada navegador tiene su propio motor:

Navegador	Motor de JavaScript
Chrome	V8
Edge	V8
Firefox	SpiderMonkey
Safari	JavaScriptCore

Cuando abrimos una página web ocurre el siguiente proceso:

1. El navegador descarga el archivo **HTML**
2. Construye la estructura interna de la página
3. Carga los archivos **CSS**
4. Ejecuta los scripts de **JavaScript**

JavaScript puede entonces:

- Leer el contenido de la página
- Modificar elementos
- responder a eventos del usuario

Este modelo de ejecución es lo que hace posible que una página web sea **interactiva**.

1.4 El DOM: cómo ve el navegador una página web

Para poder manipular una página web, el navegador necesita representar el HTML internamente.

Esta representación se llama **DOM (Document Object Model)**.

El DOM es una estructura en forma de árbol donde cada elemento del HTML se convierte en un objeto que JavaScript puede manipular.

Por ejemplo, si tenemos este HTML:

```
1 <body>
2   <h1>Título</h1>
3   <button>Pulsar</button>
4 </body>
```

El navegador lo interpreta como una estructura de objetos:

```
1 Document └─
2   body └─
3     h1 └─
4     button
```

JavaScript puede acceder a esos elementos y modificarlos.

Por ejemplo, podemos cambiar el texto de un botón, ocultar un elemento o añadir nuevos elementos a la página.

El DOM es uno de los conceptos más importantes en programación web, ya que es el puente entre **JavaScript y el contenido HTML**.

1.5 Primer ejemplo: un botón interactivo

Vamos a crear un primer ejemplo sencillo para entender cómo JavaScript interactúa con el HTML.

Creemos un archivo `index.html` con el siguiente contenido:

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Primer ejemplo JavaScript</title>
6 </head>
7
8 <body>
9
10 <h1>Primer ejemplo con JavaScript</h1>
11
12 <button id="boton">Pulsa aquí</button>
13
14 <script>
15
16 const boton = document.querySelector("#boton");
17
18 boton.addEventListener("click", () => {
19   alert("Hola mundo desde JavaScript");
20 });
21
22 </script>
23
24 </body>
25 </html>
```

Cuando el usuario pulsa el botón ocurre lo siguiente:

1. JavaScript localiza el botón en el DOM
2. Escucha el evento **click**
3. Cuando se produce el clic, ejecuta una función
4. La función muestra un mensaje en pantalla

1.6 Analizando el código paso a paso

Veamos las partes más importantes del código.

1.6.1 Seleccionar un elemento del DOM

```
1 const boton = document.querySelector("#boton");
```

Aquí estamos utilizando `document.querySelector()` para buscar un elemento en la página.

El símbolo `#` indica que estamos buscando un **id**.

En este caso seleccionamos el botón cuyo id es `boton`.

1.6.2 Escuchar un evento

```
1 boton.addEventListener("click", () => {
```

Los navegadores generan **eventos** cuando el usuario interactúa con la página.

Algunos ejemplos de eventos son:

- `click`
- `keydown`
- `mouseover`
- `submit`

En este caso estamos diciendo:

Cuando el usuario haga clic en el botón, ejecuta esta función.

1.6.3 Ejecutar una acción

```
1 alert("Hola mundo desde JavaScript");
```

La función `alert()` muestra una ventana emergente con un mensaje.

Aunque no se usa mucho en aplicaciones reales, es muy útil para aprender cómo funciona JavaScript.

1.7 Qué hemos aprendido

En este primer capítulo hemos visto los conceptos fundamentales:

- Qué es JavaScript
- Para qué se utiliza en una página web
- Dónde se ejecuta
- Qué es el DOM
- Cómo JavaScript puede reaccionar a las acciones del usuario

También hemos creado nuestro **primer programa interactivo**.

1.8 Resultado práctico

Después de este capítulo ya somos capaces de:

- Crear un archivo HTML
- Incluir código JavaScript
- Seleccionar elementos del DOM
- Detectar eventos del usuario
- Ejecutar acciones cuando ocurre un evento

Estos conceptos serán la base para comenzar a construir el juego del **Buscaminas** paso a paso.

En el siguiente capítulo aprenderemos cómo incluir JavaScript correctamente utilizando **módulos**, que es la forma moderna de organizar proyectos JavaScript.

2 Cómo incluir JavaScript correctamente

En el capítulo anterior hemos visto qué es JavaScript y cómo puede interactuar con una página web. Ahora vamos a dar el siguiente paso: **preparar la estructura real del proyecto que utilizaremos para construir el juego del Buscaminas.**

En este capítulo vamos a crear la base del proyecto y a conectar nuestro primer archivo JavaScript con la página web.

El objetivo es que, al finalizar este apartado, tengamos un proyecto funcionando donde el navegador ya es capaz de ejecutar nuestro código JavaScript.

Este será el punto de partida sobre el que iremos construyendo el juego completo.

2.1 Crear la estructura del proyecto

Antes de empezar a programar es importante organizar bien los archivos. Una buena estructura facilita la lectura del código y hace que el proyecto sea más fácil de mantener.

Vamos a crear una carpeta para nuestro proyecto llamada:

```
1  buscaminasJS
```

Dentro de esta carpeta crearemos la siguiente estructura:

```
1  buscaminasJS|├─  
2  
3  index.html|├─  
4  
5  css/|├─  
6    estilos.css|├─  
7  
8  js/|├─  
9    main.js|├─  
10   board.js|├─  
11   scores.js
```

Cada uno de estos archivos tendrá una función concreta dentro del proyecto:

- **index.html** será la página principal de la aplicación.
- **estilos.css** contendrá el diseño de la interfaz.
- **main.js** será el punto de entrada del programa.
- **board.js** contendrá la lógica del tablero del juego.
- **scores.js** se encargará de gestionar las puntuaciones.

De momento solo vamos a trabajar con `index.html` y `main.js`. Los demás archivos los iremos utilizando más adelante.

2.2 Conectar JavaScript con el documento HTML

Para que el navegador ejecute nuestro código JavaScript debemos incluir un script en el archivo `index.html`.

Abrimos el archivo `index.html` y dentro de la sección `<head>` añadimos la siguiente línea:

```
1 <script type="module" src="js/main.js"></script>
```

El atributo `type="module"` indica al navegador que el archivo se interpretará como un **módulo de JavaScript**.

Esto tiene varias ventajas importantes:

- Permite dividir el código en varios archivos.
- Cada archivo tiene su propio ámbito de variables.
- El navegador carga el script sin bloquear la página.
- Podemos usar `import` y `export` entre archivos JavaScript.

En proyectos modernos de JavaScript esta es la forma recomendada de organizar el código.

2.3 Crear el archivo main.js

Ahora vamos a crear el archivo principal de JavaScript.

Dentro de la carpeta `js` creamos un archivo llamado:

```
1 main.js
```

Este archivo será el **punto de entrada del programa**, es decir, el primer código JavaScript que se ejecutará cuando se cargue la página.

De momento añadiremos un pequeño mensaje para comprobar que todo funciona correctamente.

```
1 console.log("Buscaminas iniciado");
```

2.4 Comprobar que el script se ejecuta

Guardamos los archivos y abrimos `index.html` en el navegador.

Para ver el resultado debemos abrir la **consola del navegador**.

En la mayoría de navegadores se puede abrir con la tecla:

```
1 F12
```

Después seleccionamos la pestaña **Console**.

Si todo está configurado correctamente veremos el mensaje:

```
1 Buscaminas iniciado
```

Esto significa que:

- el navegador ha cargado el archivo `main.js`
- el script se ha ejecutado correctamente
- el proyecto está listo para empezar a programarse

2.5 Crear una función de inicio de la aplicación

En proyectos reales es habitual que el programa comience dentro de una función que inicializa la aplicación.

Vamos a modificar ligeramente el contenido de `main.js`:

```
1 function iniciarAplicacion() {  
2   console.log("Aplicación Buscaminas cargada correctamente");  
3 }  
4  
5 iniciarAplicacion();
```

Con este pequeño cambio estamos preparando la estructura que utilizaremos más adelante.

En esta función de inicio iremos añadiendo progresivamente:

- la creación del tablero
- los eventos del juego
- la inicialización de los marcadores
- la lógica de la partida

2.6 Qué hemos conseguido en este capítulo

En este apartado hemos preparado la base del proyecto del Buscaminas.

Ahora tenemos:

- una estructura de carpetas organizada
- un archivo HTML conectado con JavaScript
- un archivo `main.js` que actúa como punto de entrada del programa
- un proyecto que ya se ejecuta correctamente en el navegador

Aunque el programa todavía no hace nada visible en la página, ya hemos preparado la infraestructura necesaria para empezar a construir el juego.

En el siguiente capítulo comenzaremos a trabajar con **la sintaxis básica de JavaScript**, que nos permitirá empezar a escribir la lógica del juego.

3 Sintaxis básica de JavaScript

En los capítulos anteriores hemos preparado la estructura del proyecto y hemos conseguido ejecutar nuestro primer archivo JavaScript desde el navegador. A partir de ahora comenzaremos a aprender los elementos básicos del lenguaje que necesitaremos para programar el juego del Buscaminas.

No vamos a estudiar todo JavaScript en profundidad, sino **las herramientas necesarias para construir nuestro proyecto**. A medida que avancemos iremos utilizando estas herramientas en el desarrollo del juego.

En este capítulo veremos:

- cómo declarar variables
- los tipos de datos más comunes
- arrays y objetos
- estructuras de control
- funciones

Todos estos elementos forman la base de cualquier programa JavaScript.

3.1 Variables

Una variable es un espacio en memoria donde podemos guardar información.

En JavaScript moderno utilizamos principalmente dos palabras clave para declarar variables:

- `let`
- `const`

3.1.1 Usando `let`

La palabra clave `let` se utiliza cuando el valor de la variable puede cambiar.

```
1 let minas = 10;  
2 let tiempo = 0;
```

En este ejemplo:

- `minas` guarda el número de minas de la partida
- `tiempo` guarda los segundos que han pasado desde que empezó el juego

Posteriormente podemos modificar su valor:

```
1 tiempo = tiempo + 1;
```

3.1.2 Usando const

La palabra clave **const** se utiliza cuando el valor no va a cambiar.

```
1 const nombreJuego = "Buscaminas";
```

Una vez definida, esta variable no puede modificarse.

En general, es buena práctica utilizar **const** siempre que sea posible y reservar **let** únicamente para valores que deban cambiar.

3.2 Tipos de datos básicos

JavaScript permite trabajar con diferentes tipos de datos.

Los más utilizados en nuestro proyecto serán:

Tipo	Ejemplo
Número	10, 3.14
Texto	"Hola"
Booleano	true, false
Array	[1, 2, 3]
Objeto	{x:1, y:2}

3.2.1 Números

```
1 let filas = 8;  
2 let columnas = 8;  
3 let minas = 10;
```

Estos valores nos servirán más adelante para definir el tamaño del tablero.

3.2.2 Texto (strings)

Los textos se escriben entre comillas.

```
1 let mensaje = "Juego iniciado";
```

Podemos mostrarlos en la consola:

```
1 console.log(mensaje);
```

3.2.3 Booleanos

Un booleano solo puede tener dos valores:

- **true**
- **false**

```
1 let partidaTerminada = false;
```

Este tipo de variable es muy útil para controlar el estado del juego.

3.3 Arrays

Un **array** es una lista de valores.

Se utilizan mucho cuando necesitamos trabajar con colecciones de datos.

Por ejemplo:

```
1 let numeros = [1,2,3,4,5];
```

Podemos acceder a cada elemento utilizando su posición.

```
1 console.log(numeros[0]);
```

La numeración siempre empieza en **0**.

Los arrays serán muy importantes en nuestro proyecto, ya que el tablero del buscaminas se representará mediante una **matriz**, que no es más que un array de arrays.

3.4 Objetos

Un objeto permite agrupar información relacionada.

Por ejemplo:

```
1 const jugador = {  
2   nombre: "Ana",  
3   puntuacion: 1200  
4 };
```

Podemos acceder a sus propiedades así:

```
1 console.log(jugador.nombre);
```

Los objetos son muy útiles cuando necesitamos almacenar información estructurada.

3.5 Condicionales

Los condicionales permiten ejecutar código solo cuando se cumple una condición.

La estructura más utilizada es **if**.

```
1 let minas = 10;
2
3 if (minas > 0) {
4   console.log("Todavía quedan minas");
5 }
```

También podemos añadir una alternativa con **else**.

```
1 if (minas > 0) {
2   console.log("La partida continúa");
3 } else {
4   console.log("No quedan minas");
5 }
```

Los condicionales serán fundamentales para comprobar situaciones como:

- si el jugador ha pisado una mina
- si ha ganado la partida
- si puede colocar una bandera

3.6 Bucles

Los bucles permiten repetir una operación varias veces.

Uno de los más utilizados es el bucle **for**.

```
1 for (let i = 0; i < 5; i++) {
2   console.log(i);
3 }
```

Este bucle imprimirá en la consola:

```
1 0
2 1
3 2
4 3
```


5 4

Los bucles serán esenciales cuando tengamos que:

- recorrer las filas del tablero
- recorrer las columnas
- comprobar las celdas vecinas

3.7 Funciones

Una función es un bloque de código que realiza una tarea concreta.

Podemos definir una función así:

```
1 function saludar() {  
2   console.log("Hola");  
3 }
```

Para ejecutarla simplemente la llamamos:

```
1 saludar();
```

Las funciones también pueden recibir parámetros.

```
1 function sumar(a, b) {  
2   return a + b;  
3 }
```

Y utilizarse de esta forma:

```
1 let resultado = sumar(2,3);  
2 console.log(resultado);
```

Las funciones serán muy importantes en nuestro proyecto, porque nos permitirán dividir el programa en pequeñas tareas, por ejemplo:

- crear el tablero
- colocar minas
- abrir una celda
- comprobar si el jugador ha ganado

3.8 Funciones flecha (arrow functions)

JavaScript moderno permite escribir funciones de forma más compacta utilizando **arrow functions**.

Por ejemplo:

```
1  const mostrarMensaje = () => {  
2    console.log("Hola desde una función flecha");  
3  };
```

Estas funciones se utilizan mucho cuando trabajamos con eventos del navegador.

3.9 Probar código desde main.js

Podemos probar algunos de estos conceptos modificando temporalmente nuestro archivo `main.js`.

Por ejemplo:

```
1  function iniciarAplicacion() {  
2  
3    const filas = 8;  
4    const columnas = 8;  
5  
6    console.log("Tablero:", filas, "x", columnas);  
7  
8  }  
9  
10 iniciarAplicacion();
```

Si abrimos la consola del navegador veremos:

```
1  Tablero: 8 x 8
```

Esto confirma que nuestro código JavaScript se está ejecutando correctamente.

3.10 Qué hemos aprendido en este capítulo

En este capítulo hemos visto los elementos básicos del lenguaje JavaScript:

- variables
- tipos de datos
- arrays
- objetos
- condicionales
- bucles
- funciones

Estos conceptos serán suficientes para comenzar a programar la lógica del juego.

En el siguiente capítulo empezaremos a trabajar con el **DOM**, que nos permitirá interactuar con los elementos HTML y comenzar a construir el tablero del Buscaminas dinámicamente desde JavaScript.

4 El DOM: cómo interactuar con el HTML desde JavaScript

En los capítulos anteriores hemos preparado la estructura del proyecto y hemos visto los conceptos básicos del lenguaje JavaScript. Ahora vamos a aprender cómo **JavaScript puede interactuar con los elementos de una página web**.

Cuando el navegador carga un documento HTML, lo transforma en una estructura interna que JavaScript puede manipular. Esta estructura se llama **DOM**.

DOM significa **Document Object Model**.

Gracias al DOM, JavaScript puede:

- acceder a elementos de la página
- modificar su contenido
- cambiar estilos o clases
- crear nuevos elementos
- eliminar elementos existentes

Este concepto será fundamental para construir el juego del Buscaminas, ya que el tablero del juego se generará dinámicamente desde JavaScript.

4.1 Cómo representa el navegador una página web

Cuando el navegador lee el archivo HTML, crea una estructura en forma de árbol.

Por ejemplo, si tenemos el siguiente código HTML:

```
1 <body>
2   <h1>Buscaminas</h1>
3   <button id="boton">Jugar</button>
4 </body>
```

El navegador lo representa internamente así:

```
1 Document└─
2   body└─
3     h1└─
4     button
```

Cada uno de estos elementos se convierte en un **objeto JavaScript** que podemos manipular desde nuestro código.

4.2 Seleccionar elementos del DOM

Para poder trabajar con un elemento primero debemos **seleccionarlo**.

JavaScript proporciona varios métodos para hacerlo.

El más utilizado en JavaScript moderno es:

```
1 document.querySelector()
```

Este método permite buscar un elemento utilizando selectores similares a los de CSS.

Por ejemplo, si queremos seleccionar el botón con id `carita` de nuestro proyecto, podemos escribir:

```
1 const boton = document.querySelector("#carita");
```

El símbolo `#` indica que estamos buscando un **id**.

4.2.1 Seleccionar por clase

Si queremos seleccionar un elemento por su clase:

```
1 const panel = document.querySelector(".panel");
```

El símbolo `.` indica una **clase**.

4.2.2 Seleccionar varios elementos

Si queremos seleccionar varios elementos podemos usar:

```
1 document.querySelectorAll()
```

Por ejemplo:

```
1 const botones = document.querySelectorAll("button");
```

Esto devuelve una lista de todos los botones de la página.

4.3 Leer y modificar contenido

Una vez que tenemos un elemento del DOM podemos acceder a su contenido.

Por ejemplo:

```
1 const titulo = document.querySelector("h1");  
2  
3 console.log(titulo.textContent);
```

La propiedad `textContent` permite **leer o modificar el texto de un elemento**.

También podemos cambiar el texto:

```
1 titulo.textContent = "Buscaminas iniciado";
```

Esto modifica directamente el contenido del HTML.

4.4 Modificar clases CSS

JavaScript también puede modificar las clases de un elemento.

Para ello utilizamos `classList`.

```
1 const panel = document.querySelector("#panel_inicio");  
2  
3 panel.classList.add("activo");
```

Esto añade una clase CSS.

También podemos eliminar clases:

```
1 panel.classList.remove("activo");
```

O alternarlas:

```
1 panel.classList.toggle("activo");
```

Este mecanismo será muy útil cuando queramos **mostrar u ocultar paneles de la aplicación**.

4.5 Crear elementos desde JavaScript

El DOM también permite crear elementos nuevos.

Esto se hace mediante:

```
1 document.createElement()
```

Por ejemplo:

```
1 const div = document.createElement("div");
```

Después podemos modificar su contenido:

```
1 div.textContent = "Nueva celda";
```

Y finalmente añadirlo a la página:

```
1 document.body.appendChild(div);
```

Este proceso será clave cuando generemos **las celdas del tablero del buscaminas**.

4.6 Ejemplo práctico: crear elementos dinámicamente

Podemos probar un ejemplo sencillo en nuestro archivo `main.js`.

Modificamos temporalmente el contenido del archivo:

```
1 function iniciarAplicacion() {  
2  
3     const contenedor = document.createElement("div");  
4  
5     contenedor.textContent = "La aplicación se ha iniciado correctamente"  
6     ;  
7     document.body.appendChild(contenedor);  
8  
9 }  
10  
11 iniciarAplicacion();
```

Cuando recarguemos la página veremos que JavaScript ha creado un nuevo elemento en el documento.

Esto demuestra que el código puede **modificar la estructura del HTML mientras la página está funcionando**.

4.7 Añadir eventos a elementos

Otra de las funciones más importantes del DOM es detectar acciones del usuario.

Estas acciones se llaman **eventos**.

Algunos ejemplos de eventos son:

- `click`
- `dblclick`
- `keydown`
- `mouseover`

Podemos reaccionar a estos eventos mediante:

```
1 addEventListener()
```

Por ejemplo:

```
1 const boton = document.querySelector("#carita");
2
3 boton.addEventListener("click", () => {
4   console.log("Botón pulsado");
5 });
```

En este ejemplo estamos diciendo:

Quando el usuario haga clic en el botón, ejecuta esta función.

Este mecanismo será fundamental para el funcionamiento del juego.

Cada vez que el jugador pulse una celda del tablero se generará un evento `click`.

4.8 Qué papel tendrá el DOM en el juego Buscaminas

El DOM será el encargado de conectar el código JavaScript con la interfaz visual del juego.

JavaScript se encargará de:

- generar el tablero
- actualizar el contador de minas
- actualizar el reloj
- mostrar las celdas abiertas
- mostrar las banderas
- mostrar las minas cuando termina la partida

En otras palabras, el DOM será el puente entre **la lógica del juego** y **lo que ve el usuario en la pantalla**.

4.9 Qué hemos aprendido en este capítulo

En este capítulo hemos aprendido:

- qué es el DOM
- cómo el navegador representa el HTML
- cómo seleccionar elementos desde JavaScript

- cómo modificar contenido
- cómo modificar clases CSS
- cómo crear elementos dinámicamente
- cómo reaccionar a eventos del usuario

Estos conceptos nos permitirán empezar a construir partes reales del juego.

En el siguiente capítulo comenzaremos a generar **el tablero del Buscaminas dinámicamente desde JavaScript**, creando las primeras celdas del juego.

5 Generando el tablero

En los capítulos anteriores hemos aprendido a trabajar con el DOM y a manipular elementos HTML desde JavaScript. Ahora vamos a utilizar estos conocimientos para crear la primera parte visible del juego: **el tablero del Buscaminas**.

El tablero es la zona donde el jugador interactúa con el juego. Está formado por una cuadrícula de celdas que el jugador irá abriendo durante la partida.

En lugar de escribir todas las celdas directamente en el HTML, vamos a **generarlas dinámicamente desde JavaScript**. Esta es la forma habitual de trabajar en aplicaciones web modernas.

5.1 El contenedor del tablero

En el archivo `index.html` ya existe un elemento que servirá como contenedor del tablero.

```
html id="board_container"<div id="tablero" class="board"></div>
```

Este elemento está inicialmente vacío. Nuestro código JavaScript se encargará de rellenarlo con las celdas del juego.

Cada celda del tablero será un elemento HTML que crearemos desde JavaScript.

5.2 Representar el tablero como una matriz

Internamente, el tablero del buscaminas se representa como una **matriz**.

Una matriz es simplemente un **array de arrays**.

Por ejemplo, un tablero de 3x3 podría representarse así:

```
1 [
2   [0,0,1],
3   [1,2,1],
4   [0,1,0]
5 ]
```

Cada posición de la matriz representa una celda del tablero.

Más adelante almacenaremos en esta matriz información como:

- si la celda contiene una mina
- cuántas minas hay alrededor
- si la celda está abierta o cerrada

Pero por ahora solo necesitamos generar la estructura visual del tablero.

5.3 Crear el módulo del tablero

Para mantener el código organizado vamos a crear un nuevo archivo dentro de la carpeta `js`.

Creemos el archivo:

```
1 js/board.js
```

Este archivo se encargará de todo lo relacionado con el tablero del juego.

5.4 Crear una función para generar el tablero

Dentro de `board.js` vamos a escribir una función que cree el tablero.

```
1 export function crearTablero(filas, columnas) {
2
3   const tablero = document.querySelector("#tablero");
4
5   tablero.innerHTML = "";
6
7   for (let fila = 0; fila < filas; fila++) {
8
9     for (let column = 0; column < columnas; column++) {
10
11       const celda = document.createElement("div");
12
13       celda.classList.add("celda");
14
15       tablero.appendChild(celda);
16
17     }
18
19   }
20
21 }
```

Vamos a analizar qué hace esta función.

5.5 Seleccionar el contenedor del tablero

Primero seleccionamos el elemento donde se colocarán las celdas.

```
1 const tablero = document.querySelector("#tablero");
```

Esto nos permite manipular el contenedor desde JavaScript.

5.6 Limpiar el tablero

Antes de crear nuevas celdas eliminamos el contenido anterior.

```
1 tablero.innerHTML = "";
```

Esto será útil cuando el jugador reinicie la partida.

5.7 Crear las celdas con bucles

Utilizamos dos bucles **for** para recorrer filas y columnas.

```
1 for (let fila = 0; fila < filas; fila++) {  
2   for (let columna = 0; columna < columnas; columna++) {
```

Este tipo de bucle anidado es muy común cuando trabajamos con matrices.

5.8 Crear cada celda

Dentro del bucle creamos una celda nueva.

```
1 const celda = document.createElement("div");
```

Después le añadimos una clase CSS.

```
1 celda.classList.add("celda");
```

Esta clase permitirá aplicar estilos visuales.

5.9 Añadir la celda al tablero

Finalmente añadimos la celda al contenedor.

```
1 tablero.appendChild(celda);
```

Este proceso se repite tantas veces como celdas tenga el tablero.

5.10 Importar la función en main.js

Ahora debemos utilizar esta función desde el archivo principal del proyecto.

Abrimos `main.js` y añadimos la importación del módulo.

```
1 import { crearTablero } from "../board.js";
```

Después modificamos la función de inicio de la aplicación.

```
1 function iniciarAplicacion() {  
2  
3     crearTablero(8, 8);  
4  
5 }  
6  
7 iniciarAplicacion();
```

Esto generará un tablero de **8 filas por 8 columnas**.

5.11 Qué ocurre cuando ejecutamos el código

Cuando el navegador carga la página ocurre lo siguiente:

1. Se ejecuta `main.js`.
2. `main.js` llama a la función `crearTablero`.
3. `crearTablero` crea las celdas del tablero.
4. Las celdas se añaden al contenedor HTML.

El resultado es una cuadrícula de celdas que forman el tablero del juego.

5.12 Preparar las celdas para futuros eventos

Más adelante necesitaremos saber qué celda ha pulsado el jugador.

Para ello podemos añadir información a cada celda utilizando atributos `data`.

Modificamos ligeramente el código:

```
1 celda.dataset.fila = fila;  
2 celda.dataset.columna = columna;
```

Esto permite guardar la posición de cada celda.

Más adelante podremos leer estos valores para saber qué celda ha pulsado el jugador.

5.13 Código completo del módulo `board.js`

El archivo `board.js` debería quedar así:

```
1 export function crearTablero(filas, columnas) {
2
3   const tablero = document.querySelector("#tablero");
4
5   tablero.innerHTML = "";
6
7   for (let fila = 0; fila < filas; fila++) {
8
9     for (let columna = 0; columna < columnas; columna++) {
10
11       const celda = document.createElement("div");
12
13       celda.classList.add("celda");
14
15       celda.dataset.fila = fila;
16       celda.dataset.columna = columna;
17
18       tablero.appendChild(celda);
19
20     }
21
22   }
23
24 }
```

5.14 Qué hemos conseguido en este capítulo

En este capítulo hemos construido la primera parte real del juego:

- hemos creado el módulo del tablero
- hemos generado dinámicamente las celdas del juego
- hemos aprendido a usar bucles para crear una cuadrícula
- hemos conectado el módulo con `main.js`

Ahora el proyecto ya es capaz de **crear el tablero del Buscaminas automáticamente**.

En el siguiente capítulo empezaremos a añadir **interacción al tablero**, detectando cuándo el jugador hace clic en una celda.

6 Detectar clics en las celdas del tablero

En el capítulo anterior hemos conseguido generar el tablero del juego desde JavaScript. Cada celda del tablero se crea dinámicamente y se añade al contenedor del tablero en el DOM.

El siguiente paso para avanzar en el desarrollo del Buscaminas es permitir que el jugador **interactúe con las celdas**.

Para ello necesitamos detectar cuándo el usuario hace clic sobre una celda del tablero.

En este capítulo aprenderemos a:

- detectar eventos de clic
- saber qué celda ha sido pulsada
- utilizar los atributos `dataset`
- conectar la interacción del usuario con la lógica del juego

Esto permitirá que cada celda del tablero pueda reaccionar a las acciones del jugador.

6.1 Eventos en el navegador

Cuando el usuario interactúa con una página web, el navegador genera **eventos**.

Algunos ejemplos de eventos son:

Evento	Descripción
<code>click</code>	El usuario pulsa con el ratón
<code>dblclick</code>	Doble clic
<code>keydown</code>	Se pulsa una tecla
<code>mouseover</code>	El ratón pasa por encima

JavaScript puede **escuchar estos eventos** y ejecutar código cuando ocurren.

Para ello utilizamos el método:

```
1 addEventListener()
```

6.2 Añadir un evento a una celda

Cada celda del tablero es un elemento HTML que hemos creado con JavaScript.

Podemos añadir un evento a cada celda justo después de crearla.

Volvemos al archivo `board.js` y añadimos el siguiente código dentro del bucle donde se crean las celdas.

```
1 celda.addEventListener("click", () => {  
2  
3   console.log("Celda pulsada");  
4  
5 });
```

Con este cambio, cada vez que el usuario haga clic en una celda aparecerá un mensaje en la consola.

6.3 Saber qué celda ha sido pulsada

Para poder construir la lógica del juego necesitamos saber **qué celda concreta ha pulsado el jugador**.

En el capítulo anterior guardamos la posición de cada celda utilizando `dataset`.

```
1 celda.dataset.fila = fila;  
2 celda.dataset.columna = columna;
```

Esto significa que cada celda contiene información sobre su posición dentro del tablero.

Podemos acceder a estos valores dentro del evento.

```
1 celda.addEventListener("click", () => {  
2  
3   const fila = celda.dataset.fila;  
4   const columna = celda.dataset.columna;  
5  
6   console.log("Celda:", fila, columna);  
7  
8 });
```

Cuando el jugador pulse una celda veremos en la consola sus coordenadas.

6.4 Convertir los valores a números

Los valores almacenados en `dataset` se guardan como texto.

Si queremos utilizarlos como números es recomendable convertirlos.

```
1 const fila = Number(celda.dataset.fila);  
2 const columna = Number(celda.dataset.columna);
```

Esto nos permitirá utilizar estos valores en cálculos posteriormente.

6.5 Separar la lógica en una función

Para mantener el código organizado es mejor que el evento llame a una función específica.

En lugar de escribir toda la lógica dentro del `addEventListener`, podemos hacer lo siguiente:

```
1 celda.addEventListener("click", () => {
2
3   const fila = Number(celda.dataset.fila);
4   const columna = Number(celda.dataset.columna);
5
6   manejarClickCelda(fila, columna);
7
8 });
```

Ahora creamos la función `manejarClickCelda`.

```
1 function manejarClickCelda(fila, columna) {
2
3   console.log("Celda pulsada:", fila, columna);
4
5 }
```

Más adelante esta función se encargará de:

- abrir la celda
- comprobar si contiene una mina
- mostrar el número de minas cercanas

6.6 Código actualizado de board.js

El archivo `board.js` ahora debería tener esta estructura:

```
1 export function crearTablero(filas, columnas) {
2
3   const tablero = document.querySelector("#tablero");
4
5   tablero.innerHTML = "";
6
7   for (let fila = 0; fila < filas; fila++) {
8
9     for (let columna = 0; columna < columnas; columna++) {
10
11       const celda = document.createElement("div");
12
13       celda.classList.add("celda");
14
15       celda.dataset.fila = fila;
```

```
16     celda.dataset.columna = columna;
17
18     celda.addEventListener("click", () => {
19
20         const f = Number(celda.dataset.fila);
21         const c = Number(celda.dataset.columna);
22
23         manejarClickCelda(f, c);
24
25     });
26
27     tablero.appendChild(celda);
28
29 }
30
31 }
32
33 }
34
35 function manejarClickCelda(fila, columna) {
36
37     console.log("Celda pulsada:", fila, columna);
38
39 }
```

6.7 Qué ocurre cuando el jugador pulsa una celda

Cuando el usuario hace clic en una celda sucede lo siguiente:

1. El navegador genera un evento `click`.
2. El evento se captura con `addEventListener`.
3. Se obtiene la posición de la celda.
4. Se llama a la función `manejarClickCelda`.
5. La función procesa la acción.

Este mecanismo será el que permitirá implementar la lógica del juego.

6.8 Qué hemos conseguido en este capítulo

En este capítulo hemos añadido **interacción al tablero**.

Ahora el proyecto puede:

- detectar clics en cada celda
- identificar la posición de la celda pulsada

- ejecutar una función para procesar la acción

Este es un paso fundamental para que el jugador pueda interactuar con el juego.

En el siguiente capítulo comenzaremos a implementar **la lógica interna del tablero**, creando la matriz que almacenará la posición de las minas.

7 Matrices

Hasta ahora hemos construido la parte visible del Buscaminas: el tablero aparece en pantalla y el usuario puede hacer clic sobre sus celdas. Sin embargo, el juego todavía no tiene una **lógica interna**. Todas las celdas son visualmente iguales y el programa aún no sabe dónde están las minas ni qué valor tiene cada posición.

Para que el juego funcione de verdad necesitamos una estructura de datos que represente internamente el tablero.

Esa estructura será una **matriz**.

La matriz no se ve en pantalla. Es una representación interna que utilizará JavaScript para almacenar la información de cada celda. Gracias a ella podremos saber:

- dónde hay minas
- qué celdas están vacías
- cuántas minas hay alrededor de una posición
- qué celda ha abierto el jugador

En este capítulo vamos a construir esa matriz y a prepararla para que más adelante podamos colocar minas y calcular los números del tablero.

7.1 El tablero visible y el tablero interno

En un juego como Buscaminas conviene distinguir entre dos cosas:

- el **tablero visual**, que es lo que ve el usuario en el navegador
- el **tablero lógico**, que es la información interna que guarda el programa

Por ejemplo, una celda en pantalla puede parecer cerrada, pero internamente el programa ya sabe si contiene una mina o si tiene un número.

Esta separación es muy importante porque nos permite programar con más orden.

De ahora en adelante trabajaremos con estas dos capas:

- el DOM para mostrar el tablero
- una matriz de JavaScript para guardar el estado real del juego

7.2 Qué es una matriz en JavaScript

Una matriz es un array que contiene otros arrays.

Por ejemplo:

```
1 const matriz = [  
2   [0, 0, 0],  
3   [0, 1, 0],  
4   [0, 0, 0]  
5 ];
```

En este caso tenemos una matriz de 3 filas y 3 columnas.

Podemos acceder a cualquier posición indicando primero la fila y después la columna:

```
1 console.log(matriz[1][1]);
```

Esto mostraría el valor de la fila 1, columna 1.

En nuestro proyecto cada posición de la matriz representará una celda del tablero.

7.3 Crear una función para generar la matriz vacía

Vamos a comenzar creando una función que genere una matriz vacía del tamaño que queramos.

Abrimos el archivo `board.js` y añadimos la siguiente función:

```
1 export function crearMatriz(filas, columnas) {  
2  
3   const matriz = [];  
4  
5   for (let fila = 0; fila < filas; fila++) {  
6  
7     const filaActual = [];  
8  
9     for (let columna = 0; columna < columnas; columna++) {  
10      filaActual.push(0);  
11    }  
12  
13    matriz.push(filaActual);  
14  
15  }  
16  
17  return matriz;  
18  
19 }
```

Esta función crea una matriz donde todas las posiciones contienen inicialmente el valor 0.

7.4 Cómo funciona esta función

Vamos a entenderla paso a paso.

Primero creamos un array vacío:

```
1 const matriz = [];
```

Después recorremos las filas con un bucle:

```
1 for (let fila = 0; fila < filas; fila++) {
```

Dentro de cada fila creamos otro array:

```
1 const filaActual = [];
```

A continuación recorremos las columnas y vamos insertando ceros:

```
1 for (let column = 0; column < columnas; column++) {  
2   filaActual.push(0);  
3 }
```

Cuando la fila está completa la añadimos a la matriz principal:

```
1 matriz.push(filaActual);
```

Finalmente devolvemos la matriz:

```
1 return matriz;
```

7.5 Probar la matriz desde main.js

Vamos a comprobar que la función funciona correctamente.

En `main.js` importamos la nueva función:

```
1 import { crearTablero, crearMatriz } from "../board.js";
```

Y modificamos temporalmente la función principal:

```
1 function iniciarAplicacion() {  
2  
3   crearTablero(8, 8);  
4  
5   const matriz = crearMatriz(8, 8);  
6  
7   console.log(matriz);  
8 }
```

```
9  }  
10  
11  iniciarAplicacion();
```

Si abrimos la consola del navegador veremos una matriz de 8 filas por 8 columnas rellena de ceros. Esto significa que ya hemos creado la estructura interna del tablero.

7.6 Qué significará cada valor de la matriz

De momento todas las posiciones contienen 0, pero más adelante utilizaremos distintos valores para representar el contenido de cada celda.

Por ejemplo:

- 0 → celda vacía
- -1 → mina
- 1, 2, 3... → número de minas cercanas

Todavía no vamos a calcular los números, pero sí vamos a preparar la matriz para poder colocar minas.

7.7 Colocar minas aleatoriamente

El siguiente paso es insertar minas dentro de la matriz.

Para ello necesitamos una función que elija posiciones aleatorias y marque esas posiciones con el valor -1.

Añadimos a `board.js` la siguiente función:

```
1  export function colocarMinas(matriz, cantidadMinas) {  
2  
3    let minasColocadas = 0;  
4  
5    while (minasColocadas < cantidadMinas) {  
6  
7      const fila = Math.floor(Math.random() * matriz.length);  
8      const columna = Math.floor(Math.random() * matriz[0].length);  
9  
10     if (matriz[fila][columna] !== -1) {  
11       matriz[fila][columna] = -1;  
12       minasColocadas++;  
13     }  
14  
15   }
```

```
16  
17 }
```

Esta función modifica la matriz original y coloca las minas directamente en ella.

7.8 Cómo funciona `Math.random()`

Para generar posiciones aleatorias usamos esta expresión:

```
1 Math.floor(Math.random() * matriz.length)
```

`Math.random()` devuelve un número aleatorio entre 0 y 1.

Si lo multiplicamos por el número de filas obtenemos un valor comprendido entre 0 y el tamaño de la matriz.

Después usamos `Math.floor()` para quedarnos con la parte entera.

De esta forma obtenemos un índice válido para elegir una fila o una columna al azar.

7.9 Evitar que dos minas caigan en la misma celda

No basta con elegir posiciones aleatorias. También debemos evitar que dos minas se coloquen en la misma celda.

Por eso comprobamos:

```
1 if (matriz[fila][columna] !== -1)
```

Si en esa posición todavía no hay una mina, la colocamos:

```
1 matriz[fila][columna] = -1;
```

Y aumentamos el contador:

```
1 minasColocadas++;
```

Si ya había una mina, simplemente repetimos el intento.

7.10 Probar la colocación de minas

Ahora actualizamos `main.js` para probar esta nueva función.


```
1 import { crearTablero, crearMatriz, colocarMinas } from "./board.js";
2
3 function iniciarAplicacion() {
4
5     crearTablero(8, 8);
6
7     const matriz = crearMatriz(8, 8);
8
9     colocarMinas(matriz, 10);
10
11     console.log(matriz);
12 }
13
14
15 iniciarAplicacion();
```

En la consola veremos la matriz con varios valores `-1` repartidos de forma aleatoria.

Eso significa que ya tenemos un tablero lógico con minas colocadas.

7.11 Guardar la matriz para usarla más adelante

Hasta ahora la matriz solo se crea dentro de `iniciarAplicacion`, pero más adelante necesitaremos acceder a ella desde otras funciones.

Por eso conviene declarar una variable global de módulo en `main.js`:

```
1 let matrizJuego = [];
```

Y después asignarle el valor generado:

```
1 function iniciarAplicacion() {
2
3     crearTablero(8, 8);
4
5     matrizJuego = crearMatriz(8, 8);
6
7     colocarMinas(matrizJuego, 10);
8
9     console.log(matrizJuego);
10 }
11
12
13 iniciarAplicacion();
```

De esta forma la matriz del juego queda disponible para el resto del programa.

7.12 Código completo actualizado de board.js

Después de este capítulo, el archivo `board.js` debería quedar así:

```
1 export function crearTablero(filas, columnas) {
2
3   const tablero = document.querySelector("#tablero");
4
5   tablero.innerHTML = "";
6
7   for (let fila = 0; fila < filas; fila++) {
8
9     for (let columna = 0; columna < columnas; columna++) {
10
11       const celda = document.createElement("div");
12
13       celda.classList.add("celda");
14
15       celda.dataset.fila = fila;
16       celda.dataset.columna = columna;
17
18       celda.addEventListener("click", () => {
19
20         const f = Number(celda.dataset.fila);
21         const c = Number(celda.dataset.columna);
22
23         manejarClickCelda(f, c);
24
25       });
26
27       tablero.appendChild(celda);
28
29     }
30
31   }
32 }
33
34
35 function manejarClickCelda(fila, columna) {
36
37   console.log("Celda pulsada:", fila, columna);
38
39 }
40
41 export function crearMatriz(filas, columnas) {
42
43   const matriz = [];
44
45   for (let fila = 0; fila < filas; fila++) {
46
47     const filaActual = [];
```

```
48
49     for (let columna = 0; columna < columnas; columna++) {
50         filaActual.push(0);
51     }
52
53     matriz.push(filaActual);
54
55 }
56
57 return matriz;
58
59 }
60
61 export function colocarMinas(matriz, cantidadMinas) {
62
63     let minasColocadas = 0;
64
65     while (minasColocadas < cantidadMinas) {
66
67         const fila = Math.floor(Math.random() * matriz.length);
68         const columna = Math.floor(Math.random() * matriz[0].length);
69
70         if (matriz[fila][columna] !== -1) {
71             matriz[fila][columna] = -1;
72             minasColocadas++;
73         }
74
75     }
76
77 }
```

7.13 Qué hemos conseguido en este capítulo

En este capítulo hemos dado un paso fundamental en la construcción del juego.

Ahora el proyecto ya dispone de:

- una matriz interna para representar el tablero
- una función para crear esa matriz
- una función para colocar minas aleatoriamente
- una variable donde guardar el estado lógico de la partida

A partir de este momento el programa ya no solo muestra un tablero vacío, sino que también **conoce internamente dónde están las minas**.

En el siguiente capítulo calcularemos los números de cada celda, es decir, cuántas minas hay alrededor de cada posición del tablero.

8 Pensamiento computacional: calculando los números para las celdas

8.1 Calcular los números de cada celda

En el capítulo anterior hemos creado la matriz interna del tablero y hemos colocado las minas aleatoriamente. Ahora el programa ya sabe dónde están las minas, pero todavía falta una parte esencial del Buscaminas: **calcular los números que aparecen en las celdas**.

En este juego, cada celda que no contiene una mina muestra un número. Ese número indica cuántas minas hay en las posiciones vecinas.

Por ejemplo, si una celda tiene el valor 2, significa que en las ocho posiciones que la rodean hay exactamente dos minas.

En este capítulo vamos a aprender a:

- recorrer la matriz del tablero
- inspeccionar las celdas vecinas
- contar minas alrededor de una posición
- guardar ese número dentro de la matriz

Cuando terminemos, la matriz interna del juego ya contendrá toda la información necesaria para que el tablero funcione correctamente.

8.2 Qué significa “celdas vecinas”

Cada celda del tablero puede tener hasta ocho vecinas:

- arriba
- abajo
- izquierda
- derecha
- arriba izquierda
- arriba derecha
- abajo izquierda
- abajo derecha

Si imaginamos una celda en el centro, sus vecinas serían estas posiciones:

1	<code>fila-1, columna-1</code>	<code>fila-1, columna</code>	<code>fila-1, columna+1</code>
2	<code>fila, columna-1</code>	<code>fila, columna</code>	<code>fila, columna+1</code>
3	<code>fila+1, columna-1</code>	<code>fila+1, columna</code>	<code>fila+1, columna+1</code>

La celda central no cuenta. Solo nos interesan las que están alrededor.

8.3 Qué debemos calcular

Vamos a recorrer todas las posiciones de la matriz.

Para cada celda:

- si contiene una mina, no hacemos nada
- si no contiene una mina, contamos cuántas minas hay alrededor
- guardamos ese número en la propia matriz

Por ejemplo, si una posición está vacía pero tiene tres minas alrededor, esa celda pasará a contener el valor 3.

De esta forma la matriz quedará completamente preparada.

8.4 Crear una función para contar minas vecinas

Abrimos el archivo `board.js` y añadimos una nueva función.

```
1 export function contarMinasVecinas(matriz, fila, columna) {
2
3   let total = 0;
4
5   for (let f = fila - 1; f <= fila + 1; f++) {
6     for (let c = columna - 1; c <= columna + 1; c++) {
7
8       if (f === fila && c === columna) {
9         continue;
10      }
11
12      if (
13        f >= 0 &&
14        f < matriz.length &&
15        c >= 0 &&
16        c < matriz[0].length
17      ) {
18        if (matriz[f][c] === -1) {
19          total++;
20        }
21      }
22    }
23  }
24  }
25
26  return total;
27
28 }
```

Esta función recibe:

- la matriz del tablero
- la fila de la celda actual
- la columna de la celda actual

Y devuelve el número de minas que hay alrededor.

8.5 Cómo funciona esta función

La variable `total` almacenará el número de minas encontradas.

```
1 let total = 0;
```

Después usamos dos bucles para recorrer las posiciones vecinas:

```
1 for (let f = fila - 1; f <= fila + 1; f++) {  
2   for (let c = columna - 1; c <= columna + 1; c++) {
```

Esto recorre las tres filas y las tres columnas que rodean a la celda actual.

Pero una de esas posiciones es la propia celda central. No debemos contarla, así que la saltamos:

```
1 if (f === fila && c === columna) {  
2   continue;  
3 }
```

8.6 Comprobar los límites de la matriz

No todas las celdas tienen ocho vecinas.

Por ejemplo, una celda en una esquina solo tiene tres posiciones alrededor. Por eso debemos comprobar que las coordenadas vecinas sean válidas.

```
1 if (  
2   f >= 0 &&  
3   f < matriz.length &&  
4   c >= 0 &&  
5   c < matriz[0].length  
6 )
```

Con esta condición evitamos acceder a posiciones que no existen.

8.7 Contar minas

Una vez comprobado que la posición vecina es válida, miramos si contiene una mina.

```
1 if (matriz[f][c] === -1) {  
2   total++;  
3 }
```

Si hay una mina, incrementamos el contador.

Al final devolvemos el total.

8.8 Calcular todos los números del tablero

Ahora necesitamos una función que recorra toda la matriz y aplique este cálculo a cada celda que no sea una mina.

Añadimos en `board.js` la siguiente función:

```
1 export function calcularNumeros(matriz) {  
2  
3   for (let fila = 0; fila < matriz.length; fila++) {  
4     for (let columna = 0; columna < matriz[fila].length; columna++) {  
5  
6       if (matriz[fila][columna] !== -1) {  
7         matriz[fila][columna] = contarMinasVecinas(matriz, fila,  
8           columna);  
9       }  
10    }  
11  }  
12  
13 }
```

Esta función recorre todas las posiciones del tablero.

Si encuentra una celda que no contiene una mina, calcula cuántas minas hay alrededor y sustituye el valor 0 por el número correspondiente.

8.9 Probar el cálculo desde main.js

Ahora vamos a utilizar esta nueva función en `main.js`.

Actualizamos la importación:

```
1 import {  
2   crearTablero,
```

```
3   crearMatriz,  
4   colocarMinas,  
5   calcularNumeros  
6 } from "./board.js";
```

Y modificamos la función principal:

```
1 let matrizJuego = [];  
2  
3 function iniciarAplicacion() {  
4  
5   crearTablero(8, 8);  
6  
7   matrizJuego = crearMatriz(8, 8);  
8  
9   colocarMinas(matrizJuego, 10);  
10  
11  calcularNumeros(matrizJuego);  
12  
13  console.log(matrizJuego);  
14  
15 }  
16  
17 iniciarAplicacion();
```

Ahora la consola ya no mostrará solo ceros y minas. También veremos números en aquellas posiciones que tengan minas alrededor.

8.10 Qué aspecto tendrá la matriz ahora

Después de ejecutar `calcularNumeros`, la matriz podría quedar con un aspecto parecido a este:

```
1  [  
2    [0, 1, -1, 2, 1],  
3    [0, 1, 2, -1, 1],  
4    [0, 0, 1, 1, 1],  
5    [1, 1, 0, 0, 0],  
6    [-1, 1, 0, 0, 0]  
7  ]
```

En este ejemplo:

- -1 representa una mina
- 0 representa una celda vacía sin minas alrededor
- 1, 2, etc. indican el número de minas cercanas

Esta es exactamente la información que necesita el juego para funcionar.

8.11 Por qué todavía no se muestran los números en pantalla

Aunque la matriz ya contiene los números correctos, de momento el jugador no los ve. Eso es porque el tablero visible sigue siendo solo una cuadrícula de celdas cerradas.

La matriz es la parte lógica del juego.

Más adelante, cuando el usuario haga clic en una celda, el programa consultará la matriz para saber:

- si hay una mina
- si debe mostrar un número
- si la celda está vacía y hay que abrir más posiciones

Por tanto, este cálculo es un paso interno pero absolutamente necesario.

8.12 Código completo actualizado de board.js

Después de este capítulo, el archivo `board.js` debería quedar así:

```
1 export function crearTablero(filas, columnas) {
2
3   const tablero = document.querySelector("#tablero");
4
5   tablero.innerHTML = "";
6
7   for (let fila = 0; fila < filas; fila++) {
8
9     for (let columna = 0; columna < columnas; columna++) {
10
11       const celda = document.createElement("div");
12
13       celda.classList.add("celda");
14
15       celda.dataset.fila = fila;
16       celda.dataset.columna = columna;
17
18       celda.addEventListener("click", () => {
19
20         const f = Number(celda.dataset.fila);
21         const c = Number(celda.dataset.columna);
22
23         manejarClickCelda(f, c);
24
25       });
26
27       tablero.appendChild(celda);
28
29     }
```

```
30
31   }
32
33 }
34
35 function manejarClickCelda(fila, columna) {
36
37   console.log("Celda pulsada:", fila, columna);
38
39 }
40
41 export function crearMatriz(filas, columnas) {
42
43   const matriz = [];
44
45   for (let fila = 0; fila < filas; fila++) {
46
47     const filaActual = [];
48
49     for (let columna = 0; columna < columnas; columna++) {
50       filaActual.push(0);
51     }
52
53     matriz.push(filaActual);
54
55   }
56
57   return matriz;
58
59 }
60
61 export function colocarMinas(matriz, cantidadMinas) {
62
63   let minasColocadas = 0;
64
65   while (minasColocadas < cantidadMinas) {
66
67     const fila = Math.floor(Math.random() * matriz.length);
68     const columna = Math.floor(Math.random() * matriz[0].length);
69
70     if (matriz[fila][columna] !== -1) {
71       matriz[fila][columna] = -1;
72       minasColocadas++;
73     }
74
75   }
76
77 }
78
79 export function contarMinasVecinas(matriz, fila, columna) {
80
```

```
81     let total = 0;
82
83     for (let f = fila - 1; f <= fila + 1; f++) {
84         for (let c = columna - 1; c <= columna + 1; c++) {
85
86             if (f === fila && c === columna) {
87                 continue;
88             }
89
90             if (
91                 f >= 0 &&
92                 f < matriz.length &&
93                 c >= 0 &&
94                 c < matriz[0].length
95             ) {
96                 if (matriz[f][c] === -1) {
97                     total++;
98                 }
99             }
100         }
101     }
102 }
103
104 return total;
105 }
106 }
107
108 export function calcularNumeros(matriz) {
109
110     for (let fila = 0; fila < matriz.length; fila++) {
111         for (let columna = 0; columna < matriz[fila].length; columna++) {
112
113             if (matriz[fila][columna] !== -1) {
114                 matriz[fila][columna] = contarMinasVecinas(matriz, fila,
115                     columna);
116             }
117         }
118     }
119 }
120 }
```

8.13 Qué hemos conseguido en este capítulo

En este capítulo hemos completado la preparación lógica del tablero.

Ahora el proyecto ya es capaz de:

- colocar minas aleatoriamente

- recorrer las celdas del tablero
- calcular cuántas minas rodean a cada posición
- guardar esa información dentro de la matriz

Con esto ya tenemos el tablero lógico casi terminado.

En el siguiente capítulo empezaremos a conectar esta lógica con la parte visual del juego, haciendo que al pulsar una celda se abra y muestre su contenido.

9 Abrir celdas y mostrar el contenido

En los capítulos anteriores hemos construido la base lógica del Buscaminas. Ya somos capaces de crear una matriz interna, colocar minas y calcular cuántas minas rodean a cada celda.

Sin embargo, todavía falta lo más importante desde el punto de vista del jugador: **que al pulsar una celda ocurra algo visible en pantalla.**

En este capítulo vamos a conectar la lógica interna del juego con la parte visual del tablero. A partir de ahora, cuando el usuario haga clic en una celda, el programa consultará la matriz y mostrará en pantalla lo que hay en esa posición.

Esto nos permitirá:

- abrir celdas
- mostrar números
- mostrar minas
- empezar a dar comportamiento real al tablero

9.1 Qué debe ocurrir al pulsar una celda

Cuando el jugador hace clic en una celda, pueden ocurrir tres situaciones:

- la celda contiene una mina
- la celda contiene un número
- la celda está vacía

En este capítulo vamos a implementar las dos primeras:

- si hay una mina, mostraremos un símbolo de mina
- si hay un número, mostraremos ese número
- si el valor es 0, de momento mostraremos una celda vacía abierta

Más adelante mejoraremos el comportamiento de las celdas vacías para que se abran automáticamente sus vecinas.

9.2 Necesitamos acceder a la matriz desde el tablero

Hasta ahora el archivo `board.js` genera las celdas visuales y detecta clics, pero la matriz real del juego está en `main.js`.

Por tanto, cuando se pulse una celda, `board.js` debe poder avisar a `main.js` de qué posición se ha pulsado.

Una forma sencilla de hacerlo es permitir que `crearTablero` reciba una función como parámetro. Esa función se ejecutará cada vez que el usuario haga clic sobre una celda.

Esto nos permitirá mantener una separación clara:

- `board.js` se ocupa del tablero visual
- `main.js` se ocupa de la lógica de la partida

9.3 Modificar `crearTablero` para recibir una función

Vamos a actualizar la función `crearTablero` en `board.js`.

Antes la teníamos así:

```
1 export function crearTablero(filas, columnas) {
```

Ahora la cambiaremos a esta versión:

```
1 export function crearTablero(filas, columnas, alHacerClickCelda) {
```

Y dentro del evento de clic, en lugar de llamar a `manejarClickCelda`, llamaremos a la función recibida como parámetro.

El código quedará así:

```
1 export function crearTablero(filas, columnas, alHacerClickCelda) {
2
3   const tablero = document.querySelector("#tablero");
4
5   tablero.innerHTML = "";
6
7   for (let fila = 0; fila < filas; fila++) {
8
9     for (let columna = 0; columna < columnas; columna++) {
10
11       const celda = document.createElement("div");
12
13       celda.classList.add("celda");
14
15       celda.dataset.fila = fila;
16       celda.dataset.columna = columna;
17
18       celda.addEventListener("click", () => {
19
20         const f = Number(celda.dataset.fila);
```

```
21     const c = Number(celda.dataset.columna);
22
23     alHacerClickCelda(f, c);
24
25     });
26
27     tablero.appendChild(celda);
28
29   }
30
31 }
32
33 }
```

Con este cambio, el tablero sigue creando las celdas, pero ahora la lógica del clic se decidirá desde `main.js`.

9.4 Crear una función para abrir una celda

Ahora vamos a `main.js`, donde sí tenemos acceso a la matriz interna.

Vamos a declarar una función que abra una celda.

```
1 function abrirCelda(fila, columna) {
2
3   const valor = matrizJuego[fila][columna];
4
5   console.log("Abrir celda:", fila, columna, "valor:", valor);
6
7 }
```

Esta función busca el valor almacenado en la matriz para la posición pulsada.

De momento solo lo mostramos en consola para comprobar que el flujo funciona.

9.5 Llamar a `abrirCelda` desde `crearTablero`

Actualizamos la llamada a `crearTablero` en `main.js`.

Antes teníamos:

```
1 crearTablero(8, 8);
```

Ahora usaremos:

```
1 crearTablero(8, 8, abrirCelda);
```

De este modo, cuando el usuario haga clic sobre una celda, el tablero llamará a la función `abrirCelda`.

9.6 Localizar visualmente la celda pulsada

Además de consultar la matriz, necesitamos modificar la celda correspondiente en el DOM.

Como cada celda tiene guardadas sus coordenadas en `dataset`, podemos localizarla con un selector.

Añadimos en `main.js` esta función auxiliar:

```
1 function obtenerCeldaDOM(fila, columna) {
2
3   return document.querySelector(
4     `.celda[data-fila="${fila}"][data-columna="${columna}"]`
5   );
6
7 }
```

Esta función devuelve el elemento HTML correspondiente a una posición del tablero.

9.7 Mostrar el contenido de la celda

Ahora ya podemos completar la función `abrirCelda`.

```
1 function abrirCelda(fila, columna) {
2
3   const valor = matrizJuego[fila][columna];
4   const celda = obtenerCeldaDOM(fila, columna);
5
6   if (!celda) {
7     return;
8   }
9
10  celda.classList.add("abierta");
11
12  if (valor === -1) {
13    celda.textContent = "●";
14  } else if (valor > 0) {
15    celda.textContent = valor;
16  } else {
17    celda.textContent = "";
18  }
19
20 }
```


Veamos qué hace este código:

- obtiene el valor de la matriz
- localiza la celda en el DOM
- le añade una clase para indicar que está abierta
- muestra una mina, un número o deja la celda vacía

9.8 Evitar abrir dos veces la misma celda

Si un jugador pulsa varias veces sobre la misma celda, no queremos que el programa vuelva a procesarla continuamente.

Por eso conviene comprobar si ya está abierta.

Podemos hacerlo mirando si ya tiene la clase `abierta`.

```
1 function abrirCelda(fila, columna) {
2
3   const valor = matrizJuego[fila][columna];
4   const celda = obtenerCeldaDOM(fila, columna);
5
6   if (!celda) {
7     return;
8   }
9
10  if (celda.classList.contains("abierta")) {
11    return;
12  }
13
14  celda.classList.add("abierta");
15
16  if (valor === -1) {
17    celda.textContent = "●";
18  } else if (valor > 0) {
19    celda.textContent = valor;
20  } else {
21    celda.textContent = "";
22  }
23
24 }
```

Con esta condición evitamos abrir varias veces la misma celda.

9.9 Actualizar iniciarAplicacion

Ahora la función principal de `main.js` debe quedar así:

```
1 import {
2   crearTablero,
3   crearMatriz,
4   colocarMinas,
5   calcularNumeros
6 } from "../board.js";
7
8 let matrizJuego = [];
9
10 function iniciarAplicacion() {
11   matrizJuego = crearMatriz(8, 8);
12   colocarMinas(matrizJuego, 10);
13   calcularNumeros(matrizJuego);
14   crearTablero(8, 8, abrirCelda);
15 }
16
17 function abrirCelda(fila, columna) {
18   const valor = matrizJuego[fila][columna];
19   const celda = obtenerCeldaDOM(fila, columna);
20
21   if (!celda) {
22     return;
23   }
24
25   if (celda.classList.contains("abierta")) {
26     return;
27   }
28
29   celda.classList.add("abierta");
30
31   if (valor === -1) {
32     celda.textContent = "●";
33   } else if (valor > 0) {
34     celda.textContent = valor;
35   } else {
36     celda.textContent = "";
37   }
38 }
39
40 function obtenerCeldaDOM(fila, columna) {
41   return document.querySelector(
42     `celda[data-fila="${fila}"][data-columna="${columna}"]`
43   );
44 }
```

```
51     );  
52  
53 }  
54  
55 iniciarAplicacion();
```

9.10 Qué conseguimos ahora

A partir de este momento el juego ya tiene un comportamiento visible real.

Cuando el jugador haga clic en una celda:

- el programa consultará la matriz
- abrirá la celda correspondiente
- mostrará una mina si la hay
- mostrará un número si la celda tiene minas alrededor
- dejará la celda vacía si su valor es 0

Todavía faltan muchas mejoras, pero ya tenemos el primer funcionamiento auténtico del Buscaminas.

9.11 Código completo actualizado de board.js

Después de este capítulo, `board.js` quedará así:

```
1 export function crearTablero(filas, columnas, alHacerClickCelda) {  
2  
3     const tablero = document.querySelector("#tablero");  
4  
5     tablero.innerHTML = "";  
6  
7     for (let fila = 0; fila < filas; fila++) {  
8  
9         for (let columna = 0; columna < columnas; columna++) {  
10  
11             const celda = document.createElement("div");  
12  
13             celda.classList.add("celda");  
14  
15             celda.dataset.fila = fila;  
16             celda.dataset.columna = columna;  
17  
18             celda.addEventListener("click", () => {  
19  
20                 const f = Number(celda.dataset.fila);
```

```
21     const c = Number(celda.dataset.columna);
22
23     alHacerClickCelda(f, c);
24
25     });
26
27     tablero.appendChild(celda);
28
29     }
30
31     }
32
33 }
34
35 export function crearMatriz(filas, columnas) {
36     const matriz = [];
37
38     for (let fila = 0; fila < filas; fila++) {
39
40         const filaActual = [];
41
42         for (let columna = 0; columna < columnas; columna++) {
43             filaActual.push(0);
44         }
45
46         matriz.push(filaActual);
47
48     }
49
50     return matriz;
51 }
52
53 }
54
55 export function colocarMinas(matriz, cantidadMinas) {
56     let minasColocadas = 0;
57
58     while (minasColocadas < cantidadMinas) {
59
60         const fila = Math.floor(Math.random() * matriz.length);
61         const columna = Math.floor(Math.random() * matriz[0].length);
62
63         if (matriz[fila][columna] !== -1) {
64             matriz[fila][columna] = -1;
65             minasColocadas++;
66         }
67
68     }
69
70 }
71 }
```

```
72
73 export function contarMinasVecinas(matriz, fila, columna) {
74
75     let total = 0;
76
77     for (let f = fila - 1; f <= fila + 1; f++) {
78         for (let c = columna - 1; c <= columna + 1; c++) {
79
80             if (f === fila && c === columna) {
81                 continue;
82             }
83
84             if (
85                 f >= 0 &&
86                 f < matriz.length &&
87                 c >= 0 &&
88                 c < matriz[0].length
89             ) {
90                 if (matriz[f][c] === -1) {
91                     total++;
92                 }
93             }
94
95         }
96     }
97
98     return total;
99
100 }
101
102 export function calcularNumeros(matriz) {
103
104     for (let fila = 0; fila < matriz.length; fila++) {
105         for (let columna = 0; columna < matriz[fila].length; columna++) {
106
107             if (matriz[fila][columna] !== -1) {
108                 matriz[fila][columna] = contarMinasVecinas(matriz, fila,
109                     columna);
110             }
111         }
112     }
113
114 }
```

9.12 Qué hemos conseguido en este capítulo

En este capítulo hemos dado un paso muy importante: hemos unido la lógica interna del juego con la interfaz visual.

Ahora el proyecto ya es capaz de:

- detectar qué celda pulsa el jugador
- consultar el valor de esa posición en la matriz
- abrir visualmente la celda
- mostrar minas y números en pantalla

El juego ya empieza a parecerse a un Buscaminas real.

En el siguiente capítulo mejoraremos el comportamiento de las celdas vacías, haciendo que cuando el jugador abra una celda con valor 0 se descubran automáticamente las celdas vecinas.

10 Destapando el vacío

En el capítulo anterior hemos conseguido que el jugador pueda abrir celdas y ver su contenido. Si la celda contiene un número, se muestra en pantalla. Si contiene una mina, aparece el símbolo correspondiente. Si su valor es 0, la celda se abre pero queda vacía.

Sin embargo, en un Buscaminas real ocurre algo más: cuando el jugador abre una celda vacía, no se descubre solo esa posición, sino también las celdas vecinas que forman parte de la misma zona vacía.

Ese será el objetivo de este capítulo.

Vamos a implementar el comportamiento clásico del juego:

- si una celda tiene un número, solo se abre esa celda
- si una celda es vacía (0), se abrirán también sus vecinas
- el proceso continuará mientras sigan apareciendo celdas vacías conectadas

Este mecanismo hará que el tablero se comporte ya de una forma mucho más parecida al Buscaminas real.

10.1 Qué significa expandir una zona vacía

Una celda vacía es una celda cuyo valor en la matriz es 0. Eso significa que no hay minas en ninguna de las posiciones que la rodean.

Cuando el jugador abre una celda así, el juego puede abrir con seguridad todas sus vecinas. Algunas de esas vecinas también serán vacías, por lo que el proceso debe continuar.

Imaginemos una zona amplia del tablero donde no hay minas cerca. En lugar de obligar al usuario a pulsar celda por celda, el juego abre automáticamente toda esa región.

Ese proceso se conoce como **expansión de celdas vacías**.

10.2 El problema de la repetición

Si queremos abrir automáticamente las vecinas de una celda vacía, no basta con abrir solo las ocho posiciones que la rodean. También debemos abrir las vecinas de aquellas que sean vacías, y así sucesivamente.

Esto significa que necesitamos una función que pueda llamarse a sí misma varias veces.

En programación, este patrón se llama **recursividad**.

10.3 Idea general de la solución

La lógica será la siguiente:

1. abrir la celda actual
2. si tiene un número, detenerse
3. si tiene valor 0, recorrer sus vecinas
4. abrir cada vecina válida
5. si alguna vecina también es 0, repetir el proceso

Para que funcione bien, debemos evitar abrir dos veces la misma celda. Si no lo hacemos, el programa entraría en un bucle sin fin.

Afortunadamente ya tenemos una forma sencilla de comprobarlo: si la celda ya tiene la clase `abierta`, no volvemos a procesarla.

10.4 Crear una función para abrir vecinas

Vamos a seguir trabajando en `main.js`.

Hasta ahora teníamos una función `abrirCelda(fila, columna)` que abría una única posición. Vamos a ampliarla para que, cuando el valor sea 0, abra también las celdas vecinas.

Partimos de esta estructura:

```
1 function abrirCelda(fila, columna) {
2
3   const valor = matrizJuego[fila][columna];
4   const celda = obtenerCeldaDOM(fila, columna);
5
6   if (!celda) {
7     return;
8   }
9
10  if (celda.classList.contains("abierta")) {
11    return;
12  }
13
14  celda.classList.add("abierta");
15
16  if (valor === -1) {
17    celda.textContent = "●";
18  } else if (valor > 0) {
19    celda.textContent = valor;
20  } else {
21    celda.textContent = "";
22  }
```



```
23  
24 }
```

Ahora añadiremos la lógica de expansión.

10.5 Recorrer las ocho vecinas

Si la celda tiene valor 0, necesitaremos recorrer las posiciones que la rodean.

Podemos hacerlo con dos bucles muy parecidos a los que usamos para contar minas vecinas.

Añadimos este bloque al final de `abrirCelda`:

```
1  if (valor === 0) {  
2  
3      for (let f = fila - 1; f <= fila + 1; f++) {  
4          for (let c = columna - 1; c <= columna + 1; c++) {  
5  
6              if (f === fila && c === columna) {  
7                  continue;  
8              }  
9  
10             if (  
11                 f >= 0 &&  
12                 f < matrizJuego.length &&  
13                 c >= 0 &&  
14                 c < matrizJuego[0].length  
15             ) {  
16                 abrirCelda(f, c);  
17             }  
18  
19         }  
20     }  
21  
22 }
```

Este código hace lo siguiente:

- recorre las posiciones vecinas
- ignora la celda central
- comprueba que cada posición está dentro de la matriz
- llama de nuevo a `abrirCelda` para cada vecina válida

Ahí está la recursividad.

10.6 Por qué esta función no entra en bucle infinito

Puede parecer que llamar a `abrirCelda` desde dentro de `abrirCelda` es peligroso, pero en este caso funciona porque al principio de la función comprobamos esto:

```
1 if (celda.classList.contains("abierta")) {  
2   return;  
3 }
```

En cuanto una celda ya ha sido abierta, el programa no vuelve a procesarla.

Gracias a esa condición, cada celda se abre una sola vez.

10.7 Código completo de `abrirCelda` con expansión

La función `abrirCelda` queda ahora así:

```
1 function abrirCelda(fila, columna) {  
2  
3   const valor = matrizJuego[fila][columna];  
4   const celda = obtenerCeldaDOM(fila, columna);  
5  
6   if (!celda) {  
7     return;  
8   }  
9  
10  if (celda.classList.contains("abierta")) {  
11    return;  
12  }  
13  
14  celda.classList.add("abierta");  
15  
16  if (valor === -1) {  
17    celda.textContent = "●";  
18    return;  
19  }  
20  
21  if (valor > 0) {  
22    celda.textContent = valor;  
23    return;  
24  }  
25  
26  celda.textContent = "";  
27  
28  for (let f = fila - 1; f <= fila + 1; f++) {  
29    for (let c = columna - 1; c <= columna + 1; c++) {  
30  
31      if (f === fila && c === columna) {  
32        continue;  
33      }  
34    }  
35  }  
36}
```

```
33     }
34
35     if (
36         f >= 0 &&
37         f < matrizJuego.length &&
38         c >= 0 &&
39         c < matrizJuego[0].length
40     ) {
41         abrirCelda(f, c);
42     }
43
44 }
45 }
46
47 }
```

Observa que ahora usamos varios **return** para separar claramente los casos:

- si es mina, termina
- si es número, termina
- solo si es 0 continúa con la expansión

Esta forma de escribir el código mejora mucho su claridad.

10.8 Qué ocurre ahora cuando el jugador pulsa una celda vacía

Con esta nueva versión, cuando el jugador pulsa una celda cuyo valor es 0, el programa:

1. abre la celda actual
2. recorre sus vecinas
3. abre cada vecina
4. si alguna vecina también es 0, repite el proceso
5. se detiene en las celdas numéricas que rodean la zona vacía

Este comportamiento es exactamente el que esperamos en el Buscaminas.

10.9 Resultado práctico

A partir de este capítulo, el juego ya no abrirá solo una celda aislada.

Ahora, si el jugador pulsa una zona vacía:

- se abrirá una región entera del tablero
- aparecerán automáticamente los números de los bordes

- la experiencia de juego será mucho más natural

Este cambio hace que el proyecto dé un salto importante en jugabilidad.

10.10 Código completo actualizado de main.js

Después de este capítulo, `main.js` debería quedar así:

```
1  import {
2    crearTablero,
3    crearMatriz,
4    colocarMinas,
5    calcularNumeros
6  } from "./board.js";
7
8  let matrizJuego = [];
9
10 function iniciarAplicacion() {
11
12   matrizJuego = crearMatriz(8, 8);
13
14   colocarMinas(matrizJuego, 10);
15
16   calcularNumeros(matrizJuego);
17
18   crearTablero(8, 8, abrirCelda);
19
20 }
21
22 function abrirCelda(fila, columna) {
23
24   const valor = matrizJuego[fila][columna];
25   const celda = obtenerCeldaDOM(fila, columna);
26
27   if (!celda) {
28     return;
29   }
30
31   if (celda.classList.contains("abierta")) {
32     return;
33   }
34
35   celda.classList.add("abierta");
36
37   if (valor === -1) {
38     celda.textContent = "💣";
39     return;
40   }
41 }
```

```
42  if (valor > 0) {
43      celda.textContent = valor;
44      return;
45  }
46
47  celda.textContent = "";
48
49  for (let f = fila - 1; f <= fila + 1; f++) {
50      for (let c = columna - 1; c <= columna + 1; c++) {
51
52          if (f === fila && c === columna) {
53              continue;
54          }
55
56          if (
57              f >= 0 &&
58              f < matrizJuego.length &&
59              c >= 0 &&
60              c < matrizJuego[0].length
61          ) {
62              abrirCelda(f, c);
63          }
64      }
65  }
66
67
68 }
69
70 function obtenerCeldaDOM(fila, columna) {
71
72     return document.querySelector(
73         `.celda[data-fila="${fila}"][data-columna="${columna}"]`
74     );
75
76 }
77
78 iniciarAplicacion();
```

10.11 Qué hemos conseguido en este capítulo

En este capítulo hemos implementado una de las mecánicas más importantes del Buscaminas:

- la apertura automática de zonas vacías
- la expansión recursiva de celdas
- la detención correcta en celdas numéricas
- la prevención de aperturas repetidas

Con esto, el juego ya empieza a comportarse de una forma mucho más real.

En el siguiente capítulo vamos a añadir la otra gran mecánica del Buscaminas: **colocar y quitar banderas con el clic derecho del ratón.**

11 Eventos de ratón

En los capítulos anteriores hemos conseguido que el jugador pueda abrir celdas y que las zonas vacías se expandan automáticamente. El juego ya se parece bastante a un Buscaminas real, pero todavía falta una mecánica esencial: **las banderas**.

En Buscaminas, las banderas permiten marcar las celdas donde el jugador sospecha que hay una mina. No abren la celda, simplemente la señalan.

En este capítulo vamos a implementar esta funcionalidad usando el **clic derecho del ratón**.

Al terminar, el jugador podrá:

- poner una bandera sobre una celda cerrada
- quitar una bandera si cambia de opinión
- evitar abrir accidentalmente una celda marcada

Esta mejora aporta mucha jugabilidad y nos acerca bastante más al resultado final del proyecto.

11.1 Qué evento se produce al hacer clic derecho

Cuando el usuario pulsa el botón derecho del ratón sobre un elemento, el navegador genera un evento llamado:

```
1 contextmenu
```

Por defecto, este evento abre el menú contextual del navegador.

Pero en nuestro juego no queremos que aparezca ese menú sobre las celdas del tablero. Queremos usar el clic derecho para marcar minas.

Por eso tendremos que hacer dos cosas:

- detectar el evento `contextmenu`
- cancelar el comportamiento por defecto del navegador

Esto se consigue con:

```
1 event.preventDefault();
```

11.2 Añadir el evento de clic derecho a cada celda

Vamos a modificar el archivo `board.js`.

Hasta ahora, cada celda tenía un evento `click`. Ahora añadiremos también un evento `contextmenu`.

La función `crearTablero` pasará a recibir dos funciones:

- una para el clic izquierdo
- otra para el clic derecho

La cabecera de la función quedará así:

```
1 export function crearTablero(filas, columnas, alHacerClickCelda,  
    alHacerClickDerechoCelda) {
```

Y dentro del bucle donde creamos cada celda añadimos este nuevo evento:

```
1 celda.addEventListener("contextmenu", (event) => {  
2  
3   event.preventDefault();  
4  
5   const f = Number(celda.dataset.fila);  
6   const c = Number(celda.dataset.columna);  
7  
8   alHacerClickDerechoCelda(f, c);  
9  
10  });
```

Con este cambio, cada vez que el usuario haga clic derecho sobre una celda, el navegador no mostrará su menú contextual y en su lugar se ejecutará nuestra función.

11.3 Código actualizado de crearTablero

La función completa quedará así:

```
1 export function crearTablero(filas, columnas, alHacerClickCelda,  
    alHacerClickDerechoCelda) {  
2  
3   const tablero = document.querySelector("#tablero");  
4  
5   tablero.innerHTML = "";  
6  
7   for (let fila = 0; fila < filas; fila++) {  
8  
9     for (let columna = 0; columna < columnas; columna++) {  
10  
11       const celda = document.createElement("div");  
12  
13       celda.classList.add("celda");  
14
```



```
15     celda.dataset.fila = fila;
16     celda.dataset.columna = columna;
17
18     celda.addEventListener("click", () => {
19
20         const f = Number(celda.dataset.fila);
21         const c = Number(celda.dataset.columna);
22
23         alHacerClickCelda(f, c);
24
25     });
26
27     celda.addEventListener("contextmenu", (event) => {
28
29         event.preventDefault();
30
31         const f = Number(celda.dataset.fila);
32         const c = Number(celda.dataset.columna);
33
34         alHacerClickDerechoCelda(f, c);
35
36     });
37
38     tablero.appendChild(celda);
39
40 }
41
42 }
43
44 }
```

11.4 Crear una función para marcar banderas

Ahora pasamos a `main.js`.

Necesitamos una función que marque o desmarque una celda cuando el jugador haga clic derecho.

Vamos a crear esta función:

```
1 function alternarBandera(fila, columna) {
2
3     const celda = obtenerCeldaDOM(fila, columna);
4
5     if (!celda) {
6         return;
7     }
8
9     if (celda.classList.contains("abierta")) {
10         return;
```

```
11 }
12
13 if (celda.classList.contains("bandera")) {
14     celda.classList.remove("bandera");
15     celda.textContent = "";
16 } else {
17     celda.classList.add("bandera");
18     celda.textContent = "F";
19 }
20
21 }
```

Esta función hace lo siguiente:

- busca la celda en el DOM
- si no existe, no hace nada
- si la celda ya está abierta, no permite poner bandera
- si ya tenía bandera, la quita
- si no tenía bandera, la pone

11.5 Evitar abrir una celda con bandera

Ahora debemos mejorar la función `abrirCelda`.

Si una celda está marcada con bandera, el clic izquierdo no debe abrirla.

Por tanto, añadimos esta comprobación justo después de localizar la celda:

```
1 if (celda.classList.contains("bandera")) {
2     return;
3 }
```

La parte inicial de `abrirCelda` quedará así:

```
1 function abrirCelda(fila, columna) {
2
3     const valor = matrizJuego[fila][columna];
4     const celda = obtenerCeldaDOM(fila, columna);
5
6     if (!celda) {
7         return;
8     }
9
10    if (celda.classList.contains("abierto")) {
11        return;
12    }
13
14    if (celda.classList.contains("bandera")) {
```

```
15     return;  
16 }
```

Con esto, las banderas realmente protegen a la celda frente a una apertura accidental.

11.6 Actualizar la llamada a crearTablero

Como ahora `crearTablero` recibe dos funciones, debemos actualizar la llamada en `main.js`.

Antes teníamos:

```
1 crearTablero(8, 8, abrirCelda);
```

Ahora debe quedar así:

```
1 crearTablero(8, 8, abrirCelda, alternarBandera);
```

De esta forma:

- el clic izquierdo llamará a `abrirCelda`
- el clic derecho llamará a `alternarBandera`

11.7 Comportamiento esperado

A partir de ahora, el tablero responderá de esta manera:

- **clic izquierdo:** abre la celda
- **clic derecho:** pone o quita una bandera
- **clic izquierdo sobre una bandera:** no hace nada
- **clic derecho sobre una celda abierta:** no hace nada

Este comportamiento coincide con el funcionamiento habitual del Buscaminas.

11.8 Código completo actualizado de main.js

Después de este capítulo, `main.js` debería quedar así:

```
1 import {  
2   crearTablero,  
3   crearMatriz,  
4   colocarMinas,  
5   calcularNumeros  
6 } from "../board.js";  
7
```

```
8 let matrizJuego = [];  
9  
10 function iniciarAplicacion() {  
11     matrizJuego = crearMatriz(8, 8);  
12     colocarMinas(matrizJuego, 10);  
13     calcularNumeros(matrizJuego);  
14     crearTablero(8, 8, abrirCelda, alternarBandera);  
15 }  
16  
17 function abrirCelda(fila, columna) {  
18     const valor = matrizJuego[fila][columna];  
19     const celda = obtenerCeldaDOM(fila, columna);  
20  
21     if (!celda) {  
22         return;  
23     }  
24  
25     if (celda.classList.contains("abierta")) {  
26         return;  
27     }  
28  
29     if (celda.classList.contains("bandera")) {  
30         return;  
31     }  
32  
33     celda.classList.add("abierta");  
34  
35     if (valor === -1) {  
36         celda.textContent = "●";  
37         return;  
38     }  
39  
40     if (valor > 0) {  
41         celda.textContent = valor;  
42         return;  
43     }  
44  
45     celda.textContent = "";  
46  
47     for (let f = fila - 1; f <= fila + 1; f++) {  
48         for (let c = columna - 1; c <= columna + 1; c++) {  
49             if (f === fila && c === columna) {  
50                 continue;  
51             }  
52         }  
53     }  
54 }
```

```
59
60     if (
61         f >= 0 &&
62         f < matrizJuego.length &&
63         c >= 0 &&
64         c < matrizJuego[0].length
65     ) {
66         abrirCelda(f, c);
67     }
68
69 }
70 }
71
72 }
73
74 function alternarBandera(fila, columna) {
75
76     const celda = obtenerCeldaDOM(fila, columna);
77
78     if (!celda) {
79         return;
80     }
81
82     if (celda.classList.contains("abierta")) {
83         return;
84     }
85
86     if (celda.classList.contains("bandera")) {
87         celda.classList.remove("bandera");
88         celda.textContent = "";
89     } else {
90         celda.classList.add("bandera");
91         celda.textContent = "I";
92     }
93
94 }
95
96 function obtenerCeldaDOM(fila, columna) {
97
98     return document.querySelector(
99         `.celda[data-fila="${fila}"][data-columna="${columna}"]`
100     );
101
102 }
103
104 iniciarAplicacion();
```

11.9 Código completo actualizado de board.js

Después de este capítulo, `board.js` quedará así:

```
1 export function crearTablero(filas, columnas, alHacerClickCelda,  
  alHacerClickDerechoCelda) {  
2  
3   const tablero = document.querySelector("#tablero");  
4  
5   tablero.innerHTML = "";  
6  
7   for (let fila = 0; fila < filas; fila++) {  
8  
9     for (let columna = 0; columna < columnas; columna++) {  
10  
11      const celda = document.createElement("div");  
12  
13      celda.classList.add("celda");  
14  
15      celda.dataset.fila = fila;  
16      celda.dataset.columna = columna;  
17  
18      celda.addEventListener("click", () => {  
19  
20        const f = Number(celda.dataset.fila);  
21        const c = Number(celda.dataset.columna);  
22  
23        alHacerClickCelda(f, c);  
24  
25      });  
26  
27      celda.addEventListener("contextmenu", (event) => {  
28  
29        event.preventDefault();  
30  
31        const f = Number(celda.dataset.fila);  
32        const c = Number(celda.dataset.columna);  
33  
34        alHacerClickDerechoCelda(f, c);  
35  
36      });  
37  
38      tablero.appendChild(celda);  
39    }  
40  }  
41  
42 }  
43  
44 }  
45  
46 export function crearMatriz(filas, columnas) {
```

```
47
48   const matriz = [];
49
50   for (let fila = 0; fila < filas; fila++) {
51
52     const filaActual = [];
53
54     for (let columna = 0; columna < columnas; columna++) {
55       filaActual.push(0);
56     }
57
58     matriz.push(filaActual);
59
60   }
61
62   return matriz;
63 }
64
65 export function colocarMinas(matriz, cantidadMinas) {
66   let minasColocadas = 0;
67
68   while (minasColocadas < cantidadMinas) {
69
70     const fila = Math.floor(Math.random() * matriz.length);
71     const columna = Math.floor(Math.random() * matriz[0].length);
72
73     if (matriz[fila][columna] !== -1) {
74       matriz[fila][columna] = -1;
75       minasColocadas++;
76     }
77
78   }
79
80 }
81
82 }
83
84 export function contarMinasVecinas(matriz, fila, columna) {
85   let total = 0;
86
87   for (let f = fila - 1; f <= fila + 1; f++) {
88     for (let c = columna - 1; c <= columna + 1; c++) {
89
90       if (f === fila && c === columna) {
91         continue;
92       }
93
94       if (
95         f >= 0 &&
96         f < matriz.length &&
```

```

98         c >= 0 &&
99         c < matriz[0].length
100     ) {
101         if (matriz[f][c] === -1) {
102             total++;
103         }
104     }
105
106 }
107 }
108
109 return total;
110
111 }
112
113 export function calcularNumeros(matriz) {
114
115     for (let fila = 0; fila < matriz.length; fila++) {
116         for (let columna = 0; columna < matriz[fila].length; columna++) {
117
118             if (matriz[fila][columna] !== -1) {
119                 matriz[fila][columna] = contarMinasVecinas(matriz, fila,
120                     columna);
121             }
122         }
123     }
124
125 }
```

11.10 Qué hemos conseguido en este capítulo

En este capítulo hemos añadido otra de las mecánicas fundamentales del Buscaminas:

- detectar el clic derecho del ratón
- evitar el menú contextual del navegador
- poner y quitar banderas
- impedir que una celda marcada se abra por error

Con esto el juego ya ofrece una experiencia mucho más cercana a la del Buscaminas clásico.

En el siguiente capítulo implementaremos la lógica de **derrota y victoria**, de forma que el juego sepa cuándo el jugador pisa una mina y cuándo ha conseguido completar correctamente la partida.

12 Game Over

En los capítulos anteriores hemos construido gran parte del comportamiento del Buscaminas. Ya tenemos un tablero con minas, números, apertura de celdas, expansión de zonas vacías y colocación de banderas. Sin embargo, el juego todavía no sabe **cuándo termina una partida**.

En un Buscaminas real hay dos finales posibles:

- **derrota**, cuando el jugador abre una celda con una mina
- **victoria**, cuando el jugador consigue abrir todas las celdas seguras

En este capítulo vamos a añadir esa lógica. A partir de ahora el juego podrá reconocer cuándo la partida ha terminado y reaccionar en consecuencia.

Esto nos permitirá:

- impedir que el jugador siga interactuando cuando ya ha perdido o ganado
- mostrar visualmente el final de la partida
- preparar el juego para añadir más adelante temporizador, puntuaciones y reinicio

12.1 Necesitamos saber si la partida sigue activa

Hasta ahora el programa permite seguir abriendo celdas indefinidamente. Incluso si el jugador pulsa una mina, no existe un estado formal de “partida terminada”.

Para resolverlo vamos a crear una variable que indique si la partida sigue activa.

En `main.js` añadimos esta variable junto a `matrizJuego`:

```
1 let partidaActiva = true;
```

Esta variable tendrá dos posibles valores:

- **true** → la partida sigue en curso
- **false** → la partida ha terminado

La utilizaremos para bloquear las acciones del jugador cuando sea necesario.

12.2 Reiniciar el estado al empezar una nueva partida

Cada vez que llamemos a `iniciarAplicacion`, debemos asegurarnos de que la partida comienza activa.

Por tanto, añadimos esta línea al principio de la función:

```
1 partidaActiva = true;
```

La función quedará así:

```
1 function iniciarAplicacion() {  
2  
3   partidaActiva = true;  
4  
5   matrizJuego = crearMatriz(8, 8);  
6  
7   colocarMinas(matrizJuego, 10);  
8  
9   calcularNumeros(matrizJuego);  
10  
11  crearTablero(8, 8, abrirCelda, alternarBandera);  
12  
13 }
```

Con esto nos aseguramos de que cada nueva partida comienza correctamente.

12.3 Bloquear acciones si la partida ha terminado

Antes de abrir una celda o poner una bandera, debemos comprobar si la partida sigue activa.

Al principio de `abrirCelda` añadimos:

```
1 if (!partidaActiva) {  
2   return;  
3 }
```

Y al principio de `alternarBandera` añadimos lo mismo:

```
1 if (!partidaActiva) {  
2   return;  
3 }
```

De esta forma, cuando la partida termine, el jugador ya no podrá seguir interactuando con el tablero.

12.4 Detectar derrota

La derrota ocurre cuando el jugador abre una celda cuyo valor en la matriz es `-1`.

Hasta ahora, si el valor era `-1`, simplemente mostrábamos la mina.

```
1 if (valor === -1) {
```

```
2   celda.textContent = "💣";
3   return;
4 }
```

Ahora vamos a mejorar ese comportamiento.

Cuando el jugador pisa una mina debemos:

- marcar la partida como terminada
- mostrar la mina pulsada
- revelar todas las demás minas del tablero

La primera parte queda así:

```
1  if (valor === -1) {
2    celda.textContent = "💣";
3    partidaActiva = false;
4    mostrarTodasLasMinas();
5    return;
6 }
```

Con esto, en cuanto el jugador pulsa una mina, la partida termina.

12.5 Mostrar todas las minas al perder

Ahora necesitamos crear una función que recorra toda la matriz y revele todas las minas.

Añadimos en `main.js` la siguiente función:

```
1  function mostrarTodasLasMinas() {
2
3    for (let fila = 0; fila < matrizJuego.length; fila++) {
4      for (let columna = 0; columna < matrizJuego[fila].length; columna
5        ++ ) {
6
7        if (matrizJuego[fila][columna] === -1) {
8
9          const celda = obtenerCeldaDOM(fila, columna);
10
11          if (celda) {
12            celda.classList.add("abierta");
13            celda.textContent = "💣";
14          }
15        }
16      }
17    }
18  }
19 }
```

```
20 }
```

Esta función recorre todas las posiciones de la matriz y, si encuentra una mina, la muestra en el tablero.

Así el jugador ve claramente dónde estaban todas las minas al perder.

12.6 Detectar victoria

La victoria en Buscaminas no depende de poner todas las banderas, sino de haber abierto todas las celdas que **no contienen mina**.

Esto significa que debemos comprobar cuántas celdas seguras siguen cerradas.

Si no queda ninguna, el jugador ha ganado.

12.7 Crear una función para comprobar victoria

Añadimos en `main.js` la función `comprobarVictoria`.

```
1 function comprobarVictoria() {
2
3   for (let fila = 0; fila < matrizJuego.length; fila++) {
4     for (let columna = 0; columna < matrizJuego[fila].length; columna
5         ++ ) {
6
7       if (matrizJuego[fila][columna] !== -1) {
8
9         const celda = obtenerCeldaDOM(fila, columna);
10
11        if (celda && !celda.classList.contains("abierta")) {
12          return false;
13        }
14      }
15    }
16  }
17  return true;
18
19 }
20
21 }
```

Esta función recorre todas las celdas del tablero.

Para cada celda que no sea una mina:

- busca su equivalente en el DOM
- comprueba si está abierta

Si encuentra хотя una celda segura cerrada, devuelve **false**.

Si termina de recorrer el tablero sin encontrar ninguna, devuelve **true**.

12.8 Comprobar la victoria después de abrir una celda

Ahora debemos llamar a `comprobarVictoria()` cada vez que el jugador abra una celda.

La mejor posición para hacerlo es al final de `abrirCelda`, una vez que ya se ha procesado correctamente la celda.

Añadimos este bloque justo antes del final de la función:

```
1 if (comprobarVictoria()) {  
2   partidaActiva = false;  
3   console.log(";Has ganado!");  
4 }
```

Sin embargo, hay un detalle importante: cuando una celda tiene valor `-1` o un número, la función hace **return** antes de llegar al final.

Para que la comprobación funcione correctamente en todos los casos, conviene reorganizar ligeramente la función.

12.9 Reorganizar abrirCelda para comprobar la victoria

La versión actualizada de `abrirCelda` quedará así:

```
1 function abrirCelda(fila, columna) {  
2  
3   if (!partidaActiva) {  
4     return;  
5   }  
6  
7   const valor = matrizJuego[fila][columna];  
8   const celda = obtenerCeldaDOM(fila, columna);  
9  
10  if (!celda) {  
11    return;  
12  }  
13  
14  if (celda.classList.contains("abierta")) {  
15    return;  
16  }
```

```
17
18     if (celda.classList.contains("bandera")) {
19         return;
20     }
21
22     celda.classList.add("abierta");
23
24     if (valor === -1) {
25         celda.textContent = "●";
26         partidaActiva = false;
27         mostrarTodasLasMinas();
28         return;
29     }
30
31     if (valor > 0) {
32         celda.textContent = valor;
33     } else {
34         celda.textContent = "";
35
36         for (let f = fila - 1; f <= fila + 1; f++) {
37             for (let c = columna - 1; c <= columna + 1; c++) {
38
39                 if (f === fila && c === columna) {
40                     continue;
41                 }
42
43                 if (
44                     f >= 0 &&
45                     f < matrizJuego.length &&
46                     c >= 0 &&
47                     c < matrizJuego[0].length
48                 ) {
49                     abrirCelda(f, c);
50                 }
51             }
52         }
53     }
54 }
55
56 if (comprobarVictoria()) {
57     partidaActiva = false;
58     console.log("¡Has ganado!");
59 }
60
61 }
```

Fíjate en que ahora:

- si es mina, se termina la partida inmediatamente
- si es número, se muestra

- si es 0, se expande
- después de procesar una celda segura, se comprueba si ya se ha ganado

12.10 Bloquear banderas cuando la partida termina

Como ya comentamos antes, también debemos impedir que el jugador coloque banderas después de ganar o perder.

La función `alternarBandera` queda así:

```
1 function alternarBandera(fila, columna) {
2
3   if (!partidaActiva) {
4     return;
5   }
6
7   const celda = obtenerCeldaDOM(fila, columna);
8
9   if (!celda) {
10    return;
11  }
12
13  if (celda.classList.contains("abierta")) {
14    return;
15  }
16
17  if (celda.classList.contains("bandera")) {
18    celda.classList.remove("bandera");
19    celda.textContent = "";
20  } else {
21    celda.classList.add("bandera");
22    celda.textContent = "I";
23  }
24
25 }
```

12.11 Qué ocurre ahora en una partida

Con estas mejoras, el juego ya puede finalizar correctamente.

12.11.1 Si el jugador pulsa una mina

- se muestra la mina pulsada
- se revelan todas las demás minas

- la partida termina
- ya no se pueden abrir más celdas ni poner banderas

12.11.2 Si el jugador abre todas las celdas seguras

- el programa detecta que ya no quedan casillas sin abrir
- marca la partida como terminada
- impide seguir interactuando con el tablero

Todavía no mostramos un mensaje visual de victoria o derrota, pero la lógica ya está implementada.

12.12 Código completo actualizado de main.js

Después de este capítulo, `main.js` debería quedar así:

```
1 import {
2   crearTablero,
3   crearMatriz,
4   colocarMinas,
5   calcularNumeros
6 } from "./board.js";
7
8 let matrizJuego = [];
9 let partidaActiva = true;
10
11 function iniciarAplicacion() {
12
13   partidaActiva = true;
14
15   matrizJuego = crearMatriz(8, 8);
16
17   colocarMinas(matrizJuego, 10);
18
19   calcularNumeros(matrizJuego);
20
21   crearTablero(8, 8, abrirCelda, alternarBandera);
22 }
23
24 function abrirCelda(fila, columna) {
25
26   if (!partidaActiva) {
27     return;
28   }
29
30   const valor = matrizJuego[fila][columna];
31   const celda = obtenerCeldaDOM(fila, columna);
```



```
33
34     if (!celda) {
35         return;
36     }
37
38     if (celda.classList.contains("abierta")) {
39         return;
40     }
41
42     if (celda.classList.contains("bandera")) {
43         return;
44     }
45
46     celda.classList.add("abierta");
47
48     if (valor === -1) {
49         celda.textContent = "●";
50         partidaActiva = false;
51         mostrarTodasLasMinas();
52         return;
53     }
54
55     if (valor > 0) {
56         celda.textContent = valor;
57     } else {
58         celda.textContent = "";
59
60         for (let f = fila - 1; f <= fila + 1; f++) {
61             for (let c = columna - 1; c <= columna + 1; c++) {
62
63                 if (f === fila && c === columna) {
64                     continue;
65                 }
66
67                 if (
68                     f >= 0 &&
69                     f < matrizJuego.length &&
70                     c >= 0 &&
71                     c < matrizJuego[0].length
72                 ) {
73                     abrirCelda(f, c);
74                 }
75             }
76         }
77     }
78 }
79
80 if (comprobarVictoria()) {
81     partidaActiva = false;
82     console.log("¡Has ganado!");
83 }
```

```
84
85 }
86
87 function alternarBandera(fila, columna) {
88
89     if (!partidaActiva) {
90         return;
91     }
92
93     const celda = obtenerCeldaDOM(fila, columna);
94
95     if (!celda) {
96         return;
97     }
98
99     if (celda.classList.contains("abierta")) {
100         return;
101     }
102
103     if (celda.classList.contains("bandera")) {
104         celda.classList.remove("bandera");
105         celda.textContent = "";
106     } else {
107         celda.classList.add("bandera");
108         celda.textContent = "🚩";
109     }
110
111 }
112
113 function mostrarTodasLasMinas() {
114
115     for (let fila = 0; fila < matrizJuego.length; fila++) {
116         for (let columna = 0; columna < matrizJuego[fila].length; columna
117             ++ ) {
118
119             if (matrizJuego[fila][columna] === -1) {
120
121                 const celda = obtenerCeldaDOM(fila, columna);
122
123                 if (celda) {
124                     celda.classList.add("abierta");
125                     celda.textContent = "💣";
126                 }
127             }
128
129         }
130     }
131
132 }
133
```

```
134 function comprobarVictoria() {
135
136     for (let fila = 0; fila < matrizJuego.length; fila++) {
137         for (let columna = 0; columna < matrizJuego[fila].length; columna
138             ++ ) {
139
140             if (matrizJuego[fila][columna] !== -1) {
141
142                 const celda = obtenerCeldaDOM(fila, columna);
143
144                 if (celda && !celda.classList.contains("abierta")) {
145                     return false;
146                 }
147             }
148         }
149     }
150 }
151
152 return true;
153 }
154
155 function obtenerCeldaDOM(fila, columna) {
156
157     return document.querySelector(
158         `.celda[data-fila="${fila}"][data-columna="${columna}"]`
159     );
160 }
161
162 }
163
164 iniciarAplicacion();
```

12.13 Qué hemos conseguido en este capítulo

En este capítulo hemos añadido la lógica que permite finalizar correctamente una partida de Buscaminas.

Ahora el proyecto ya es capaz de:

- detectar cuándo el jugador pisa una mina
- revelar todas las minas al perder
- impedir seguir jugando tras la derrota
- comprobar si todas las celdas seguras han sido abiertas
- detectar correctamente la victoria
- bloquear la interacción una vez terminada la partida

Con esto, el juego ya tiene una estructura jugable completa.

En el siguiente capítulo añadiremos dos elementos muy importantes de la interfaz: **el contador de tiempo y el reinicio de partida**, para que el juego se parezca todavía más al proyecto final.

13 Timers

En el capítulo anterior hemos conseguido que el juego ya tenga una lógica completa de victoria y derrota. El jugador puede abrir celdas, colocar banderas, perder si pisa una mina y ganar si descubre todas las celdas seguras.

Ahora vamos a incorporar dos elementos muy importantes para que el Buscaminas se parezca mucho más al proyecto final:

- un **temporizador**
- un **botón para reiniciar la partida**

El temporizador permitirá medir cuánto tiempo tarda el jugador en completar la partida. El botón de reinicio permitirá comenzar una nueva partida sin necesidad de recargar manualmente la página.

Estas dos mejoras son muy habituales en juegos de navegador y nos ayudarán a entender dos conceptos muy importantes en JavaScript:

- la ejecución periódica de código con `setInterval` (**el timer**)
- la gestión de eventos en elementos de la interfaz

13.1 El panel superior del juego

En nuestro `index.html` ya existen varios elementos que forman parte del panel de control del juego.

Entre ellos están:

- el contador de minas
- el botón central con la carita
- el contador de tiempo

En este capítulo vamos a empezar a usar dos de esos elementos:

- `#reloj`
- `#carita`

El reloj mostrará los segundos que lleva la partida. La carita actuará como botón para reiniciar.

13.2 Qué es un temporizador en JavaScript

JavaScript permite ejecutar código repetidamente cada cierto tiempo mediante la función:

```
1 setInterval()
```

Su funcionamiento general es este:

```
1 const temporizador = setInterval(() => {  
2   console.log("Ha pasado un segundo");  
3 }, 1000);
```

El valor 1000 indica mil milisegundos, es decir, un segundo.

El problema es que, una vez iniciado, el temporizador seguirá ejecutándose hasta que lo detengamos. Para detenerlo utilizamos:

```
1 clearInterval(temporizador);
```

En nuestro juego necesitaremos:

- iniciar el reloj al comenzar una partida
- detenerlo cuando la partida termine
- reiniciarlo cuando el jugador empiece otra partida

13.3 Variables para controlar el tiempo

Vamos a declarar en `main.js` dos nuevas variables de módulo:

```
1 let tiempo = 0;  
2 let temporizador = null;
```

Estas variables tendrán el siguiente papel:

- `tiempo` almacenará los segundos transcurridos
- `temporizador` guardará el identificador del `setInterval`

De esta forma podremos detener el intervalo cuando queramos.

Ahora la parte superior de `main.js` quedará así:

```
1 let matrizJuego = [];  
2 let partidaActiva = true;  
3 let tiempo = 0;  
4 let temporizador = null;
```

13.4 Mostrar el tiempo en pantalla

Necesitamos una función que actualice el contador visual del reloj.

Añadimos en `main.js` esta función:

```
1 function actualizarReloj() {  
2  
3     const reloj = document.querySelector("#reloj");  
4  
5     if (reloj) {  
6         reloj.textContent = tiempo;  
7     }  
8  
9 }
```

Esta función busca el elemento del DOM con id `reloj` y escribe en él el valor actual de la variable `tiempo`.

13.5 Iniciar el temporizador

Ahora vamos a crear una función para arrancar el reloj.

```
1 function iniciarTemporizador() {  
2  
3     detenerTemporizador();  
4  
5     temporizador = setInterval(() => {  
6         tiempo++;  
7         actualizarReloj();  
8     }, 1000);  
9  
10 }
```

Esta función hace varias cosas:

- primero detiene cualquier temporizador anterior
- crea un nuevo `setInterval`
- cada segundo incrementa la variable `tiempo`
- actualiza el valor visual del reloj

La llamada a `detenerTemporizador()` al principio es importante para evitar que se creen varios intervalos al mismo tiempo.

13.6 Detener el temporizador

Añadimos ahora la función complementaria:

```
1 function detenerTemporizador() {
```

```
2
3   if (temporizador !== null) {
4       clearInterval(temporizador);
5       temporizador = null;
6   }
7
8 }
```

Con esto podremos parar el reloj cuando el jugador gane, pierda o salga de la partida.

13.7 Reiniciar el reloj

También necesitamos una función para volver a poner el tiempo a cero.

```
1 function reiniciarReloj() {
2     tiempo = 0;
3     actualizarReloj();
4 }
```

Esta función se utilizará cada vez que comience una nueva partida.

13.8 Poner en marcha el temporizador al iniciar la partida

Ahora debemos modificar la función `iniciarAplicacion()` para que:

- ponga el tiempo a cero
- actualice el reloj
- arranque el temporizador

La función quedará así:

```
1 function iniciarAplicacion() {
2
3     partidaActiva = true;
4
5     reiniciarReloj();
6     iniciarTemporizador();
7
8     matrizJuego = crearMatriz(8, 8);
9
10    colocarMinas(matrizJuego, 10);
11
12    calcularNumeros(matrizJuego);
13
14    crearTablero(8, 8, abrirCelda, alternarBandera);
15
16 }
```


De este modo, cada nueva partida empezará con el reloj en 0 y el contador comenzará a avanzar automáticamente.

13.9 Detener el reloj al perder

Si el jugador pisa una mina, la partida termina. Por tanto, también debe detenerse el temporizador.

Buscamos este bloque dentro de `abrirCelda`:

```
1 if (valor === -1) {  
2   celda.textContent = "💣";  
3   partidaActiva = false;  
4   mostrarTodasLasMinas();  
5   return;  
6 }
```

Y lo sustituimos por este:

```
1 if (valor === -1) {  
2   celda.textContent = "💣";  
3   partidaActiva = false;  
4   detenerTemporizador();  
5   mostrarTodasLasMinas();  
6   return;  
7 }
```

Así, cuando el jugador pierde, el reloj se congela.

13.10 Detener el reloj al ganar

También debemos detener el temporizador si el jugador gana la partida.

Buscamos este bloque al final de `abrirCelda`:

```
1 if (comprobarVictoria()) {  
2   partidaActiva = false;  
3   console.log("¡Has ganado!");  
4 }
```

Y lo sustituimos por este:

```
1 if (comprobarVictoria()) {  
2   partidaActiva = false;  
3   detenerTemporizador();  
4   console.log("¡Has ganado!");  
5 }
```

De esta forma, el tiempo final de la partida queda fijado en el momento exacto en que se completa el tablero.

13.11 Añadir el botón de reinicio

Ahora vamos a usar el botón con id `carita`.

Este botón debe servir para comenzar una nueva partida cuando el usuario haga clic sobre él.

Vamos a crear una función para registrar este evento.

Añadimos en `main.js`:

```
1 function configurarBotonReinicio() {  
2  
3   const botonReinicio = document.querySelector("#carita");  
4  
5   if (botonReinicio) {  
6     botonReinicio.addEventListener("click", () => {  
7       iniciarAplicacion();  
8     });  
9   }  
10  
11 }
```

Esta función busca el botón y le asocia un evento `click` que vuelve a iniciar el juego.

13.12 Cuándo debemos llamar a configurarBotonReinicio

Esta función debe ejecutarse una sola vez cuando la aplicación arranca por primera vez.

Por tanto, al final del archivo, en lugar de llamar directamente a `iniciarAplicacion()`, haremos lo siguiente:

```
1 configurarBotonReinicio();  
2 iniciarAplicacion();
```

Así conseguimos que:

- se configure el botón
- se inicie la primera partida al cargar la página

13.13 Evitar múltiples listeners en el botón

Es importante que `configurarBotonReinicio()` se llame una sola vez. Si la llamáramos dentro de `iniciarAplicacion()`, cada reinicio añadiría un nuevo listener al mismo botón.

Eso provocaría un comportamiento incorrecto.

Por eso es mejor dejar la configuración de eventos generales fuera del flujo de reinicio.

13.14 Resultado práctico

A partir de este capítulo, el juego ya se comportará así:

- al comenzar una partida, el reloj empieza en 0
- cada segundo aumenta en una unidad
- si el jugador pierde, el reloj se detiene
- si el jugador gana, el reloj se detiene
- si el jugador pulsa la carita, comienza una nueva partida
- al reiniciar, el reloj vuelve a cero y vuelve a arrancar

Esto hace que la experiencia de uso sea mucho más cercana a la del juego real.

13.15 Código completo actualizado de main.js

Después de este capítulo, `main.js` debería quedar así:

```
1 import {
2   crearTablero,
3   crearMatriz,
4   colocarMinas,
5   calcularNumeros
6 } from "./board.js";
7
8 let matrizJuego = [];
9 let partidaActiva = true;
10 let tiempo = 0;
11 let temporizador = null;
12
13 function iniciarAplicacion() {
14   partidaActiva = true;
15
16   reiniciarReloj();
17   iniciarTemporizador();
18
19   matrizJuego = crearMatriz(8, 8);
20
21   colocarMinas(matrizJuego, 10);
22
23   calcularNumeros(matrizJuego);
24
25 }
```

```
26   crearTablero(8, 8, abrirCelda, alternarBandera);
27
28 }
29
30 function abrirCelda(fila, columna) {
31
32   if (!partidaActiva) {
33     return;
34   }
35
36   const valor = matrizJuego[fila][columna];
37   const celda = obtenerCeldaDOM(fila, columna);
38
39   if (!celda) {
40     return;
41   }
42
43   if (celda.classList.contains("abierta")) {
44     return;
45   }
46
47   if (celda.classList.contains("bandera")) {
48     return;
49   }
50
51   celda.classList.add("abierta");
52
53   if (valor === -1) {
54     celda.textContent = "●";
55     partidaActiva = false;
56     detenerTemporizador();
57     mostrarTodasLasMinas();
58     return;
59   }
60
61   if (valor > 0) {
62     celda.textContent = valor;
63   } else {
64     celda.textContent = "";
65
66     for (let f = fila - 1; f <= fila + 1; f++) {
67       for (let c = columna - 1; c <= columna + 1; c++) {
68
69         if (f === fila && c === columna) {
70           continue;
71         }
72
73         if (
74           f >= 0 &&
75           f < matrizJuego.length &&
76           c >= 0 &&
```

```
77         c < matrizJuego[0].length
78     ) {
79         abrirCelda(f, c);
80     }
81
82     }
83 }
84 }
85
86 if (comprobarVictoria()) {
87     partidaActiva = false;
88     detenerTemporizador();
89     console.log("¡Has ganado!");
90 }
91
92 }
93
94 function alternarBandera(fila, columna) {
95
96     if (!partidaActiva) {
97         return;
98     }
99
100     const celda = obtenerCeldaDOM(fila, columna);
101
102     if (!celda) {
103         return;
104     }
105
106     if (celda.classList.contains("abierta")) {
107         return;
108     }
109
110     if (celda.classList.contains("bandera")) {
111         celda.classList.remove("bandera");
112         celda.textContent = "";
113     } else {
114         celda.classList.add("bandera");
115         celda.textContent = "I";
116     }
117
118 }
119
120 function mostrarTodasLasMinas() {
121
122     for (let fila = 0; fila < matrizJuego.length; fila++) {
123         for (let columna = 0; columna < matrizJuego[fila].length; columna
124             ++ ) {
125
126             if (matrizJuego[fila][columna] === -1) {
```

```
127     const celda = obtenerCeldaDOM(fila, columna);
128
129     if (celda) {
130         celda.classList.add("abierta");
131         celda.textContent = "●";
132     }
133
134 }
135
136 }
137 }
138
139 }
140
141 function comprobarVictoria() {
142
143     for (let fila = 0; fila < matrizJuego.length; fila++) {
144         for (let columna = 0; columna < matrizJuego[fila].length; columna
145             ++ ) {
146
147             if (matrizJuego[fila][columna] !== -1) {
148
149                 const celda = obtenerCeldaDOM(fila, columna);
150
151                 if (celda && !celda.classList.contains("abierta")) {
152                     return false;
153                 }
154             }
155
156         }
157     }
158
159     return true;
160 }
161
162
163 function obtenerCeldaDOM(fila, columna) {
164
165     return document.querySelector(
166         `.celda[data-fila="${fila}"][data-columna="${columna}"]`
167     );
168 }
169
170
171 function actualizarReloj() {
172
173     const reloj = document.querySelector("#reloj");
174
175     if (reloj) {
176         reloj.textContent = tiempo;
```

```
177     }
178
179 }
180
181 function iniciarTemporizador() {
182     detenerTemporizador();
183
184     temporizador = setInterval(() => {
185         tiempo++;
186         actualizarReloj();
187     }, 1000);
188 }
189
190 }
191
192 function detenerTemporizador() {
193
194     if (temporizador !== null) {
195         clearInterval(temporizador);
196         temporizador = null;
197     }
198
199 }
200
201 function reiniciarReloj() {
202     tiempo = 0;
203     actualizarReloj();
204 }
205
206 function configurarBotonReinicio() {
207
208     const botonReinicio = document.querySelector("#carita");
209
210     if (botonReinicio) {
211         botonReinicio.addEventListener("click", () => {
212             iniciarAplicacion();
213         });
214     }
215
216 }
217
218 configurarBotonReinicio();
219 iniciarAplicacion();
```

13.16 Qué hemos conseguido en este capítulo

En este capítulo hemos añadido dos piezas fundamentales de la interfaz del juego:

- un temporizador funcional

- un botón de reinicio de la partida

Ahora el proyecto ya es capaz de:

- contar los segundos que dura una partida
- mostrar ese tiempo en pantalla
- detener el reloj al ganar o perder
- reiniciar el tablero con un clic sobre la carita
- comenzar una nueva partida sin recargar la página

Con esto el Buscaminas ya se parece mucho más al proyecto final.

En el siguiente capítulo incorporaremos **el contador de minas y la selección de dificultad**, para que la aplicación permita cambiar entre distintos tamaños de tablero y distintas cantidades de minas.

14 Añadir el contador de minas y la selección de dificultad

En el capítulo anterior hemos incorporado el temporizador y el reinicio de partida. El juego ya tiene una estructura bastante completa, pero todavía faltan dos elementos fundamentales para acercarnos al proyecto final:

- el **contador de minas**
- la **selección de dificultad desde el menú**

El contador de minas permitirá mostrar al jugador cuántas minas hay en la partida. La selección de dificultad permitirá cambiar entre distintos tamaños de tablero y distintas cantidades de minas, haciendo que el juego sea más variado y más parecido a una aplicación real.

En este capítulo vamos a preparar la lógica necesaria para manejar diferentes configuraciones de partida y actualizar la interfaz en función de la dificultad elegida.

14.1 Qué dificultades tendrá el juego

Nuestro Buscaminas tendrá tres niveles de dificultad:

- fácil
- media
- difícil

Cada nivel tendrá una configuración distinta de:

- número de filas
- número de columnas
- número de minas

Por ejemplo, podemos empezar con esta propuesta:

```
1 easy → 8 filas, 8 columnas, 10 minas
2 medium → 12 filas, 12 columnas, 25 minas
3 hard → 16 filas, 16 columnas, 40 minas
```

Más adelante podrías ajustar estos valores, pero de momento son adecuados para construir el juego.

14.2 Crear un objeto de configuración

Vamos a declarar en `main.js` un objeto que almacene las distintas dificultades.

Añadimos este bloque al principio del archivo, después de las variables globales:

```
1  const dificultades = {
2    easy: {
3      filas: 8,
4      columnas: 8,
5      minas: 10,
6      titulo: "Partida fácil"
7    },
8    medium: {
9      filas: 12,
10     columnas: 12,
11     minas: 25,
12     titulo: "Partida normal"
13   },
14   hard: {
15     filas: 16,
16     columnas: 16,
17     minas: 40,
18     titulo: "Partida difícil"
19   }
20 };
```

Este objeto nos permitirá acceder fácilmente a la configuración de cada modo de juego.

Por ejemplo:

```
1  dificultades.easy.minas
```

devolvería 10.

14.3 Guardar la dificultad actual

Ahora necesitamos una variable que indique qué dificultad está activa en cada momento.

Añadimos en `main.js`:

```
1  let dificultadActual = "easy";
```

Esto significa que, cuando se cargue la aplicación, la partida comenzará por defecto en modo fácil.

14.4 Modificar `iniciarAplicacion` para usar la dificultad elegida

Hasta ahora `iniciarAplicacion()` creaba siempre una partida de 8x8 con 10 minas.

Vamos a cambiarlo para que tome los datos desde el objeto `dificultades`.

La función quedará así:

```
1 function iniciarAplicacion() {
2
3     partidaActiva = true;
4
5     reiniciarReloj();
6     iniciarTemporizador();
7
8     const configuracion = dificultades[dificultadActual];
9
10    matrizJuego = crearMatriz(configuracion.filas, configuracion.columnas
11                               );
12    colocarMinas(matrizJuego, configuracion.minas);
13
14    calcularNumeros(matrizJuego);
15
16    crearTablero(
17        configuracion.filas,
18        configuracion.columnas,
19        abrirCelda,
20        alternarBandera
21    );
22
23    actualizarContadorMinas();
24    actualizarTituloPartida();
25
26 }
```

Ahora el tamaño del tablero y el número de minas dependen de la dificultad seleccionada.

14.5 Mostrar el número de minas en pantalla

En el HTML ya existe un elemento con id `minas` que actuará como contador visual.

Necesitamos una función que actualice su contenido.

Añadimos en `main.js`:

```
1 function actualizarContadorMinas() {
2
3     const contadorMinas = document.querySelector("#minas");
4     const configuracion = dificultades[dificultadActual];
5
6     if (contadorMinas) {
7         contadorMinas.textContent = configuracion.minas;
8     }
9
10 }
```

Esta función busca el elemento del DOM y muestra el número de minas correspondiente a la dificultad actual.

14.6 Actualizar el título de la partida

En el proyecto también existe un elemento con id `tituloPartida`.

Vamos a utilizarlo para mostrar el nombre del nivel seleccionado.

Añadimos esta función:

```
1 function actualizarTituloPartida() {  
2  
3   const titulo = document.querySelector("#tituloPartida");  
4   const configuracion = dificultades[dificultadActual];  
5  
6   if (titulo) {  
7     titulo.textContent = configuracion.titulo;  
8   }  
9  
10 }
```

De esta forma, cuando el usuario cambie de dificultad, el título del panel de partida se actualizará automáticamente.

14.7 Detectar los clics del menú de dificultad

En el `index.html` ya existen botones del menú superior con estos identificadores:

- `menu_partida_facil`
- `menu_partida_media`
- `menu_partida_dificil`

Cada uno de ellos tiene además un atributo `data-difficulty`.

Ahora debemos registrar eventos sobre esos botones para que cambien la dificultad actual e inicien una nueva partida.

14.8 Crear una función para configurar el menú

Añadimos en `main.js` esta nueva función:

```
1 function configurarMenuDificultad() {  
2
```

```
3  const botones = document.querySelectorAll("[data-difficulty]");
4
5  botones.forEach((boton) => {
6    boton.addEventListener("click", () => {
7
8      const nuevaDificultad = boton.dataset.difficulty;
9
10     if (!nuevaDificultad) {
11       return;
12     }
13
14     dificultadActual = nuevaDificultad;
15     iniciarAplicacion();
16
17   });
18 });
19
20 }
```

Vamos a entender qué hace este código:

- selecciona todos los elementos que tengan `data-difficulty`
- recorre esa lista con `forEach`
- añade un `click` a cada botón
- obtiene la dificultad elegida desde `dataset`
- actualiza la variable `dificultadActual`
- inicia una nueva partida con esa configuración

14.9 Qué significa usar dataset aquí

En el HTML, un botón puede tener algo como esto:

```
1 <button data-difficulty="easy">Fácil</button>
```

Desde JavaScript podemos leer ese valor así:

```
1 boton.dataset.difficulty
```

Esto nos evita tener que escribir una función distinta para cada botón.

Es una forma muy cómoda y moderna de conectar HTML y JavaScript.

14.10 Configurar también el botón de inicio fácil

Como ahora la dificultad depende del menú, conviene dejar claro qué ocurre al cargar la página por primera vez.

La variable:

```
1 let dificultadActual = "easy";
```

hará que la primera partida sea fácil.

Y más adelante, cuando el usuario pulse un botón del menú, esa dificultad cambiará.

14.11 Cuándo llamar a configurarMenuDificultad

Esta función, igual que `configurarBotonReinicio`, debe ejecutarse una sola vez al arrancar la aplicación.

Por tanto, al final del archivo haremos:

```
1 configurarBotonReinicio();  
2 configurarMenuDificultad();  
3 iniciarAplicacion();
```

De este modo:

- se configuran los eventos generales
- se inicia la primera partida
- el juego ya queda listo para reaccionar a los cambios de dificultad

14.12 Qué ocurre ahora cuando el usuario pulsa una dificultad

A partir de este momento, si el jugador pulsa por ejemplo el botón “Difícil”, el programa hará lo siguiente:

1. leerá `data-difficulty="hard"`
2. actualizará `dificultadActual`
3. reiniciará el reloj
4. generará una nueva matriz con más filas y columnas
5. colocará el número de minas correspondiente
6. dibujará el nuevo tablero
7. actualizará el contador de minas
8. actualizará el título de la partida

Esto hace que el juego ya sea mucho más flexible y cercano a una aplicación completa.

14.13 Código completo actualizado de main.js

Después de este capítulo, `main.js` debería quedar así:

```
1  import {
2    crearTablero,
3    crearMatriz,
4    colocarMinas,
5    calcularNumeros
6  } from "./board.js";
7
8  let matrizJuego = [];
9  let partidaActiva = true;
10 let tiempo = 0;
11 let temporizador = null;
12 let dificultadActual = "easy";
13
14 const dificultades = {
15   easy: {
16     filas: 8,
17     columnas: 8,
18     minas: 10,
19     titulo: "Partida fácil"
20   },
21   medium: {
22     filas: 12,
23     columnas: 12,
24     minas: 25,
25     titulo: "Partida normal"
26   },
27   hard: {
28     filas: 16,
29     columnas: 16,
30     minas: 40,
31     titulo: "Partida difícil"
32   }
33 };
34
35 function iniciarAplicacion() {
36   partidaActiva = true;
37   reiniciarReloj();
38   iniciarTemporizador();
39
40   const configuracion = dificultades[dificultadActual];
41
42   matrizJuego = crearMatriz(configuracion.filas, configuracion.columnas);
43
44   colocarMinas(matrizJuego, configuracion.minas);
45
46 }
```

```
47
48     calcularNumeros(matrizJuego);
49
50     crearTablero(
51         configuracion.filas,
52         configuracion.columnas,
53         abrirCelda,
54         alternarBandera
55     );
56
57     actualizarContadorMinas();
58     actualizarTituloPartida();
59
60 }
61
62 function abrirCelda(fila, columna) {
63
64     if (!partidaActiva) {
65         return;
66     }
67
68     const valor = matrizJuego[fila][columna];
69     const celda = obtenerCeldaDOM(fila, columna);
70
71     if (!celda) {
72         return;
73     }
74
75     if (celda.classList.contains("abierta")) {
76         return;
77     }
78
79     if (celda.classList.contains("bandera")) {
80         return;
81     }
82
83     celda.classList.add("abierta");
84
85     if (valor === -1) {
86         celda.textContent = "●";
87         partidaActiva = false;
88         detenerTemporizador();
89         mostrarTodasLasMinas();
90         return;
91     }
92
93     if (valor > 0) {
94         celda.textContent = valor;
95     } else {
96         celda.textContent = "";
97     }
```



```

98     for (let f = fila - 1; f <= fila + 1; f++) {
99         for (let c = columna - 1; c <= columna + 1; c++) {
100
101             if (f === fila && c === columna) {
102                 continue;
103             }
104
105             if (
106                 f >= 0 &&
107                 f < matrizJuego.length &&
108                 c >= 0 &&
109                 c < matrizJuego[0].length
110             ) {
111                 abrirCelda(f, c);
112             }
113         }
114     }
115 }
116 }
117
118 if (comprobarVictoria()) {
119     partidaActiva = false;
120     detenerTemporizador();
121     console.log(";Has ganado!");
122 }
123
124 }
125
126 function alternarBandera(fila, columna) {
127
128     if (!partidaActiva) {
129         return;
130     }
131
132     const celda = obtenerCeldaDOM(fila, columna);
133
134     if (!celda) {
135         return;
136     }
137
138     if (celda.classList.contains("abierta")) {
139         return;
140     }
141
142     if (celda.classList.contains("bandera")) {
143         celda.classList.remove("bandera");
144         celda.textContent = "";
145     } else {
146         celda.classList.add("bandera");
147         celda.textContent = "1";
148     }

```

```
149
150 }
151
152 function mostrarTodasLasMinas() {
153
154     for (let fila = 0; fila < matrizJuego.length; fila++) {
155         for (let columna = 0; columna < matrizJuego[fila].length; columna
156             ++ ) {
157
158             if (matrizJuego[fila][columna] === -1) {
159
160                 const celda = obtenerCeldaDOM(fila, columna);
161
162                 if (celda) {
163                     celda.classList.add("abierta");
164                     celda.textContent = "●";
165                 }
166             }
167         }
168     }
169 }
170
171 }
172
173 function comprobarVictoria() {
174
175     for (let fila = 0; fila < matrizJuego.length; fila++) {
176         for (let columna = 0; columna < matrizJuego[fila].length; columna
177             ++ ) {
178
179             if (matrizJuego[fila][columna] !== -1) {
180
181                 const celda = obtenerCeldaDOM(fila, columna);
182
183                 if (celda && !celda.classList.contains("abierta")) {
184                     return false;
185                 }
186             }
187         }
188     }
189 }
190
191 return true;
192 }
193
194 function obtenerCeldaDOM(fila, columna) {
195
196     return document.querySelector(
```

```
198     `.celda[data-fila="${fila}"][data-columna="${columna}"]`  
199   );  
200  
201 }  
202  
203 function actualizarReloj() {  
204  
205     const reloj = document.querySelector("#reloj");  
206  
207     if (reloj) {  
208         reloj.textContent = tiempo;  
209     }  
210  
211 }  
212  
213 function iniciarTemporizador() {  
214  
215     detenerTemporizador();  
216  
217     temporizador = setInterval(() => {  
218         tiempo++;  
219         actualizarReloj();  
220     }, 1000);  
221  
222 }  
223  
224 function detenerTemporizador() {  
225  
226     if (temporizador !== null) {  
227         clearInterval(temporizador);  
228         temporizador = null;  
229     }  
230  
231 }  
232  
233 function reiniciarReloj() {  
234     tiempo = 0;  
235     actualizarReloj();  
236 }  
237  
238 function actualizarContadorMinas() {  
239  
240     const contadorMinas = document.querySelector("#minas");  
241     const configuracion = dificultades[dificultadActual];  
242  
243     if (contadorMinas) {  
244         contadorMinas.textContent = configuracion.minas;  
245     }  
246  
247 }  
248
```

```
249 function actualizarTituloPartida() {
250
251     const titulo = document.querySelector("#tituloPartida");
252     const configuracion = dificultades[dificultadActual];
253
254     if (titulo) {
255         titulo.textContent = configuracion.titulo;
256     }
257
258 }
259
260 function configurarBotonReinicio() {
261
262     const botonReinicio = document.querySelector("#carita");
263
264     if (botonReinicio) {
265         botonReinicio.addEventListener("click", () => {
266             iniciarAplicacion();
267         });
268     }
269
270 }
271
272 function configurarMenuDificultad() {
273
274     const botones = document.querySelectorAll("[data-difficulty]");
275
276     botones.forEach((boton) => {
277         boton.addEventListener("click", () => {
278
279             const nuevaDificultad = boton.dataset.difficulty;
280
281             if (!nuevaDificultad) {
282                 return;
283             }
284
285             dificultadActual = nuevaDificultad;
286             iniciarAplicacion();
287
288         });
289     });
290
291 }
292
293 configurarBotonReinicio();
294 configurarMenuDificultad();
295 iniciarAplicacion();
```

14.14 Qué hemos conseguido en este capítulo

En este capítulo hemos añadido dos elementos muy importantes para la interfaz del juego:

- el contador visual de minas
- la selección de dificultad desde el menú

Ahora el proyecto ya es capaz de:

- iniciar partidas con distintos tamaños de tablero
- cambiar el número de minas según la dificultad
- actualizar el título de la partida
- mostrar el número de minas correspondiente
- reiniciar automáticamente el juego al cambiar de nivel

Con esto el Buscaminas se acerca mucho más a la estructura del proyecto final.

En el siguiente capítulo añadiremos la **navegación entre paneles** de la aplicación, para que los botones del menú permitan mostrar Inicio, Partida, Puntuaciones, Ayuda y Licencia.

15 Navegación entre paneles de la aplicación

Hasta ahora nos hemos centrado casi exclusivamente en la lógica interna del juego: creación del tablero, minas, números, banderas, temporizador, reinicio y dificultades. Sin embargo, el proyecto final no es solo una pantalla con un tablero. También incluye varios apartados accesibles desde el menú superior.

En la aplicación existen distintos paneles:

- Inicio
- Partida
- Puntuaciones
- Ayuda
- Licencia

El objetivo de este capítulo es conseguir que, al pulsar los botones del menú, se muestre el panel correspondiente y se oculten los demás.

Con esto empezaremos a transformar el juego en una aplicación web completa, con navegación interna y distintas secciones.

15.1 Qué entendemos por panel

En este proyecto, un panel es una sección del HTML que representa una parte concreta de la interfaz.

Por ejemplo:

- el panel de inicio muestra una bienvenida
- el panel de partida contiene el tablero
- el panel de puntuaciones mostrará la clasificación
- el panel de ayuda explica cómo jugar
- el panel de licencia indica el uso del material

En el `index.html`, estos paneles ya existen como elementos `<section>` con un identificador propio.

Por ejemplo:

```
1 <section id="panel_inicio" class="panel">
```

o

```
1 <section id="panel_partida" class="panel hidden">
```

Nuestro trabajo será controlar desde JavaScript cuál de ellos está visible en cada momento.

15.2 La idea general de la navegación

La navegación será muy sencilla.

Cuando el usuario pulse un botón del menú:

- se ocultarán todos los paneles
- se mostrará únicamente el panel seleccionado

Para conseguirlo utilizaremos dos elementos clave:

- una clase CSS llamada `hidden`
 - una función JavaScript que muestre un panel concreto
-

15.3 Cómo ocultar y mostrar paneles

En el HTML ya se utiliza la clase `hidden` para ocultar algunos paneles.

Por ejemplo:

```
1 <section id="panel_puntuaciones" class="panel hidden">
```

Si un elemento tiene la clase `hidden`, no debe mostrarse en pantalla.

Normalmente esto se consigue en CSS con una regla como esta:

```
1 .hidden {  
2   display: none;  
3 }
```

Más adelante podremos revisar el CSS si fuera necesario, pero desde JavaScript lo único que haremos será añadir o quitar esa clase.

15.4 Crear una función para mostrar un panel

Vamos a empezar creando en `main.js` una función llamada `mostrarPanel`.

```
1 function mostrarPanel(idPanel) {  
2  
3   const paneles = document.querySelectorAll(".panel");  
4  
5   paneles.forEach((panel) => {  
6     panel.classList.add("hidden");  
7   });  
8  
9   const panelActivo = document.querySelector(`#${idPanel}`);  
10  
11   if (panelActivo) {  
12     panelActivo.classList.remove("hidden");  
13   }  
14  
15 }
```

Esta función hace tres cosas:

1. selecciona todos los paneles
2. añade la clase `hidden` a todos ellos
3. busca el panel cuyo id hemos indicado y le quita la clase `hidden`

Con esto conseguimos que siempre haya un único panel visible.

15.5 Cómo funciona `querySelectorAll(".panel")`

La línea:

```
1 const paneles = document.querySelectorAll(".panel");
```

selecciona todos los elementos que tengan la clase `panel`.

En nuestro proyecto, eso incluye:

- `panel_inicio`
- `panel_partida`
- `panel_puntuaciones`
- `panel_ayuda`
- `panel_licencia`

Después usamos `forEach` para recorrerlos uno por uno y ocultarlos.

15.6 Mostrar el panel inicial al cargar la aplicación

Cuando la aplicación se carga por primera vez, conviene mostrar el panel de inicio.

Por tanto, al final de `iniciarAplicacion()` añadiremos una llamada a `mostrarPanel`, pero aquí hay que tener cuidado.

No queremos que cada nueva partida nos devuelva al panel de inicio, porque cuando el usuario elige una dificultad debe ver el panel de partida.

Por eso, de momento no pondremos `mostrarPanel` dentro de `iniciarAplicacion()`. En lugar de eso, controlaremos la navegación desde eventos del menú.

15.7 Crear una función para configurar la navegación

Ahora vamos a registrar eventos sobre los botones del menú superior.

En el HTML ya existen botones como estos:

```
1 <button id="menu_inicio" data-panel="panel_inicio">Inicio</button>
2 <button id="menu_puntuaciones" data-panel="panel_puntuaciones">
  Puntuaciones</button>
3 <button id="menu_ayuda" data-panel="panel_ayuda">Ayuda</button>
4 <button id="menu_licencia" data-panel="panel_licencia">Licencia</button>
  >
```

Todos ellos comparten algo muy útil: tienen un atributo `data-panel`.

Eso nos permitirá crear una única función para todos.

Añadimos en `main.js`:

```
1 function configurarNavegacion() {
2
3   const botones = document.querySelectorAll("[data-panel]");
4
5   botones.forEach((boton) => {
6     boton.addEventListener("click", () => {
7
8       const idPanel = boton.dataset.panel;
```

```
9
10     if (!idPanel) {
11         return;
12     }
13
14     mostrarPanel(idPanel);
15
16     });
17 });
18
19 }
```

Con esto, cada botón del menú mostrará el panel indicado en su atributo `data-panel`.

15.8 Qué ocurre con los botones de dificultad

Hay un detalle importante: los botones de dificultad también tienen `data-panel="panel_partida"` y además tienen `data-difficulty`.

Eso significa que, cuando el usuario pulse uno de esos botones:

- la función de navegación mostrará el panel de partida
- la función de dificultad cambiará la configuración del juego e iniciará una nueva partida

Esto no es un problema. Al contrario, es justo lo que necesitamos.

Por ejemplo, si el usuario pulsa “Difícil”, deben ocurrir dos cosas:

- mostrarse el panel de partida
- iniciarse una nueva partida difícil

Y eso se consigue precisamente porque ambos comportamientos conviven.

15.9 Qué pasa con el temporizador al salir del panel de partida

En el proyecto final ya trabajamos el caso en el que, si el usuario abandona la partida para ir a Inicio, Ayuda, Licencia o Puntuaciones, el temporizador debe detenerse y resetearse.

Como estamos construyendo el tutorial paso a paso, aquí vamos a introducir primero la navegación básica. En el siguiente ajuste refinaremos el comportamiento para que salir del panel de partida también pare el juego.

De momento nos centraremos en conseguir que la navegación funcione visualmente.

15.10 Configurar la navegación al arrancar

Igual que hicimos con el botón de reinicio y con el menú de dificultad, esta configuración debe ejecutarse una sola vez al arrancar la aplicación.

Por tanto, al final del archivo sustituimos:

```
1 configurarBotonReinicio();
2 configurarMenuDificultad();
3 iniciarAplicacion();
```

por:

```
1 configurarBotonReinicio();
2 configurarMenuDificultad();
3 configurarNavegacion();
4 mostrarPanel("panel_inicio");
5 iniciarAplicacion();
```

Sin embargo, esta secuencia presenta un pequeño matiz.

Si mostramos primero `panel_inicio` y después el usuario no ha pulsado ninguna dificultad, el tablero puede haberse inicializado en segundo plano aunque no sea visible. Esto no rompe nada, pero no es la experiencia más limpia.

La alternativa es aceptarlo de momento, porque simplifica mucho la explicación, y más adelante ya refinaremos el flujo para que la partida empiece cuando corresponda.

Para un tutorial guiado en este punto, esta solución es suficientemente clara.

15.11 Qué comportamiento tendrá ahora la aplicación

A partir de este momento:

- al cargar la aplicación, se mostrará el panel de inicio
- si el usuario pulsa Inicio, se mostrará ese panel
- si pulsa Puntuaciones, se mostrará el panel de puntuaciones
- si pulsa Ayuda, se mostrará el panel de ayuda

- si pulsa Licencia, se mostrará el panel de licencia
- si pulsa Fácil, Normal o Difícil, se mostrará el panel de partida

Es decir, la interfaz ya empezará a funcionar como una aplicación multipanel.

15.12 Código completo actualizado de main.js

Después de este capítulo, `main.js` debería quedar así:

```
1 import {
2   crearTablero,
3   crearMatriz,
4   colocarMinas,
5   calcularNumeros
6 } from "./board.js";
7
8 let matrizJuego = [];
9 let partidaActiva = true;
10 let tiempo = 0;
11 let temporizador = null;
12 let dificultadActual = "easy";
13
14 const dificultades = {
15   easy: {
16     filas: 8,
17     columnas: 8,
18     minas: 10,
19     titulo: "Partida fácil"
20   },
21   medium: {
22     filas: 12,
23     columnas: 12,
24     minas: 25,
25     titulo: "Partida normal"
26   },
27   hard: {
28     filas: 16,
29     columnas: 16,
30     minas: 40,
31     titulo: "Partida difícil"
32   }
33 };
34
35 function iniciarAplicacion() {
36
37   partidaActiva = true;
```

```
38
39     reiniciarReloj();
40     iniciarTemporizador();
41
42     const configuracion = dificultades[dificultadActual];
43
44     matrizJuego = crearMatriz(configuracion.filas, configuracion.columnas
45         );
46     colocarMinas(matrizJuego, configuracion.minas);
47
48     calcularNumeros(matrizJuego);
49
50     crearTablero(
51         configuracion.filas,
52         configuracion.columnas,
53         abrirCelda,
54         alternarBandera
55     );
56
57     actualizarContadorMinas();
58     actualizarTituloPartida();
59 }
60
61 function abrirCelda(fila, columna) {
62
63     if (!partidaActiva) {
64         return;
65     }
66
67     const valor = matrizJuego[fila][columna];
68     const celda = obtenerCeldaDOM(fila, columna);
69
70     if (!celda) {
71         return;
72     }
73
74     if (celda.classList.contains("abierta")) {
75         return;
76     }
77
78     if (celda.classList.contains("bandera")) {
79         return;
80     }
81
82     celda.classList.add("abierta");
83
84     if (valor === -1) {
85         celda.textContent = "●";
86         partidaActiva = false;
87     }
```

```
88     detenerTemporizador();
89     mostrarTodasLasMinas();
90     return;
91 }
92
93 if (valor > 0) {
94     celda.textContent = valor;
95 } else {
96     celda.textContent = "";
97
98     for (let f = fila - 1; f <= fila + 1; f++) {
99         for (let c = columna - 1; c <= columna + 1; c++) {
100
101             if (f === fila && c === columna) {
102                 continue;
103             }
104
105             if (
106                 f >= 0 &&
107                 f < matrizJuego.length &&
108                 c >= 0 &&
109                 c < matrizJuego[0].length
110             ) {
111                 abrirCelda(f, c);
112             }
113
114         }
115     }
116 }
117
118 if (comprobarVictoria()) {
119     partidaActiva = false;
120     detenerTemporizador();
121     console.log("¡Has ganado!");
122 }
123
124 }
125
126 function alternarBandera(fila, columna) {
127
128     if (!partidaActiva) {
129         return;
130     }
131
132     const celda = obtenerCeldaDOM(fila, columna);
133
134     if (!celda) {
135         return;
136     }
137
138     if (celda.classList.contains("abierta")) {
```

```
139     return;
140   }
141
142   if (celda.classList.contains("bandera")) {
143     celda.classList.remove("bandera");
144     celda.textContent = "";
145   } else {
146     celda.classList.add("bandera");
147     celda.textContent = "F";
148   }
149
150 }
151
152 function mostrarTodasLasMinas() {
153
154   for (let fila = 0; fila < matrizJuego.length; fila++) {
155     for (let columna = 0; columna < matrizJuego[fila].length; columna
156         ++ ) {
157
158       if (matrizJuego[fila][columna] === -1) {
159
160         const celda = obtenerCeldaDOM(fila, columna);
161
162         if (celda) {
163           celda.classList.add("abierta");
164           celda.textContent = "●";
165         }
166       }
167     }
168   }
169 }
170
171 }
172
173 function comprobarVictoria() {
174
175   for (let fila = 0; fila < matrizJuego.length; fila++) {
176     for (let columna = 0; columna < matrizJuego[fila].length; columna
177         ++ ) {
178
179       if (matrizJuego[fila][columna] !== -1) {
180
181         const celda = obtenerCeldaDOM(fila, columna);
182
183         if (celda && !celda.classList.contains("abierta")) {
184           return false;
185         }
186       }
187     }
188   }
```

```
188     }
189   }
190
191   return true;
192
193 }
194
195 function obtenerCeldaDOM(fila, columna) {
196
197   return document.querySelector(
198     `.celda[data-fila="${fila}"][data-columna="${columna}"]`
199   );
200
201 }
202
203 function actualizarReloj() {
204
205   const reloj = document.querySelector("#reloj");
206
207   if (reloj) {
208     reloj.textContent = tiempo;
209   }
210
211 }
212
213 function iniciarTemporizador() {
214
215   detenerTemporizador();
216
217   temporizador = setInterval(() => {
218     tiempo++;
219     actualizarReloj();
220   }, 1000);
221
222 }
223
224 function detenerTemporizador() {
225
226   if (temporizador !== null) {
227     clearInterval(temporizador);
228     temporizador = null;
229   }
230
231 }
232
233 function reiniciarReloj() {
234   tiempo = 0;
235   actualizarReloj();
236 }
237
238 function actualizarContadorMinas() {
```



```
239
240     const contadorMinas = document.querySelector("#minas");
241     const configuracion = dificultades[dificultadActual];
242
243     if (contadorMinas) {
244         contadorMinas.textContent = configuracion.minas;
245     }
246
247 }
248
249 function actualizarTituloPartida() {
250
251     const titulo = document.querySelector("#tituloPartida");
252     const configuracion = dificultades[dificultadActual];
253
254     if (titulo) {
255         titulo.textContent = configuracion.titulo;
256     }
257
258 }
259
260 function mostrarPanel(idPanel) {
261
262     const paneles = document.querySelectorAll(".panel");
263
264     paneles.forEach((panel) => {
265         panel.classList.add("hidden");
266     });
267
268     const panelActivo = document.querySelector(`#${idPanel}`);
269
270     if (panelActivo) {
271         panelActivo.classList.remove("hidden");
272     }
273
274 }
275
276 function configurarBotonReinicio() {
277
278     const botonReinicio = document.querySelector("#carita");
279
280     if (botonReinicio) {
281         botonReinicio.addEventListener("click", () => {
282             iniciarAplicacion();
283         });
284     }
285
286 }
287
288 function configurarMenuDificultad() {
289
```

```
290     const botones = document.querySelectorAll("[data-difficulty]");
291
292     botones.forEach((boton) => {
293         boton.addEventListener("click", () => {
294
295             const nuevaDificultad = boton.dataset.difficulty;
296
297             if (!nuevaDificultad) {
298                 return;
299             }
300
301             dificultadActual = nuevaDificultad;
302             iniciarAplicacion();
303
304         });
305     });
306
307 }
308
309 function configurarNavegacion() {
310
311     const botones = document.querySelectorAll("[data-panel]");
312
313     botones.forEach((boton) => {
314         boton.addEventListener("click", () => {
315
316             const idPanel = boton.dataset.panel;
317
318             if (!idPanel) {
319                 return;
320             }
321
322             mostrarPanel(idPanel);
323
324         });
325     });
326
327 }
328
329 configurarBotonReinicio();
330 configurarMenuDificultad();
331 configurarNavegacion();
332 mostrarPanel("panel_inicio");
333 iniciarAplicacion();
```

15.13 Qué hemos conseguido en este capítulo

En este capítulo hemos añadido la navegación entre los distintos paneles de la aplicación.

Ahora el proyecto ya es capaz de:

- ocultar y mostrar paneles dinámicamente
- navegar entre Inicio, Partida, Puntuaciones, Ayuda y Licencia
- usar `data-panel` para conectar HTML y JavaScript
- mostrar el panel adecuado al pulsar en el menú superior

Con esto, el Buscaminas ya deja de ser solo un tablero y se convierte en una aplicación web con varias secciones.

En el siguiente capítulo refinaremos esta navegación para que, al salir de la partida, **el temporizador se detenga y el estado del juego se reinicie correctamente**, igual que ocurre en el proyecto final.

16 Detener la partida al salir del panel de juego

En el capítulo anterior hemos conseguido que la aplicación navegue entre distintos paneles: Inicio, Partida, Puntuaciones, Ayuda y Licencia. Sin embargo, todavía queda un comportamiento importante por mejorar para que el resultado se parezca al proyecto final.

Ahora mismo, si el jugador está en plena partida y pulsa en Inicio, Puntuaciones, Ayuda o Licencia, el panel cambia correctamente, pero el juego sigue vivo en segundo plano. Eso provoca dos problemas:

- el temporizador puede seguir corriendo
- la partida queda “a medias” aunque ya no estemos en el panel de juego

En una aplicación bien terminada, al salir del panel de partida debemos considerar que el usuario **abandona la partida actual**.

Por eso, en este capítulo vamos a hacer que al salir de la partida:

- se detenga el temporizador
- el reloj vuelva a cero
- la partida deje de estar activa

Con esta mejora, la navegación será mucho más coherente.

16.1 Qué entendemos por “salir de la partida”

En nuestra aplicación, el panel donde se juega es:

```
1 panel_partida
```

Por tanto, decimos que el usuario sale de la partida cuando:

- el panel actual es `panel_partida`
- y el nuevo panel que quiere mostrar es otro distinto

Por ejemplo:

- de `panel_partida` a `panel_inicio`
- de `panel_partida` a `panel_puntuaciones`
- de `panel_partida` a `panel_ayuda`
- de `panel_partida` a `panel_licencia`

En todos esos casos debemos cerrar correctamente la partida actual.

16.2 Necesitamos recordar qué panel está activo

Para poder detectar si el usuario está saliendo del panel de partida, necesitamos saber cuál era el panel visible antes del cambio.

Hasta ahora la función `mostrarPanel(idPanel)` simplemente ocultaba todos los paneles y mostraba uno nuevo, pero no guardaba información sobre cuál era el panel anterior.

Vamos a solucionarlo declarando una nueva variable global de módulo en `main.js`:

```
1 let panelActual = "panel_inicio";
```

Esta variable almacenará el identificador del panel visible en cada momento.

Al comenzar la aplicación asumiremos que el panel inicial es `panel_inicio`.

16.3 Crear una función para abandonar la partida

Ahora vamos a agrupar en una función todas las acciones que deben hacerse cuando el usuario abandona la partida.

Añadimos en `main.js`:

```
1 function abandonarPartida() {  
2  
3   partidaActiva = false;  
4   detenerTemporizador();  
5   reiniciarReloj();  
6  
7 }
```

Esta función hace exactamente lo que necesitamos:

- marca la partida como no activa
- detiene el reloj
- devuelve el contador de tiempo a cero

De momento no vamos a borrar el tablero ni reiniciar la matriz, porque una nueva partida ya se generará cuando el usuario pulse una dificultad o el botón de reinicio.

16.4 Modificar mostrarPanel para detectar la salida del juego

Ahora actualizaremos la función `mostrarPanel`.

Antes la teníamos así:

```
1 function mostrarPanel(idPanel) {
2
3   const paneles = document.querySelectorAll(".panel");
4
5   paneles.forEach((panel) => {
6     panel.classList.add("hidden");
7   });
8
9   const panelActivo = document.querySelector(`#${idPanel}`);
10
11   if (panelActivo) {
12     panelActivo.classList.remove("hidden");
13   }
14
15 }
```

La vamos a sustituir por esta versión:

```
1 function mostrarPanel(idPanel) {
2
3   if (panelActual === "panel_partida" && idPanel !== "panel_partida") {
4     abandonarPartida();
5   }
6
7   const paneles = document.querySelectorAll(".panel");
8
9   paneles.forEach((panel) => {
10     panel.classList.add("hidden");
11   });
12
13   const panelActivo = document.querySelector(`#${idPanel}`);
14
15   if (panelActivo) {
16     panelActivo.classList.remove("hidden");
17     panelActual = idPanel;
18   }
19
20 }
```

16.5 Qué hace esta nueva versión

La parte más importante es esta:

```
1 if (panelActual === "panel_partida" && idPanel !== "panel_partida") {  
2   abandonarPartida();  
3 }
```

Esta condición significa:

- si el panel visible actualmente es el de partida
- y el nuevo panel no es el de partida
- entonces estamos saliendo del juego

En ese momento llamamos a `abandonarPartida()`.

Después, como ya hacíamos antes:

- ocultamos todos los paneles
- mostramos el panel solicitado
- actualizamos la variable `panelActual`

16.6 Por qué esta solución es sencilla y didáctica

Podríamos diseñar una arquitectura más compleja con un router, clases o eventos personalizados, pero en este proyecto no merece la pena.

Para alumnos de SMR, esta solución tiene varias ventajas:

- es fácil de leer
- usa solo variables y funciones sencillas
- permite ver claramente cuándo se cambia de panel
- conecta muy bien con el funcionamiento real del programa

Es, por tanto, una solución muy adecuada para una práctica guiada.

16.7 Qué ocurre si el usuario cambia de dificultad

Este caso es importante.

Cuando el usuario pulsa Fácil, Normal o Difícil:

- el botón tiene `data-panel="panel_partida"`
- además tiene `data-difficulty`

Eso significa que:

- `mostrarPanel("panel_partida")` mantiene el panel de partida visible
- la condición de salida no se cumple, porque seguimos dentro de `panel_partida`
- después `configurarMenuDificultad()` llama a `iniciarAplicacion()`

Es decir, cambiar de dificultad **no se considera salir de la partida**, sino empezar una nueva partida en el mismo panel.

Y eso es exactamente lo que queremos.

16.8 Qué ocurre si el usuario pulsa Inicio o Ayuda en mitad de una partida

Ahora el comportamiento será mucho más correcto:

1. el usuario está en `panel_partida`
2. pulsa un botón con `data-panel="panel_inicio"` o cualquier otro panel no partida
3. `mostrarPanel()` detecta que se abandona la partida
4. se llama a `abandonarPartida()`
5. el temporizador se detiene
6. el reloj vuelve a cero
7. se muestra el nuevo panel

Así el juego ya no sigue corriendo en segundo plano.

16.9 Ajustar el arranque inicial

Como ahora estamos usando la variable `panelActual`, conviene que el flujo final del archivo sea coherente.

Al final de `main.js` seguiremos teniendo:

```
1 configurarBotonReinicio();
2 configurarMenuDificultad();
3 configurarNavegacion();
4 mostrarPanel("panel_inicio");
5 iniciarAplicacion();
```

Esto hace que:

- la interfaz arranque mostrando el panel de inicio
- el juego se inicialice en segundo plano con la dificultad actual

Más adelante todavía refinaremos este comportamiento para parecerse del todo al proyecto final, pero por ahora es suficiente para mantener una lógica clara y comprensible.

16.10 Código completo actualizado de main.js

Después de este capítulo, `main.js` debería quedar así:

```
1 import {
2   crearTablero,
3   crearMatriz,
4   colocarMinas,
5   calcularNumeros
6 } from "./board.js";
7
8 let matrizJuego = [];
9 let partidaActiva = true;
10 let tiempo = 0;
11 let temporizador = null;
12 let dificultadActual = "easy";
13 let panelActual = "panel_inicio";
14
15 const dificultades = {
16   easy: {
17     filas: 8,
18     columnas: 8,
19     minas: 10,
20     titulo: "Partida fácil"
21   },
22   medium: {
23     filas: 12,
24     columnas: 12,
25     minas: 25,
26     titulo: "Partida normal"
```

```
27     },
28     hard: {
29         filas: 16,
30         columnas: 16,
31         minas: 40,
32         titulo: "Partida difícil"
33     }
34 };
35
36 function iniciarAplicacion() {
37     partidaActiva = true;
38     reiniciarReloj();
39     iniciarTemporizador();
40
41     const configuracion = dificultades[dificultadActual];
42
43     matrizJuego = crearMatriz(configuracion.filas, configuracion.columnas
44         );
45     colocarMinas(matrizJuego, configuracion.minas);
46
47     calcularNumeros(matrizJuego);
48
49     crearTablero(
50         configuracion.filas,
51         configuracion.columnas,
52         abrirCelda,
53         alternarBandera
54     );
55
56     actualizarContadorMinas();
57     actualizarTituloPartida();
58 }
59
60 function abrirCelda(fila, columna) {
61     if (!partidaActiva) {
62         return;
63     }
64
65     const valor = matrizJuego[fila][columna];
66     const celda = obtenerCeldaDOM(fila, columna);
67
68     if (!celda) {
69         return;
70     }
71
72     if (celda.classList.contains("abierta")) {
```

```
77     return;
78 }
79
80 if (celda.classList.contains("bandera")) {
81     return;
82 }
83
84 celda.classList.add("abierta");
85
86 if (valor === -1) {
87     celda.textContent = "●";
88     partidaActiva = false;
89     detenerTemporizador();
90     mostrarTodasLasMinas();
91     return;
92 }
93
94 if (valor > 0) {
95     celda.textContent = valor;
96 } else {
97     celda.textContent = "";
98
99     for (let f = fila - 1; f <= fila + 1; f++) {
100         for (let c = columna - 1; c <= columna + 1; c++) {
101
102             if (f === fila && c === columna) {
103                 continue;
104             }
105
106             if (
107                 f >= 0 &&
108                 f < matrizJuego.length &&
109                 c >= 0 &&
110                 c < matrizJuego[0].length
111             ) {
112                 abrirCelda(f, c);
113             }
114         }
115     }
116 }
117
118
119 if (comprobarVictoria()) {
120     partidaActiva = false;
121     detenerTemporizador();
122     console.log("¡Has ganado!");
123 }
124
125 }
126
127 function alternarBandera(fila, columna) {
```

```
128
129     if (!partidaActiva) {
130         return;
131     }
132
133     const celda = obtenerCeldaDOM(fila, columna);
134
135     if (!celda) {
136         return;
137     }
138
139     if (celda.classList.contains("abierta")) {
140         return;
141     }
142
143     if (celda.classList.contains("bandera")) {
144         celda.classList.remove("bandera");
145         celda.textContent = "";
146     } else {
147         celda.classList.add("bandera");
148         celda.textContent = "🚩";
149     }
150
151 }
152
153 function mostrarTodasLasMinas() {
154
155     for (let fila = 0; fila < matrizJuego.length; fila++) {
156         for (let columna = 0; columna < matrizJuego[fila].length; columna
157             ++ ) {
158
159             if (matrizJuego[fila][columna] === -1) {
160
161                 const celda = obtenerCeldaDOM(fila, columna);
162
163                 if (celda) {
164                     celda.classList.add("abierta");
165                     celda.textContent = "💣";
166                 }
167             }
168
169         }
170     }
171
172 }
173
174 function comprobarVictoria() {
175
176     for (let fila = 0; fila < matrizJuego.length; fila++) {
177         for (let columna = 0; columna < matrizJuego[fila].length; columna
```

```
    ++) {
178
179     if (matrizJuego[filas][columna] !== -1) {
180
181         const celda = obtenerCeldaDOM(filas, columna);
182
183         if (celda && !celda.classList.contains("abierto")) {
184             return false;
185         }
186
187     }
188
189 }
190 }
191
192 return true;
193
194 }
195
196 function obtenerCeldaDOM(filas, columna) {
197
198     return document.querySelector(
199         `.celda[data-filas="${filas}"][data-columna="${columna}"]`
200     );
201
202 }
203
204 function actualizarReloj() {
205
206     const reloj = document.querySelector("#reloj");
207
208     if (reloj) {
209         reloj.textContent = tiempo;
210     }
211
212 }
213
214 function iniciarTemporizador() {
215
216     detenerTemporizador();
217
218     temporizador = setInterval(() => {
219         tiempo++;
220         actualizarReloj();
221     }, 1000);
222
223 }
224
225 function detenerTemporizador() {
226
227     if (temporizador !== null) {
```

```
228     clearInterval(temporizador);
229     temporizador = null;
230 }
231
232 }
233
234 function reiniciarReloj() {
235     tiempo = 0;
236     actualizarReloj();
237 }
238
239 function actualizarContadorMinas() {
240
241     const contadorMinas = document.querySelector("#minas");
242     const configuracion = dificultades[dificultadActual];
243
244     if (contadorMinas) {
245         contadorMinas.textContent = configuracion.minas;
246     }
247
248 }
249
250 function actualizarTituloPartida() {
251
252     const titulo = document.querySelector("#tituloPartida");
253     const configuracion = dificultades[dificultadActual];
254
255     if (titulo) {
256         titulo.textContent = configuracion.titulo;
257     }
258
259 }
260
261 function abandonarPartida() {
262
263     partidaActiva = false;
264     detenerTemporizador();
265     reiniciarReloj();
266
267 }
268
269 function mostrarPanel(idPanel) {
270
271     if (panelActual === "panel_partida" && idPanel !== "panel_partida") {
272         abandonarPartida();
273     }
274
275     const paneles = document.querySelectorAll(".panel");
276
277     paneles.forEach((panel) => {
278         panel.classList.add("hidden");
```

```
279     });
280
281     const panelActivo = document.querySelector(`#${idPanel}`);
282
283     if (panelActivo) {
284         panelActivo.classList.remove("hidden");
285         panelActual = idPanel;
286     }
287
288 }
289
290 function configurarBotonReinicio() {
291
292     const botonReinicio = document.querySelector("#carita");
293
294     if (botonReinicio) {
295         botonReinicio.addEventListener("click", () => {
296             iniciarAplicacion();
297         });
298     }
299
300 }
301
302 function configurarMenuDificultad() {
303
304     const botones = document.querySelectorAll("[data-difficulty]");
305
306     botones.forEach((boton) => {
307         boton.addEventListener("click", () => {
308
309             const nuevaDificultad = boton.dataset.difficulty;
310
311             if (!nuevaDificultad) {
312                 return;
313             }
314
315             dificultadActual = nuevaDificultad;
316             iniciarAplicacion();
317
318         });
319     });
320
321 }
322
323 function configurarNavegacion() {
324
325     const botones = document.querySelectorAll("[data-panel]");
326
327     botones.forEach((boton) => {
328         boton.addEventListener("click", () => {
329
```

```
330     const idPanel = boton.dataset.panel;
331
332     if (!idPanel) {
333         return;
334     }
335
336     mostrarPanel(idPanel);
337
338     });
339 });
340
341 }
342
343 configurarBotonReinicio();
344 configurarMenuDificultad();
345 configurarNavegacion();
346 mostrarPanel("panel_inicio");
347 iniciarAplicacion();
```

16.11 Qué hemos conseguido en este capítulo

En este capítulo hemos mejorado la navegación para que el estado del juego sea coherente cuando el usuario abandona la partida.

Ahora el proyecto ya es capaz de:

- detectar cuándo se sale del panel de juego
- detener el temporizador en ese momento
- reiniciar el reloj al abandonar la partida
- marcar la partida como no activa
- mantener una navegación más limpia y lógica entre paneles

Con esta mejora, la aplicación se comporta mucho mejor y se acerca todavía más al proyecto final.

En el siguiente capítulo añadiremos la **gestión de puntuaciones con localStorage**, de forma que el juego pueda guardar los resultados del usuario y mostrarlos después en el panel de puntuaciones.

17 LocalStorage

Hasta ahora ya tenemos un Buscaminas bastante completo: tablero, minas, números, banderas, victoria, derrota, temporizador, reinicio, dificultades y navegación entre paneles. El siguiente paso para acercarnos todavía más al proyecto final es añadir un sistema de **puntuaciones persistentes**.

En este apartado vamos a guardar y mostrar puntuaciones en almacenamiento persistente con `localStorage`.

Persistente significa que las puntuaciones no se perderán al recargar la página o cerrar el navegador. Para conseguirlo utilizaremos una herramienta del navegador llamada:

```
1 localStorage
```

Gracias a `localStorage`, el juego podrá:

- guardar las mejores partidas
- recuperarlas más adelante
- mostrarlas en el panel de puntuaciones
- borrar la clasificación cuando el usuario lo desee

Este capítulo es importante porque introduce un concepto muy útil en aplicaciones web: **almacenar información en el navegador**.

17.1 Qué es localStorage

`localStorage` es un sistema de almacenamiento que proporcionan los navegadores.

Permite guardar pares de **clave y valor** en el propio navegador del usuario.

Por ejemplo, podríamos guardar algo así:

```
1 localStorage.setItem("usuario", "Juan");
```

Y leerlo más tarde:

```
1 const nombre = localStorage.getItem("usuario");
```

La información permanece guardada incluso aunque cerremos la pestaña o apaguemos el navegador.

En nuestro proyecto utilizaremos `localStorage` para almacenar una lista de puntuaciones.

17.2 Qué información guardaremos en cada puntuación

Cada puntuación del juego debe contener al menos:

- el nombre del jugador
- la puntuación obtenida
- la dificultad
- el tiempo empleado

Podemos representarlo mediante un objeto como este:

```
1 {  
2   nombre: "Ana",  
3   puntos: 1500,  
4   dificultad: "easy",  
5   tiempo: 32  
6 }
```

Como necesitaremos guardar varias puntuaciones, realmente almacenaremos un **array de objetos**.

17.3 Crear el módulo `scores.js`

Hasta ahora el archivo `scores.js` existía en la estructura del proyecto pero aún no lo estábamos utilizando. Ha llegado el momento de darle contenido.

Creemos o abrimos el archivo:

```
1 js/scores.js
```

En este archivo iremos colocando toda la lógica relacionada con las puntuaciones.

17.4 Guardar y recuperar arrays con JSON

Hay un detalle importante: `localStorage` solo puede guardar **texto**.

Eso significa que, si queremos guardar un array o un objeto, primero debemos convertirlo a texto. Para ello utilizamos:

- `JSON.stringify()` para convertir a texto
- `JSON.parse()` para recuperar el objeto original

Por ejemplo:

```
1 const datos = [{ nombre: "Ana", puntos: 100 }];
2 const texto = JSON.stringify(datos);
3
4 localStorage.setItem("scores", texto);
```

Y para leerlo:

```
1 const guardado = localStorage.getItem("scores");
2 const puntuaciones = JSON.parse(guardado);
```

Este patrón será la base de nuestro sistema de puntuaciones.

17.5 Crear una función para obtener las puntuaciones guardadas

Vamos a empezar escribiendo en `scores.js` una función que recupere la lista de puntuaciones del navegador.

```
1 export function obtenerPuntuaciones() {
2
3   const datos = localStorage.getItem("scores");
4
5   if (!datos) {
6     return [];
7   }
8
9   return JSON.parse(datos);
10
11 }
```

Esta función hace lo siguiente:

- busca la clave `scores` en `localStorage`
- si no existe, devuelve un array vacío
- si existe, convierte el texto en un array de objetos y lo devuelve

De este modo, siempre podremos trabajar con una lista de puntuaciones sin preocuparnos de si ya había datos guardados o no.

17.6 Crear una función para guardar una nueva puntuación

Ahora añadimos una función que inserte una puntuación nueva y la vuelva a guardar.

```
1 export function guardarPuntuacion(puntuacion) {
2
3   const puntuaciones = obtenerPuntuaciones();
```

```
4
5   puntuaciones.push(puntuacion);
6
7   puntuaciones.sort((a, b) => b.puntos - a.puntos);
8
9   localStorage.setItem("scores", JSON.stringify(puntuaciones));
10
11 }
```

Vamos a analizar qué ocurre aquí:

- primero recuperamos las puntuaciones actuales
- añadimos la nueva con `push`
- ordenamos el array de mayor a menor puntuación
- guardamos el resultado actualizado en `localStorage`

Esto hará que la clasificación quede siempre ordenada.

17.7 Limitar el número de puntuaciones guardadas

En muchos juegos no interesa guardar una lista infinita. Lo habitual es conservar solo las mejores puntuaciones.

Podemos hacerlo fácilmente recortando el array después de ordenarlo.

Modificamos la función así:

```
1 export function guardarPuntuacion(puntuacion) {
2
3   const puntuaciones = obtenerPuntuaciones();
4
5   puntuaciones.push(puntuacion);
6
7   puntuaciones.sort((a, b) => b.puntos - a.puntos);
8
9   const mejoresPuntuaciones = puntuaciones.slice(0, 10);
10
11   localStorage.setItem("scores", JSON.stringify(mejoresPuntuaciones));
12
13 }
```

Con `slice(0, 10)` nos quedamos solo con las diez mejores.

17.8 Crear una función para borrar las puntuaciones

También necesitaremos una forma de eliminar la clasificación.

Añadimos esta función a `scores.js`:

```
1 export function borrarPuntuaciones() {
2   localStorage.removeItem("scores");
3 }
```

Con esto el navegador elimina completamente la clave `scores`.

17.9 Generar el HTML de la tabla de puntuaciones

Ahora debemos mostrar las puntuaciones dentro del panel correspondiente.

En el `index.html` ya existe un contenedor con id:

```
1 tablaPuntuaciones
```

Vamos a crear una función en `scores.js` que genere el contenido HTML de la clasificación.

```
1 export function generarTablaPuntuaciones() {
2
3   const puntuaciones = obtenerPuntuaciones();
4
5   if (puntuaciones.length === 0) {
6     return "<p>No hay puntuaciones guardadas.</p>";
7   }
8
9   let html = `
10     <table>
11       <thead>
12         <tr>
13           <th>Posición</th>
14           <th>Nombre</th>
15           <th>Puntos</th>
16           <th>Dificultad</th>
17           <th>Tiempo</th>
18         </tr>
19       </thead>
20       <tbody>
21     `;
22
23     puntuaciones.forEach((puntuacion, index) => {
24       html += `
25         <tr>
26           <td>${index + 1}</td>
27           <td>${puntuacion.nombre}</td>
28           <td>${puntuacion.puntos}</td>
29           <td>${puntuacion.dificultad}</td>
30           <td>${puntuacion.tiempo}</td>
31         </tr>`
```

```
32     `;  
33   });  
34  
35   html += `  
36     </tbody>  
37     </table>  
38   `;  
39  
40   return html;  
41  
42 }
```

Esta función devuelve un texto HTML que representa la tabla de clasificación.

Más adelante lo insertaremos en el DOM.

17.10 Código completo de `scores.js`

Después de estos pasos, el archivo `scores.js` debería quedar así:

```
1  export function obtenerPuntuaciones() {  
2  
3    const datos = localStorage.getItem("scores");  
4  
5    if (!datos) {  
6      return [];  
7    }  
8  
9    return JSON.parse(datos);  
10  
11  }  
12  
13  export function guardarPuntuacion(puntuacion) {  
14  
15    const puntuaciones = obtenerPuntuaciones();  
16  
17    puntuaciones.push(puntuacion);  
18  
19    puntuaciones.sort((a, b) => b.puntos - a.puntos);  
20  
21    const mejoresPuntuaciones = puntuaciones.slice(0, 10);  
22  
23    localStorage.setItem("scores", JSON.stringify(mejoresPuntuaciones));  
24  
25  }  
26  
27  export function borrarPuntuaciones() {  
28    localStorage.removeItem("scores");  
29  }
```

```
30
31 export function generarTablaPuntuaciones() {
32
33     const puntuaciones = obtenerPuntuaciones();
34
35     if (puntuaciones.length === 0) {
36         return "<p>No hay puntuaciones guardadas.</p>";
37     }
38
39     let html = `
40         <table>
41             <thead>
42                 <tr>
43                     <th>Posición</th>
44                     <th>Nombre</th>
45                     <th>Puntos</th>
46                     <th>Dificultad</th>
47                     <th>Tiempo</th>
48                 </tr>
49             </thead>
50             <tbody>
51     `;
52
53     puntuaciones.forEach((puntuacion, index) => {
54         html += `
55             <tr>
56                 <td>${index + 1}</td>
57                 <td>${puntuacion.nombre}</td>
58                 <td>${puntuacion.puntos}</td>
59                 <td>${puntuacion.dificultad}</td>
60                 <td>${puntuacion.tiempo}</td>
61             </tr>
62         `;
63     });
64
65     html += `
66         </tbody>
67     </table>
68     `;
69
70     return html;
71
72 }
```

17.11 Importar el módulo de puntuaciones en main.js

Ahora tenemos que conectar este módulo con el resto de la aplicación.

Al principio de `main.js`, añadimos esta nueva importación:

```
1 import {
2   guardarPuntuacion,
3   borrarPuntuaciones,
4   generarTablaPuntuaciones
5 } from "./scores.js";
```

De esta forma `main.js` podrá:

- guardar una nueva puntuación al ganar
- borrar la clasificación cuando el usuario pulse el botón
- actualizar el panel de puntuaciones

17.12 Crear una función para calcular los puntos

No basta con guardar el tiempo. También necesitamos calcular una puntuación numérica.

Podemos usar una fórmula sencilla que tenga en cuenta:

- el número de minas
- el tamaño del tablero
- el tiempo invertido

Añadimos esta función en `main.js`:

```
1 function calcularPuntos() {
2
3   const configuracion = dificultades[dificultadActual];
4   const totalCeldas = configuracion.filas * configuracion.columnas;
5
6   if (tiempo === 0) {
7     return 0;
8   }
9
10  return Math.floor((configuracion.minas * 1000) / tiempo + totalCeldas
11    );
12 }
```

Esta fórmula es simple, didáctica y suficiente para el tutorial.

Cuantas más minas haya y menos tiempo tarde el jugador, más puntuación obtendrá.

17.13 Guardar la puntuación al ganar

Ahora debemos actuar cuando el jugador gana la partida.

Hasta ahora, al final de `abrirCelda`, hacíamos esto:

```
1 if (comprobarVictoria()) {
2   partidaActiva = false;
3   detenerTemporizador();
4   console.log("¡Has ganado!");
5 }
```

Vamos a reemplazarlo por una versión que además abra el cuadro para introducir el nombre del jugador.

```
1 if (comprobarVictoria()) {
2   partidaActiva = false;
3   detenerTemporizador();
4   mostrarDialogoPuntuacion();
5 }
```

De este modo, cuando se gane la partida, se abrirá el diálogo correspondiente.

17.14 Mostrar el diálogo para guardar la puntuación

En el `index.html` ya existe un elemento `<dialog>` con id:

```
1 dialogoPuntuacion
```

Vamos a crear en `main.js` una función para mostrarlo.

```
1 function mostrarDialogoPuntuacion() {
2
3   const dialogo = document.querySelector("#dialogoPuntuacion");
4   const inputNombre = document.querySelector("#nombreJugador");
5
6   if (!dialogo) {
7     return;
8   }
9
10  if (inputNombre) {
11    inputNombre.value = "";
12  }
13
14  dialogo.showModal();
15
16 }
```

Esta función:

- localiza el diálogo
- vacía el campo de nombre

- abre la ventana modal

17.15 Guardar realmente la puntuación

Ahora necesitamos procesar el formulario del diálogo.

Vamos a crear una función que configure el botón de guardar.

Añadimos en `main.js`:

```
1 function configurarDialogoPuntuacion() {
2
3   const formulario = document.querySelector("#formPuntuacion");
4   const dialogo = document.querySelector("#dialogoPuntuacion");
5   const inputNombre = document.querySelector("#nombreJugador");
6   const botonCancelar = document.querySelector("#cancelarGuardar");
7
8   if (botonCancelar && dialogo) {
9     botonCancelar.addEventListener("click", () => {
10       dialogo.close();
11     });
12   }
13
14   if (formulario && dialogo && inputNombre) {
15     formulario.addEventListener("submit", (event) => {
16       event.preventDefault();
17
18       const nombre = inputNombre.value.trim() || "Anónimo";
19
20       const puntuacion = {
21         nombre,
22         puntos: calcularPuntos(),
23         dificultad: dificultadActual,
24         tiempo
25       };
26
27       guardarPuntuacion(puntuacion);
28       actualizarPanelPuntuaciones();
29
30       dialogo.close();
31       mostrarPanel("panel_puntuaciones");
32     });
33   }
34
35 }
```

Esta función hace bastante trabajo:

- gestiona el botón de cancelar

- intercepta el envío del formulario
- obtiene el nombre del jugador
- calcula la puntuación
- guarda la partida en `localStorage`
- actualiza la tabla visible
- cierra el diálogo
- navega al panel de puntuaciones

17.16 Crear una función para refrescar el panel de puntuaciones

Necesitamos una forma cómoda de actualizar el contenido del panel.

Añadimos en `main.js`:

```
1 function actualizarPanelPuntuaciones() {
2
3   const contenedor = document.querySelector("#tablaPuntuaciones");
4
5   if (contenedor) {
6     contenedor.innerHTML = generarTablaPuntuaciones();
7   }
8
9 }
```

Con esta función, cada vez que guardemos o borremos puntuaciones, la tabla se volverá a dibujar.

17.17 Configurar el botón de borrar puntuaciones

En el HTML ya existe un botón con id:

```
1 borrarPuntuaciones
```

Vamos a conectarlo con JavaScript.

Añadimos en `main.js`:

```
1 function configurarBotonBorrarPuntuaciones() {
2
3   const boton = document.querySelector("#borrarPuntuaciones");
4
5   if (boton) {
6     boton.addEventListener("click", () => {
7       borrarPuntuaciones();
8       actualizarPanelPuntuaciones();
9     });
10  }
```

```
11  
12 }
```

Así, cuando el usuario pulse el botón, se eliminará la clasificación y se refrescará el panel.

17.18 Actualizar las puntuaciones al navegar al panel correspondiente

Es buena idea que, cada vez que el usuario vaya al panel de puntuaciones, se reconstruya la tabla.

Podemos hacerlo dentro de `mostrarPanel`.

Buscamos esta parte:

```
1 if (panelActivo) {  
2   panelActivo.classList.remove("hidden");  
3   panelActual = idPanel;  
4 }
```

Y la ampliamos así:

```
1 if (panelActivo) {  
2   panelActivo.classList.remove("hidden");  
3   panelActual = idPanel;  
4  
5   if (idPanel === "panel_puntuaciones") {  
6     actualizarPanelPuntuaciones();  
7   }  
8 }
```

Con esto, el panel de puntuaciones siempre mostrará los datos más recientes.

17.19 Registrar las nuevas configuraciones al arrancar

Al final del archivo, además de las configuraciones anteriores, ahora debemos ejecutar también:

- `configurarDialogoPuntuacion()`
- `configurarBotonBorrarPuntuaciones()`

El final de `main.js` quedará así:

```
1 configurarBotonReinicio();  
2 configurarMenuDificultad();  
3 configurarNavegacion();  
4 configurarDialogoPuntuacion();  
5 configurarBotonBorrarPuntuaciones();  
6 mostrarPanel("panel_inicio");  
7 iniciarAplicacion();
```

17.20 Código completo actualizado de main.js

Después de este capítulo, `main.js` debería quedar así:

```
1  import {
2    crearTablero,
3    crearMatriz,
4    colocarMinas,
5    calcularNumeros
6  } from "./board.js";
7
8  import {
9    guardarPuntuacion,
10   borrarPuntuaciones,
11   generarTablaPuntuaciones
12 } from "./scores.js";
13
14 let matrizJuego = [];
15 let partidaActiva = true;
16 let tiempo = 0;
17 let temporizador = null;
18 let dificultadActual = "easy";
19 let panelActual = "panel_inicio";
20
21 const dificultades = {
22   easy: {
23     filas: 8,
24     columnas: 8,
25     minas: 10,
26     titulo: "Partida fácil"
27   },
28   medium: {
29     filas: 12,
30     columnas: 12,
31     minas: 25,
32     titulo: "Partida normal"
33   },
34   hard: {
35     filas: 16,
36     columnas: 16,
37     minas: 40,
38     titulo: "Partida difícil"
39   }
40 };
41
42 function iniciarAplicacion() {
43   partidaActiva = true;
44   reiniciarReloj();
45   iniciarTemporizador();
46 }
```

```
48
49     const configuracion = dificultades[dificultadActual];
50
51     matrizJuego = crearMatriz(configuracion.filas, configuracion.columnas
52         );
53     colocarMinas(matrizJuego, configuracion.minas);
54
55     calcularNumeros(matrizJuego);
56
57     crearTablero(
58         configuracion.filas,
59         configuracion.columnas,
60         abrirCelda,
61         alternarBandera
62     );
63
64     actualizarContadorMinas();
65     actualizarTituloPartida();
66
67 }
68
69 function abrirCelda(fila, columna) {
70
71     if (!partidaActiva) {
72         return;
73     }
74
75     const valor = matrizJuego[fila][columna];
76     const celda = obtenerCeldaDOM(fila, columna);
77
78     if (!celda) {
79         return;
80     }
81
82     if (celda.classList.contains("abierta")) {
83         return;
84     }
85
86     if (celda.classList.contains("bandera")) {
87         return;
88     }
89
90     celda.classList.add("abierta");
91
92     if (valor === -1) {
93         celda.textContent = "●";
94         partidaActiva = false;
95         detenerTemporizador();
96         mostrarTodasLasMinas();
97         return;
```

```
98     }
99
100    if (valor > 0) {
101        celda.textContent = valor;
102    } else {
103        celda.textContent = "";
104
105        for (let f = fila - 1; f <= fila + 1; f++) {
106            for (let c = columna - 1; c <= columna + 1; c++) {
107
108                if (f === fila && c === columna) {
109                    continue;
110                }
111
112                if (
113                    f >= 0 &&
114                    f < matrizJuego.length &&
115                    c >= 0 &&
116                    c < matrizJuego[0].length
117                ) {
118                    abrirCelda(f, c);
119                }
120            }
121        }
122    }
123 }
124
125 if (comprobarVictoria()) {
126     partidaActiva = false;
127     detenerTemporizador();
128     mostrarDialogoPuntuacion();
129 }
130
131 }
132
133 function alternarBandera(fila, columna) {
134
135     if (!partidaActiva) {
136         return;
137     }
138
139     const celda = obtenerCeldaDOM(fila, columna);
140
141     if (!celda) {
142         return;
143     }
144
145     if (celda.classList.contains("abierta")) {
146         return;
147     }
148 }
```

```
149     if (celda.classList.contains("bandera")) {
150         celda.classList.remove("bandera");
151         celda.textContent = "";
152     } else {
153         celda.classList.add("bandera");
154         celda.textContent = "⚡";
155     }
156
157 }
158
159 function mostrarTodasLasMinas() {
160
161     for (let fila = 0; fila < matrizJuego.length; fila++) {
162         for (let columna = 0; columna < matrizJuego[fila].length; columna
163             ++ ) {
164
165             if (matrizJuego[fila][columna] === -1) {
166
167                 const celda = obtenerCeldaDOM(fila, columna);
168
169                 if (celda) {
170                     celda.classList.add("abierta");
171                     celda.textContent = "●";
172                 }
173             }
174
175         }
176     }
177
178 }
179
180 function comprobarVictoria() {
181
182     for (let fila = 0; fila < matrizJuego.length; fila++) {
183         for (let columna = 0; columna < matrizJuego[fila].length; columna
184             ++ ) {
185
186             if (matrizJuego[fila][columna] !== -1) {
187
188                 const celda = obtenerCeldaDOM(fila, columna);
189
190                 if (celda && !celda.classList.contains("abierta")) {
191                     return false;
192                 }
193             }
194
195         }
196     }
197
198 }
```



```
198     return true;
199 }
200 }
201
202 function obtenerCeldaDOM(fila, columna) {
203
204     return document.querySelector(
205         `.celda[data-fila="${fila}"][data-columna="${columna}"]`
206     );
207 }
208 }
209
210 function actualizarReloj() {
211
212     const reloj = document.querySelector("#reloj");
213
214     if (reloj) {
215         reloj.textContent = tiempo;
216     }
217 }
218 }
219
220 function iniciarTemporizador() {
221
222     detenerTemporizador();
223
224     temporizador = setInterval(() => {
225         tiempo++;
226         actualizarReloj();
227     }, 1000);
228 }
229 }
230
231 function detenerTemporizador() {
232
233     if (temporizador !== null) {
234         clearInterval(temporizador);
235         temporizador = null;
236     }
237 }
238 }
239
240 function reiniciarReloj() {
241     tiempo = 0;
242     actualizarReloj();
243 }
244
245 function actualizarContadorMinas() {
246
247     const contadorMinas = document.querySelector("#minas");
248     const configuracion = dificultades[dificultadActual];
```

```
249
250     if (contadorMinas) {
251         contadorMinas.textContent = configuracion.minas;
252     }
253
254 }
255
256 function actualizarTituloPartida() {
257
258     const titulo = document.querySelector("#tituloPartida");
259     const configuracion = dificultades[dificultadActual];
260
261     if (titulo) {
262         titulo.textContent = configuracion.titulo;
263     }
264
265 }
266
267 function calcularPuntos() {
268
269     const configuracion = dificultades[dificultadActual];
270     const totalCeldas = configuracion.filas * configuracion.columnas;
271
272     if (tiempo === 0) {
273         return 0;
274     }
275
276     return Math.floor((configuracion.minas * 1000) / tiempo + totalCeldas
277         );
278 }
279
280 function mostrarDialogoPuntuacion() {
281
282     const dialogo = document.querySelector("#dialogoPuntuacion");
283     const inputNombre = document.querySelector("#nombreJugador");
284
285     if (!dialogo) {
286         return;
287     }
288
289     if (inputNombre) {
290         inputNombre.value = "";
291     }
292
293     dialogo.showModal();
294
295 }
296
297 function actualizarPanelPuntuaciones() {
298
```

```
299     const contenedor = document.querySelector("#tablaPuntuaciones");
300
301     if (contenedor) {
302         contenedor.innerHTML = generarTablaPuntuaciones();
303     }
304
305 }
306
307 function abandonarPartida() {
308
309     partidaActiva = false;
310     detenerTemporizador();
311     reiniciarReloj();
312
313 }
314
315 function mostrarPanel(idPanel) {
316
317     if (panelActual === "panel_partida" && idPanel !== "panel_partida") {
318         abandonarPartida();
319     }
320
321     const paneles = document.querySelectorAll(".panel");
322
323     paneles.forEach((panel) => {
324         panel.classList.add("hidden");
325     });
326
327     const panelActivo = document.querySelector(`#${idPanel}`);
328
329     if (panelActivo) {
330         panelActivo.classList.remove("hidden");
331         panelActual = idPanel;
332
333         if (idPanel === "panel_puntuaciones") {
334             actualizarPanelPuntuaciones();
335         }
336     }
337
338 }
339
340 function configurarBotonReinicio() {
341
342     const botonReinicio = document.querySelector("#carita");
343
344     if (botonReinicio) {
345         botonReinicio.addEventListener("click", () => {
346             iniciarAplicacion();
347         });
348     }
349 }
```

```
350 }
351
352 function configurarMenuDificultad() {
353
354     const botones = document.querySelectorAll("[data-difficulty]");
355
356     botones.forEach((boton) => {
357         boton.addEventListener("click", () => {
358
359             const nuevaDificultad = boton.dataset.difficulty;
360
361             if (!nuevaDificultad) {
362                 return;
363             }
364
365             dificultadActual = nuevaDificultad;
366             iniciarAplicacion();
367
368         });
369     });
370 }
371
372
373 function configurarNavegacion() {
374
375     const botones = document.querySelectorAll("[data-panel]");
376
377     botones.forEach((boton) => {
378         boton.addEventListener("click", () => {
379
380             const idPanel = boton.dataset.panel;
381
382             if (!idPanel) {
383                 return;
384             }
385
386             mostrarPanel(idPanel);
387
388         });
389     });
390 }
391
392
393 function configurarDialogoPuntuacion() {
394
395     const formulario = document.querySelector("#formPuntuacion");
396     const dialogo = document.querySelector("#dialogoPuntuacion");
397     const inputNombre = document.querySelector("#nombreJugador");
398     const botonCancelar = document.querySelector("#cancelarGuardar");
399
400     if (botonCancelar && dialogo) {
```

```
401     botonCancelar.addEventListener("click", () => {
402         dialogo.close();
403     });
404 }
405
406 if (formulario && dialogo && inputNombre) {
407     formulario.addEventListener("submit", (event) => {
408         event.preventDefault();
409
410         const nombre = inputNombre.value.trim() || "Anónimo";
411
412         const puntuacion = {
413             nombre,
414             puntos: calcularPuntos(),
415             dificultad: dificultadActual,
416             tiempo
417         };
418
419         guardarPuntuacion(puntuacion);
420         actualizarPanelPuntuaciones();
421
422         dialogo.close();
423         mostrarPanel("panel_puntuaciones");
424     });
425 }
426
427 }
428
429 function configurarBotonBorrarPuntuaciones() {
430
431     const boton = document.querySelector("#borrarPuntuaciones");
432
433     if (boton) {
434         boton.addEventListener("click", () => {
435             borrarPuntuaciones();
436             actualizarPanelPuntuaciones();
437         });
438     }
439
440 }
441
442 configurarBotonReinicio();
443 configurarMenuDificultad();
444 configurarNavegacion();
445 configurarDialogoPuntuacion();
446 configurarBotonBorrarPuntuaciones();
447 mostrarPanel("panel_inicio");
448 iniciarAplicacion();
```

17.21 Qué hemos conseguido en este capítulo

En este capítulo hemos añadido una de las últimas piezas importantes del proyecto:

- almacenamiento de puntuaciones en `localStorage`
- generación de una tabla de clasificación
- apertura del diálogo al ganar
- guardado del nombre, tiempo, dificultad y puntos
- borrado de puntuaciones
- actualización automática del panel de clasificación

Con esto, el Buscaminas ya se acerca mucho al resultado final de la aplicación completa.

En el siguiente capítulo podemos centrarnos en **pulir la interfaz y ajustar el flujo final de arranque y navegación** para que el comportamiento coincida todavía mejor con el proyecto definitivo.

18 Retoques finales

A lo largo de los capítulos anteriores hemos construido paso a paso un juego de Buscaminas funcional. Ya tenemos la lógica principal, la navegación entre paneles, el temporizador, el sistema de puntuaciones y la gestión de distintas dificultades. El último paso consiste en **cerrar correctamente la interfaz** y **ajustar el flujo final de arranque y navegación** para que la aplicación se comporte de forma coherente y quede lista como proyecto completo.

En este capítulo vamos a hacer tres mejoras importantes:

- mejorar el aspecto visual del juego con clases y estilos más claros
- evitar que la aplicación inicie una partida “en segundo plano” al cargar el panel de inicio
- conseguir que la navegación y el arranque final se comporten como una aplicación terminada

Este capítulo no añade grandes algoritmos nuevos, pero sí aporta algo fundamental en cualquier proyecto real: **acabado, coherencia y experiencia de uso**.

18.1 El problema del arranque actual

Hasta ahora el final de `main.js` tenía esta secuencia:

```
1 configurarBotonReinicio();
2 configurarMenuDificultad();
3 configurarNavegacion();
4 configurarDialogoPuntuacion();
5 configurarBotonBorrarPuntuaciones();
6 mostrarPanel("panel_inicio");
7 iniciarAplicacion();
```

Esto funciona, pero tiene un inconveniente.

Nada más cargar la aplicación:

- se muestra el panel de inicio
- pero también se crea una partida en segundo plano
- el temporizador empieza a correr aunque el usuario todavía no haya empezado a jugar

Desde el punto de vista técnico no rompe el programa, pero desde el punto de vista de la experiencia de usuario no es lo ideal.

Lo correcto es que:

- la aplicación arranque mostrando Inicio
- no exista ninguna partida activa todavía

- la partida solo empiece cuando el usuario elija una dificultad o pulse reinicio desde el panel de juego

18.2 Introducir la idea de “no hay partida iniciada”

Para resolver esto, vamos a distinguir entre dos estados:

- la aplicación está abierta
- hay o no hay una partida realmente iniciada

Hasta ahora la variable `partidaActiva` indicaba si la partida seguía viva o no, pero al arrancar el programa la estábamos poniendo a `true` siempre que llamábamos a `iniciarAplicacion()`.

La solución más sencilla es esta:

- al cargar la aplicación, mostramos Inicio
- actualizamos reloj y minas a valores por defecto
- no arrancamos `iniciarAplicacion()` automáticamente
- solo llamamos a `iniciarAplicacion()` cuando el usuario empieza una partida

18.3 Preparar la interfaz inicial

Vamos a crear una función que deje preparada la interfaz al arrancar.

Añadimos en `main.js`:

```
1 function prepararInterfazInicial() {  
2  
3     partidaActiva = false;  
4     detenerTemporizador();  
5     reiniciarReloj();  
6     actualizarContadorMinas();  
7     actualizarTituloPartida();  
8  
9 }
```

Esta función deja la aplicación en un estado limpio:

- no hay partida activa
- el temporizador está parado
- el reloj está a cero
- el contador de minas muestra el valor de la dificultad actual
- el título del panel de partida está preparado

De esta manera la interfaz ya queda consistente desde el principio, aunque todavía no se haya generado un tablero nuevo.

18.4 Iniciar partida solo cuando corresponda

Ahora debemos cambiar la lógica del menú.

Hasta ahora, cuando el usuario pulsaba un botón con `data-difficulty`, ocurría esto:

- se actualizaba `difficultadActual`
- se llamaba a `iniciarAplicacion()`

Eso ya está bien. Lo que debemos hacer es **eliminar la llamada automática a `iniciarAplicacion()` al final del archivo.**

Así, la primera partida no comenzará hasta que el usuario elija Fácil, Normal o Difícil.

18.5 Ajustar el botón de reinicio

Hay un caso especial: el botón de reinicio (`#carita`).

Si el usuario pulsa la carita mientras está en el panel de partida, debe comenzar una nueva partida con la dificultad actual.

Pero si está en otro panel, el comportamiento más lógico es:

- mostrar el panel de partida
- iniciar una nueva partida

Vamos a mejorar la función `configurarBotonReinicio()` para que haga justamente eso.

Sustituimos su contenido por este:

```
1 function configurarBotonReinicio() {
2
3   const botonReinicio = document.querySelector("#carita");
4
5   if (botonReinicio) {
6     botonReinicio.addEventListener("click", () => {
7       mostrarPanel("panel_partida");
8       iniciarAplicacion();
9     });
10  }
11
12 }
```

Con esto conseguimos que la carita siempre tenga sentido como “empezar una partida”.

18.6 Mejorar el abandono de partida

La función `abandonarPartida()` ya detenía el reloj y marcaba la partida como no activa.

Vamos a dejarla así, de forma definitiva:

```
1 function abandonarPartida() {
2
3     partidaActiva = false;
4     detenerTemporizador();
5     reiniciarReloj();
6
7 }
```

Esto es suficiente para el tutorial porque:

- la partida deja de estar activa
- el tiempo se resetea
- el siguiente inicio generará un tablero nuevo

No necesitamos complicarlo más para este proyecto.

18.7 Ajustar la navegación para que no rompa el flujo

La función `mostrarPanel(idPanel)` ya detectaba cuándo salíamos del panel de partida.

La mantendremos con esta lógica:

```
1 function mostrarPanel(idPanel) {
2
3     if (panelActual === "panel_partida" && idPanel !== "panel_partida") {
4         abandonarPartida();
5     }
6
7     const paneles = document.querySelectorAll(".panel");
8
9     paneles.forEach((panel) => {
10         panel.classList.add("hidden");
11     });
12
13     const panelActivo = document.querySelector(`#${idPanel}`);
14
15     if (panelActivo) {
16         panelActivo.classList.remove("hidden");
17         panelActual = idPanel;
18
19         if (idPanel === "panel_puntuaciones") {
20             actualizarPanelPuntuaciones();
21         }
22     }
23 }
```

```
21     }  
22   }  
23  
24 }
```

Con esto conseguimos que:

- salir de la partida la detenga correctamente
- entrar en puntuaciones refresque la tabla
- el resto de paneles funcionen igual de bien

18.8 Mejorar la experiencia visual de las celdas

Hasta ahora la lógica del juego ya funciona, pero para que la interfaz quede bien terminada conviene asociar mejor los estados visuales a clases CSS.

Las celdas pueden estar en varios estados:

- cerrada
- abierta
- con bandera

Y además pueden contener:

- una mina
- un número
- una celda vacía

Ya estábamos usando clases como `abierta` y `bandera`, así que ahora conviene apoyarnos más en ellas visualmente desde el CSS.

18.9 Añadir estilos mínimos recomendados

En `css/estilos.css`, asegúrate de tener reglas equivalentes a estas:

```
1 body {  
2   font-family: Arial, sans-serif;  
3 }  
4  
5 .hidden {  
6   display: none;  
7 }  
8  
9 .board {
```

```
10   display: grid;
11   gap: 2px;
12   justify-content: start;
13   margin-top: 1rem;
14 }
15
16 .celda {
17   width: 32px;
18   height: 32px;
19   border: 1px solid #999;
20   background-color: #cfcfcf;
21   display: flex;
22   align-items: center;
23   justify-content: center;
24   font-weight: bold;
25   cursor: pointer;
26   user-select: none;
27 }
28
29 .celda.abierta {
30   background-color: #f1f1f1;
31   border-color: #bbb;
32   cursor: default;
33 }
34
35 .celda.bandera {
36   background-color: #ffe7a8;
37 }
38
39 .hud {
40   display: flex;
41   gap: 1rem;
42   align-items: center;
43   justify-content: center;
44   margin: 1rem 0;
45 }
46
47 .counter-box {
48   background: #222;
49   color: #fff;
50   padding: 0.5rem 1rem;
51   border-radius: 6px;
52   text-align: center;
53 }
54
55 .face-button {
56   padding: 0.5rem 1rem;
57   font-size: 1.2rem;
58   cursor: pointer;
59 }
60
```

```
61 table {
62   border-collapse: collapse;
63   width: 100%;
64   margin-top: 1rem;
65 }
66
67 th,
68 td {
69   border: 1px solid #ccc;
70   padding: 0.5rem;
71   text-align: center;
72 }
```

Estos estilos no son complejos, pero mejoran mucho el aspecto del juego y permiten que la interfaz tenga un acabado más claro y agradable.

18.10 Ajustar dinámicamente el tamaño del tablero

Cuando cambiamos de dificultad, el número de celdas cambia. Para que el tablero se vea bien, conviene ajustar automáticamente el número de columnas del grid.

Podemos hacerlo dentro de `crearTablero` en `board.js`.

Añade esta línea justo después de seleccionar el contenedor:

```
1 tablero.style.gridTemplateColumns = `repeat(${columnas}, 32px)`;
```

La función quedará así:

```
1 export function crearTablero(filas, columnas, alHacerClickCelda,
2   alHacerClickDerechoCelda) {
3   const tablero = document.querySelector("#tablero");
4
5   tablero.innerHTML = "";
6   tablero.style.gridTemplateColumns = `repeat(${columnas}, 32px)`;
7
8   for (let fila = 0; fila < filas; fila++) {
9     for (let columna = 0; columna < columnas; columna++) {
10
11       const celda = document.createElement("div");
12
13       celda.classList.add("celda");
14       celda.dataset.fila = fila;
15       celda.dataset.columna = columna;
16
17       celda.addEventListener("click", () => {
18
19         const f = Number(celda.dataset.fila);
```

```
20     const c = Number(celda.dataset.columna);
21
22     alHacerClickCelda(f, c);
23
24 });
25
26 celda.addEventListener("contextmenu", (event) => {
27
28     event.preventDefault();
29
30     const f = Number(celda.dataset.fila);
31     const c = Number(celda.dataset.columna);
32
33     alHacerClickDerechoCelda(f, c);
34
35 });
36
37 tablero.appendChild(celda);
38
39 }
40 }
41
42 }
```

Con esto el tablero se adapta visualmente a cada dificultad.

18.11 Refinar el flujo de arranque definitivo

Ahora ya podemos dejar el final de `main.js` como debe quedar en la versión terminada del tutorial.

En lugar de esto:

```
1 configurarBotonReinicio();
2 configurarMenuDificultad();
3 configurarNavegacion();
4 configurarDialogoPuntuacion();
5 configurarBotonBorrarPuntuaciones();
6 mostrarPanel("panel_inicio");
7 iniciarAplicacion();
```

lo dejaremos así:

```
1 configurarBotonReinicio();
2 configurarMenuDificultad();
3 configurarNavegacion();
4 configurarDialogoPuntuacion();
5 configurarBotonBorrarPuntuaciones();
6 prepararInterfazInicial();
```

```
7  mostrarPanel("panel_inicio");
```

Ahora el flujo es correcto:

- se registran todos los eventos
- se prepara la interfaz
- se muestra el panel de inicio
- todavía no hay partida en marcha
- la partida empieza cuando el usuario elige una dificultad o pulsa la carita

18.12 Código final de `main.js`

La versión final de `main.js`, tras este último ajuste del tutorial, debería quedar así:

```
1  import {
2    crearTablero,
3    crearMatriz,
4    colocarMinas,
5    calcularNumeros
6  } from "./board.js";
7
8  import {
9    guardarPuntuacion,
10   borrarPuntuaciones,
11   generarTablaPuntuaciones
12 } from "./scores.js";
13
14 let matrizJuego = [];
15 let partidaActiva = false;
16 let tiempo = 0;
17 let temporizador = null;
18 let dificultadActual = "easy";
19 let panelActual = "panel_inicio";
20
21 const dificultades = {
22   easy: {
23     filas: 8,
24     columnas: 8,
25     minas: 10,
26     titulo: "Partida fácil"
27   },
28   medium: {
29     filas: 12,
30     columnas: 12,
31     minas: 25,
32     titulo: "Partida normal"
33   },
34   hard: {
```

```
35     filas: 16,  
36     columnas: 16,  
37     minas: 40,  
38     titulo: "Partida difícil"  
39   }  
40 };  
41  
42 function iniciarAplicacion() {  
43   partidaActiva = true;  
44   reiniciarReloj();  
45   iniciarTemporizador();  
46  
47   const configuracion = dificultades[dificultadActual];  
48  
49   matrizJuego = crearMatriz(configuracion.filas, configuracion.columnas  
50   );  
51  
52   colocarMinas(matrizJuego, configuracion.minas);  
53  
54   calcularNumeros(matrizJuego);  
55  
56   crearTablero(  
57     configuracion.filas,  
58     configuracion.columnas,  
59     abrirCelda,  
60     alternarBandera  
61   );  
62  
63   actualizarContadorMinas();  
64   actualizarTituloPartida();  
65  
66 }  
67  
68 function abrirCelda(fila, columna) {  
69  
70   if (!partidaActiva) {  
71     return;  
72   }  
73  
74   const valor = matrizJuego[fila][columna];  
75   const celda = obtenerCeldaDOM(fila, columna);  
76  
77   if (!celda) {  
78     return;  
79   }  
80  
81   if (celda.classList.contains("abierta")) {  
82     return;  
83   }  
84 }
```



```
85
86     if (celda.classList.contains("bandera")) {
87         return;
88     }
89
90     celda.classList.add("abierta");
91
92     if (valor === -1) {
93         celda.textContent = "●";
94         partidaActiva = false;
95         detenerTemporizador();
96         mostrarTodasLasMinas();
97         return;
98     }
99
100    if (valor > 0) {
101        celda.textContent = valor;
102    } else {
103        celda.textContent = "";
104    }
105
106    for (let f = fila - 1; f <= fila + 1; f++) {
107        for (let c = columna - 1; c <= columna + 1; c++) {
108
109            if (f === fila && c === columna) {
110                continue;
111            }
112
113            if (
114                f >= 0 &&
115                f < matrizJuego.length &&
116                c >= 0 &&
117                c < matrizJuego[0].length
118            ) {
119                abrirCelda(f, c);
120            }
121        }
122    }
123
124
125    if (comprobarVictoria()) {
126        partidaActiva = false;
127        detenerTemporizador();
128        mostrarDialogoPuntuacion();
129    }
130
131 }
132
133 function alternarBandera(fila, columna) {
134
135     if (!partidaActiva) {
```

```
136     return;
137 }
138
139 const celda = obtenerCeldaDOM(fila, columna);
140
141 if (!celda) {
142     return;
143 }
144
145 if (celda.classList.contains("abierta")) {
146     return;
147 }
148
149 if (celda.classList.contains("bandera")) {
150     celda.classList.remove("bandera");
151     celda.textContent = "";
152 } else {
153     celda.classList.add("bandera");
154     celda.textContent = "I";
155 }
156
157 }
158
159 function mostrarTodasLasMinas() {
160
161     for (let fila = 0; fila < matrizJuego.length; fila++) {
162         for (let columna = 0; columna < matrizJuego[fila].length; columna
163             ++ ) {
164
165             if (matrizJuego[fila][columna] === -1) {
166
167                 const celda = obtenerCeldaDOM(fila, columna);
168
169                 if (celda) {
170                     celda.classList.add("abierta");
171                     celda.textContent = "●";
172                 }
173             }
174         }
175     }
176 }
177
178 }
179
180 function comprobarVictoria() {
181
182     for (let fila = 0; fila < matrizJuego.length; fila++) {
183         for (let columna = 0; columna < matrizJuego[fila].length; columna
184             ++ ) {
```

```
185     if (matrizJuego[filas][columna] !== -1) {
186
187         const celda = obtenerCeldaDOM(filas, columna);
188
189         if (celda && !celda.classList.contains("abierto")) {
190             return false;
191         }
192     }
193 }
194
195 }
196 }
197
198 return true;
199
200 }
201
202 function obtenerCeldaDOM(filas, columna) {
203
204     return document.querySelector(
205         `celda[data-filas="${filas}"][data-columna="${columna}"]`
206     );
207 }
208
209
210 function actualizarReloj() {
211
212     const reloj = document.querySelector("#reloj");
213
214     if (reloj) {
215         reloj.textContent = tiempo;
216     }
217 }
218
219
220 function iniciarTemporizador() {
221
222     detenerTemporizador();
223
224     temporizador = setInterval(() => {
225         tiempo++;
226         actualizarReloj();
227     }, 1000);
228
229 }
230
231 function detenerTemporizador() {
232
233     if (temporizador !== null) {
234         clearInterval(temporizador);
235         temporizador = null;
```

```
236     }
237
238 }
239
240 function reiniciarReloj() {
241     tiempo = 0;
242     actualizarReloj();
243 }
244
245 function actualizarContadorMinas() {
246
247     const contadorMinas = document.querySelector("#minas");
248     const configuracion = dificultades[dificultadActual];
249
250     if (contadorMinas) {
251         contadorMinas.textContent = configuracion.minas;
252     }
253
254 }
255
256 function actualizarTituloPartida() {
257
258     const titulo = document.querySelector("#tituloPartida");
259     const configuracion = dificultades[dificultadActual];
260
261     if (titulo) {
262         titulo.textContent = configuracion.titulo;
263     }
264
265 }
266
267 function calcularPuntos() {
268
269     const configuracion = dificultades[dificultadActual];
270     const totalCeldas = configuracion.filas * configuracion.columnas;
271
272     if (tiempo === 0) {
273         return 0;
274     }
275
276     return Math.floor((configuracion.minas * 1000) / tiempo + totalCeldas);
277
278 }
279
280 function mostrarDialogoPuntuacion() {
281
282     const dialogo = document.querySelector("#dialogoPuntuacion");
283     const inputNombre = document.querySelector("#nombreJugador");
284
285     if (!dialogo) {
```

```
286     return;
287 }
288
289 if (inputNombre) {
290     inputNombre.value = "";
291 }
292
293 dialogo.showModal();
294
295 }
296
297 function actualizarPanelPuntuaciones() {
298
299     const contenedor = document.querySelector("#tablaPuntuaciones");
300
301     if (contenedor) {
302         contenedor.innerHTML = generarTablaPuntuaciones();
303     }
304 }
305
306 function abandonarPartida() {
307
308     partidaActiva = false;
309     detenerTemporizador();
310     reiniciarReloj();
311 }
312
313 }
314
315 function mostrarPanel(idPanel) {
316
317     if (panelActual === "panel_partida" && idPanel !== "panel_partida") {
318         abandonarPartida();
319     }
320
321     const paneles = document.querySelectorAll(".panel");
322
323     paneles.forEach((panel) => {
324         panel.classList.add("hidden");
325     });
326
327     const panelActivo = document.querySelector(`#${idPanel}`);
328
329     if (panelActivo) {
330         panelActivo.classList.remove("hidden");
331         panelActual = idPanel;
332
333         if (idPanel === "panel_puntuaciones") {
334             actualizarPanelPuntuaciones();
335         }
336     }
```

```
337
338 }
339
340 function configurarBotonReinicio() {
341
342     const botonReinicio = document.querySelector("#carita");
343
344     if (botonReinicio) {
345         botonReinicio.addEventListener("click", () => {
346             mostrarPanel("panel_partida");
347             iniciarAplicacion();
348         });
349     }
350
351 }
352
353 function configurarMenuDificultad() {
354
355     const botones = document.querySelectorAll("[data-difficulty]");
356
357     botones.forEach((boton) => {
358         boton.addEventListener("click", () => {
359
360             const nuevaDificultad = boton.dataset.difficulty;
361
362             if (!nuevaDificultad) {
363                 return;
364             }
365
366             dificultadActual = nuevaDificultad;
367             iniciarAplicacion();
368
369         });
370     });
371
372 }
373
374 function configurarNavegacion() {
375
376     const botones = document.querySelectorAll("[data-panel]");
377
378     botones.forEach((boton) => {
379         boton.addEventListener("click", () => {
380
381             const idPanel = boton.dataset.panel;
382
383             if (!idPanel) {
384                 return;
385             }
386
387             mostrarPanel(idPanel);
```

```
388
389     });
390   });
391
392 }
393
394 function configurarDialogoPuntuacion() {
395
396   const formulario = document.querySelector("#formPuntuacion");
397   const dialogo = document.querySelector("#dialogoPuntuacion");
398   const inputNombre = document.querySelector("#nombreJugador");
399   const botonCancelar = document.querySelector("#cancelarGuardar");
400
401   if (botonCancelar && dialogo) {
402     botonCancelar.addEventListener("click", () => {
403       dialogo.close();
404     });
405   }
406
407   if (formulario && dialogo && inputNombre) {
408     formulario.addEventListener("submit", (event) => {
409       event.preventDefault();
410
411       const nombre = inputNombre.value.trim() || "Anónimo";
412
413       const puntuacion = {
414         nombre,
415         puntos: calcularPuntos(),
416         dificultad: dificultadActual,
417         tiempo
418       };
419
420       guardarPuntuacion(puntuacion);
421       actualizarPanelPuntuaciones();
422
423       dialogo.close();
424       mostrarPanel("panel_puntuaciones");
425     });
426   }
427 }
428
429
430 function configurarBotonBorrarPuntuaciones() {
431
432   const boton = document.querySelector("#borrarPuntuaciones");
433
434   if (boton) {
435     boton.addEventListener("click", () => {
436       borrarPuntuaciones();
437       actualizarPanelPuntuaciones();
438     });
439   }
440 }
```

```
439     }
440
441   }
442
443   function prepararInterfazInicial() {
444     partidaActiva = false;
445     detenerTemporizador();
446     reiniciarReloj();
447     actualizarContadorMinas();
448     actualizarTituloPartida();
449
450   }
451
452   configurarBotonReinicio();
453   configurarMenuDificultad();
454   configurarNavegacion();
455   configurarDialogoPuntuacion();
456   configurarBotonBorrarPuntuaciones();
457   prepararInterfazInicial();
458   mostrarPanel("panel_inicio");
```

18.13 Código final ajustado de board.js

El archivo `board.js` puede quedar así:

```
1  export function crearTablero(filas, columnas, alHacerClickCelda,
2    alHacerClickDerechoCelda) {
3
4    const tablero = document.querySelector("#tablero");
5
6    tablero.innerHTML = "";
7    tablero.style.gridTemplateColumns = `repeat(${columnas}, 32px)`;
8
9    for (let fila = 0; fila < filas; fila++) {
10
11      for (let columna = 0; columna < columnas; columna++) {
12
13        const celda = document.createElement("div");
14
15        celda.classList.add("celda");
16
17        celda.dataset.fila = fila;
18        celda.dataset.columna = columna;
19
20        celda.addEventListener("click", () => {
21
22          const f = Number(celda.dataset.fila);
23          const c = Number(celda.dataset.columna);
```



```
23
24     alHacerClickCelda(f, c);
25
26 });
27
28 celda.addEventListener("contextmenu", (event) => {
29
30     event.preventDefault();
31
32     const f = Number(celda.dataset.fila);
33     const c = Number(celda.dataset.columna);
34
35     alHacerClickDerechoCelda(f, c);
36
37 });
38
39 tablero.appendChild(celda);
40
41 }
42
43 }
44
45 }
46
47 export function crearMatriz(filas, columnas) {
48
49     const matriz = [];
50
51     for (let fila = 0; fila < filas; fila++) {
52
53         const filaActual = [];
54
55         for (let columna = 0; columna < columnas; columna++) {
56             filaActual.push(0);
57         }
58
59         matriz.push(filaActual);
60
61     }
62
63     return matriz;
64
65 }
66
67 export function colocarMinas(matriz, cantidadMinas) {
68
69     let minasColocadas = 0;
70
71     while (minasColocadas < cantidadMinas) {
72
73         const fila = Math.floor(Math.random() * matriz.length);
```

```
74     const columna = Math.floor(Math.random() * matriz[0].length);
75
76     if (matriz[fila][columna] !== -1) {
77         matriz[fila][columna] = -1;
78         minasColocadas++;
79     }
80
81 }
82
83 }
84
85 export function contarMinasVecinas(matriz, fila, columna) {
86
87     let total = 0;
88
89     for (let f = fila - 1; f <= fila + 1; f++) {
90         for (let c = columna - 1; c <= columna + 1; c++) {
91
92             if (f === fila && c === columna) {
93                 continue;
94             }
95
96             if (
97                 f >= 0 &&
98                 f < matriz.length &&
99                 c >= 0 &&
100                 c < matriz[0].length
101             ) {
102                 if (matriz[f][c] === -1) {
103                     total++;
104                 }
105             }
106
107         }
108     }
109
110     return total;
111 }
112
113
114 export function calcularNumeros(matriz) {
115
116     for (let fila = 0; fila < matriz.length; fila++) {
117         for (let columna = 0; columna < matriz[fila].length; columna++) {
118
119             if (matriz[fila][columna] !== -1) {
120                 matriz[fila][columna] = contarMinasVecinas(matriz, fila,
121                     columna);
122             }
123         }
124     }
125 }
```

```
124     }  
125  
126 }
```

18.14 Qué hemos conseguido al finalizar el tutorial

Con este último capítulo hemos cerrado el proyecto de forma coherente.

Ahora la aplicación:

- arranca mostrando el panel de inicio
- no inicia partidas automáticamente en segundo plano
- comienza una partida solo cuando el usuario lo decide
- navega correctamente entre paneles
- detiene la partida al salir del panel de juego
- muestra un tablero adaptado visualmente a cada dificultad
- mantiene una interfaz mucho más limpia y lógica

Con esto damos por terminado el tutorial guiado del Buscaminas. El alumnado ha recorrido un proceso completo en el que ha aprendido:

- JavaScript moderno en el navegador
- manipulación del DOM
- eventos
- matrices y algoritmos
- temporizadores
- almacenamiento en `localStorage`
- modularización del código
- navegación entre paneles
- desarrollo completo de una pequeña aplicación web interactiva

El resultado es ya un proyecto final plenamente funcional y suficientemente cercano a una aplicación real como para servir de solución modelo y de base para futuras ampliaciones.